

Designing AI Chip

Hardware and Software

Bjarke Hammersholt Rouné
2026

What is this document?

I was going to start an AI chip company and this document contains my thinking about how to make an AI chip, both software and hardware. It's written for someone else who would be responsible for or work on an AI chip project - it is some of what you'd need to know.

I thought others might be interested in understanding the area of AI chips, as well, so most of this should be easily understandable at a hobbyist level. Some parts are more technical and require a deeper background knowledge - those are marked in bold as "for nerds".

This document also contains anecdotes from my time in Silicon Valley that I hope will be entertaining and perhaps illuminating. They are clearly marked and so can be skipped if that doesn't interest you.

I know that this document is now being used as part of AI chip company employee onboarding in some places, which I think is a good idea, since much of the material in this document is not easily available elsewhere.

Who am I to write this

I have worked at Google on TPUs and at Nvidia on GPUs in positions of technical leadership and also at other companies that produce AI software and hardware like Amazon and Facebook. I was the technical software lead for TPUv3 at Google and had much involvement in choosing and designing features for the TPU hardware back then. This is my profession.

Why did I write this

I eventually decided not to start an AI chip company for personal reasons. Part of my motivation for starting an AI chip company was that I believe that there is a better way to do things - to the benefit of investors, perhaps, but also to the benefit of customers. I hope to achieve some of the positive effects that I had hoped my company would achieve by instead writing this document and making it freely available.

You can comment

Feel free to comment on this document to me directly or in public [on this LinkedIn post](#) (or, if you wish, anywhere). You may also comment directly in this document using Google Docs comments, but you'll have to ask me for access to do that first. This was initially open to all, but that turned out to be unworkable since people would make unintentional edit suggestions or select parts of or the whole document, causing it to be highlighted for every reader.

Executive summary

My primary contention in this document is that the eventual future of AI accelerators will be AI CPUs - traditional-ish CPUs with big caches and systolic arrays and where the caches and systolic arrays take up most of the chip area and power. There are *many* details to get right to get a competitive AI chip from that idea and going over those one by one is a big part of the content in this document. Though this document is not specific to that idea - the underlying factors covered in this document are the same regardless of which design you prefer.

The biggest tension you'll find in this document is between, on one side, maximizing tokens per dollar and, on the other side, grossly overprovisioning your AI chip with collectively extremely expensive and unnecessary features and capacities so that customers will receive something that resembles what they are already using and used to. The recommendations I'm giving in this document assume that you have a most excellent software team that can make this all work so that customers can use your chip easily even without gross overprovisioning. *You will absolutely not have an easy time hiring a software team that can do this.* So keep that in mind.

If you are working on AI software, you may also want to read [this post on AI compiler testing](#). Since I already covered testing in that document, I will not get into testing here in this document. Unrelated: I do have some ideas for [interpretable transformers](#) that I think are interesting.

Will I work for your company?

I'm retired and not looking for a job. I do not do paid consulting. I will not run your startup and I'm not interested in receiving funding. I can't find a founder, cofounder or employees for you. But I do usually answer questions (for free) on LinkedIn, be it in private DMs or comments.

Proposal for a hardware design (summary).....	4
Systolic arrays - the foundation of AI hardware.....	5
Larger systolic arrays are more efficient.....	7
Forces limiting the size of systolic arrays.....	11
Numerics or how to make a cheap systolic array.....	14
Structured sparsity for systolic arrays.....	18
Mono-sized systolic arrays are unbalanced.....	21
Chips with larger systolic arrays are easier to make.....	24
Advanced section: Non-square systolic arrays.....	26
The tyranny of the cycles and systolic array DMAs.....	31
Anecdote: Systolic arrays are hot.....	33
Compression for activations and weights.....	36
Kinds of AI workload - training, decode, prefill.....	38
Training.....	39
Decode.....	42
Prefill.....	47
Decode-prefill (managed) aggregation vs disaggregation.....	48

Expensive HBM storage capacity.....	50
Weights and KV caches as reasons for large expensive HBM memories.....	50
Reducing the need for large memories on AI chips.....	54
Parallelization of an AI assistant.....	59
Parallelization concepts.....	60
Parallelization using duplication.....	62
Between layers parallelization of decode using pipelining.....	63
Between layers parallelization of prefill using pipelining.....	67
Within layers parallelization.....	68
Mixture of Experts (MoE).....	72
Network topologies for AI in a MoE world.....	75
Networking concepts.....	76
Link types.....	83
The specter of multi-hop waste.....	86
The big 3: Reduce-scatter, all-gather and all-reduce.....	89
Mixture of experts versus network topology.....	92
A proposal for a networking approach.....	93
Co-design.....	100
My experience with co-design.....	101
Intellectual quality and good judgement.....	103
Also learn from the world outside your company.....	108
Co-design needs to be grounded in the product.....	109
Combinatorial search.....	110
A critical look at GPU features for AI.....	112
Kernels and warp specialization.....	112
Path divergence (vector subsets).....	114
Software managed cache.....	114
Warp scheduling (memory access pipelining).....	115
Needing a host computer.....	116
The need for many threads and kernel switching overhead.....	117
Software pipelining for AI kernels.....	119
What is (software) pipelining.....	119
Software pipelining is difficult by hand.....	121
Write a tiled software pipelining library with op fusion.....	124
Why not X, Y or Z?.....	129
Software pipelining on Nvidia GPUs aka Warp Scheduling.....	130
Hiring for your AI chip project.....	132
Sourcing candidates over email.....	132
Interviewing engineers.....	136
Talking to engineers.....	137
Objections and answers.....	142

OS interruption of the systolic arrays.....	142
Few have access to the kind of CPU required for this.....	142
The science is missing.....	144
This design may not be so generally applicable.....	144
Tokens per dollar isn't everything.....	145
What about analog compute?.....	146
Why more FLOPs?.....	146
Structured sparsity sucks.....	147
Systolic arrays are bad for depthwise convolution.....	150
Are routers superior to tori?.....	151
What about existing AI CPUs?.....	151

Proposal for a hardware design (summary)

Here's a summary of my proposed design. I recommend reading this section last, but you can read it up front if you wish. The defining features are that this design is easy to do in hardware, as easy to program as a CPU and is very cheap on a computation / watts basis. I doubt anyone making an AI chip will simply adopt my plan as-is, but I would suggest going through this design and understanding, for each deviation, exactly why you think deviating from this is a good idea. I would surely also have modified the design over time if I had embarked on actually building this.

"Hold on, this is just a CPU with a systolic array attached, you mean to say you can outcompete sophisticated GPUs and AI ASICs with just this?" Yes, that's what I believe. Most of this chip is cache and systolic array, so you do not have to deviate from a CPU approach for scalar and vector compute. CPUs are the most programmable thing and both hardware and software engineers already know how to build and use them. That's the idea. You'll need to read the whole document to see why this works, but here are some of the highlights of the design:

- Design a traditional CPU with extra vector compute (and registers) and attach a systolic array to each core. If your name is Intel or AMD, probably use x86, otherwise find something fast and cheap (to you) that you can make or license.
- Have a large software managed SRAM cache (MBs) that can potentially double as a traditional automatic cache if part of it is unused by a given program.
- Support DMAs between all memories and networks. Do SRAM->network->SRAM DMAs. Offer HW 8-bit huffman compression/decompression for DMAs at HBM / link speed.
- No host computer required - your chip is a general-purpose CPU already. Ensure your chip can run Linux and TensorFlow / PyTorch. Add scalar cores with no systolic arrays.
- Do not supply a large amount of HBM for your chip. Fit weights by using multiple chips.
- Supply an SSD with your chip. Enable KV-cache to go here by using sparse attention.
- Systolic arrays support only 7 bit multiplies, not 16 bit. Sums are wider and saturate.
- Systolic arrays support 1-of-2 sparsity for the RHS - simple and allows 2x throughput.
- Primary network is a 1D or 2D torus of copper traces. Use (cheap) custom motherboards that snap onto each other in 1D or 2D, connecting the traces. No expensive cables.

- Write an XLA backend for your chip. Most users will use this device through XLA. Also support custom kernels and custom XLA ops written in standard C++ (with intrinsics).
- This design works also for training, but focuses on inference at first - that's easier.

This avoids the expensive parts of AI chips that you don't really need, it's straightforward to make this hardware in RTL and you can program it like any other CPU. I don't know why something like this isn't the industry standard for AI chips. The industry seems to prefer pursuing novel non-CPU architectures instead.

This chip requires significant software sophistication to use well, and you'll have to do some AI research to get sparse (indexed) attention working well along with 1:2 sparsity and, in the low-HBM configuration suggested here, it requires a large minimum installation size to run a large LLM. It is an option to modify the design to be less economical to get around these factors, which I think is still a good design, but then it will be more expensive. I cannot predict which choice there will optimize your profit, but what's suggested here is what is best in terms of delivering maximum tokens per dollar at high volumes while still being easy to program.

[[[Technical note on Groq's LPU for nerds Groq's LPU design doesn't require any HBM. Is this the same? It's the same in terms of distributing weights to avoid needing large per-chip memory capacity for weights, leading to a matching requirement of a large installation size. However, it is not the same for dealing with KV cache requirements. Groq uses a low-batch (and therefore low token latency) approach for decode (output tokens), which drops KV cache requirements dramatically - that's how they don't need any HBM to store the KV cache. This means Groq's chips do not (cannot) use large systolic arrays (which require higher batch), which necessarily leads to worse power and area efficiency. Groq's approach leads to very high memory bandwidth requirements, which the LPU has since it uses SRAM in place of HBM. The design I'm proposing here instead uses indexed attention (which leads to much lower memory BW requirements) to enable offloading the KV cache to SSD, so large batch and therefore large systolic arrays can still be used. But then you have to make sparse (indexed) attention work - if you (and your customers) can't figure that one out, you'll have to add more HBM instead, as indeed most AI chips do. In short, Groq's low-batch approach is for expensive ("premium") low-latency tokens while the approach I'm suggesting is for economical medium-latency tokens. I believe that Groq's approach is valuable, but only for a tiny super rich niche of the market, and what I'm proposing addresses the needs of broad humanity and most customers.]]]

Systolic arrays - the foundation of AI hardware

[Systolic arrays](#) are the foundation of AI hardware (i.e. deep learning hardware). Why?

Almost all mathematical operations (OPs) in a deep learning model like a transformer are happening inside matrix multiplication. Large systolic arrays are the most efficient way to implement matrix multiplication in hardware. So, systolic arrays are the foundation of AI hardware.

An AI accelerator chip like an Nvidia GPU (e.g. Blackwell) or a Google TPU is then a way to access a systolic array. There are more elements on an AI chip than systolic arrays, but the systolic arrays are the most important element, like the engine of a car.

This document is not an explanation of what a systolic array is, or how they can be used to do matrix multiplication - I assume that you know that already, but you can get an AI assistant like Gemini to explain it to you if you haven't heard of them before.

If tomorrow a better kind of AI is developed that doesn't rely on matrix multiplication, then the conclusions in this document may all have to change. This doesn't seem to be the way things are going, since matrix multiplication has been king for a long time now, but it's a risk for any AI hardware chip producer that their current AI chip may not be useful tomorrow depending on what happens in AI research. There is no way around that situation. It was true ten years ago and it is true today.

AI chips like Nvidia GPUs and Google TPUs have a number of large systolic arrays on them. If an AI chip is made properly, then most of the electricity consumed by AI is going into powering those systolic arrays. So when you hear of AI using a lot of electricity, that's actually systolic arrays using a lot of electricity, at least for properly designed AI hardware. This is not because systolic arrays are power inefficient - electricity requirements would be much higher without systolic arrays.

It is the systolic array that defines the upper limit of how fast and power efficient an AI chip can be. Indeed, when we talk about *utilization* of an AI chip, which should ideally be near 100%, we are mostly talking about how busy the systolic array is. The rest of the chip exists to keep the systolic array busy.

On an Nvidia GPU, the systolic arrays are inside what Nvidia calls a [TensorCore](#). Google uses the term [Matrix Multiply Unit \(MXU\)](#) to refer to their systolic arrays (you might notice that "multiply" doesn't have an X in it, but x is sometimes used to mean multiplication, as in $2 \times 3 = 6$, so they call it MXU with an X). AMD calls their systolic arrays [Matrix Cores](#). Intel calls the interface to their systolic arrays [Advanced Matrix Extensions \(AMX\)](#). On Amazon Inferentia/Trainium the systolic arrays are called [NeuronCores](#). It may sound like there is a wealth of different AI chip approaches, and there of course *are* differences, but it's all systolic arrays with some other stuff holding it together. It's all systolic arrays. The brand names are for marketing. There *are* AI chips that don't contain systolic arrays, like very old Nvidia GPUs. Those are not ideal for most AI applications compared to chips that do contain systolic arrays, due to poor power and area efficiency.

So next time you hear of an AI chip with Northern Lights-Speed Einstein Genius Cores, or Newton AppleFall Eureka Cores or *whatever* they are going to call them, just remember that they are talking about systolic arrays. It's all systolic arrays. For your reference, systolic arrays were [invented in 1978](#). We are now seeing AI chips coming out with large systolic arrays in recent times because AI has created demand for such a thing. It's not an advanced modern

thing, it's old technology with now, all of a sudden, a large customer base. If Deep Learning had been invented in 1978, there would have been a rush of systolic array chips, I mean AI chips, back then, too. It would have been the same thing. It's not something sophisticatedly modern.

You might say that high memory and network bandwidths are also important for AI, so it isn't just about systolic arrays. Yes, but why do we need high bandwidth for AI? It's because systolic arrays are so efficient that you need high bandwidth to feed them - otherwise they will be waiting idle for more work to do. Without systolic arrays, the chips would be much slower and you wouldn't need nearly as much bandwidth per chip. The AI revolution is carried on the shoulders of the cost effectiveness of systolic arrays.

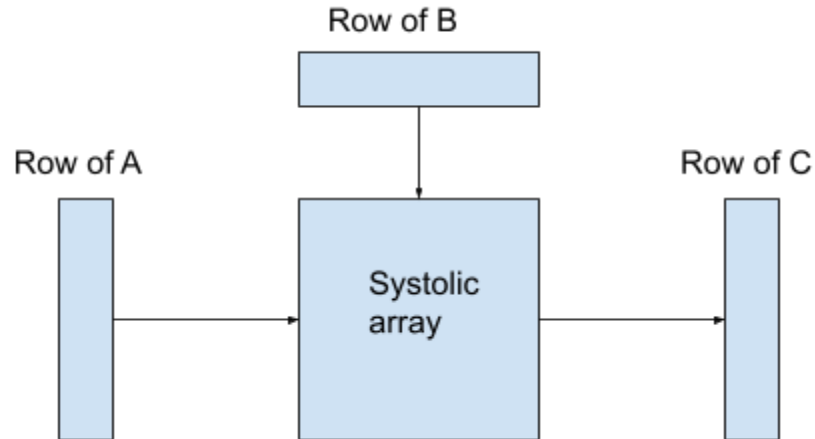
So now you know The engine of AI chips is systolic arrays and every company calls their particular systolic array chip something different.

I'm going to focus on Nvidia GPUs and Google TPUs for the rest of this document because I worked on both of these at Google and Nvidia, so I know them well, and also because these are the top two AI chip products in the industry as of 2026. There are many other systolic array chips, I mean AI chips, though.

[[[Exceptions for nerds There are a few exceptions. Groq's LPU doesn't have large systolic arrays, and maybe no systolic arrays at all. The LPU is for low token latency, not economical AI. Nvidia licensed Groq technology and this is exactly what Jensen (Nvidia's CEO) said at GTC 2026 - it's for expensive and low latency tokens, not general economical AI. There are also analog chips, using light or electricity, that try to achieve economies in other ways - these are unproven, but possibly something that will matter in the future if they will be proven. **]]]**

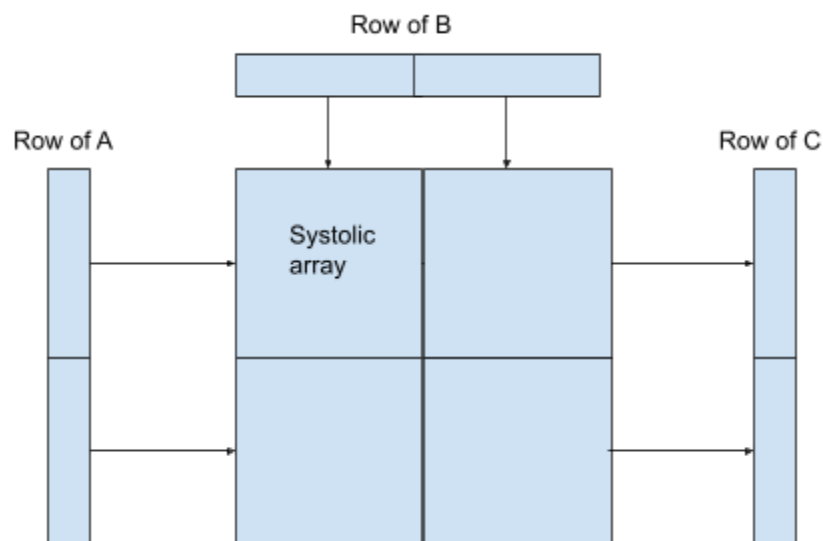
Larger systolic arrays are more efficient

This section is one of the only sections in this document that has a little bit of math in it. So, to make up for that, it also has some (badly drawn) pictures. Suppose you want to multiply two $N \times N$ matrices A and B to create the matrix product $C = A * B$. Then you can visualize a systolic array like this:

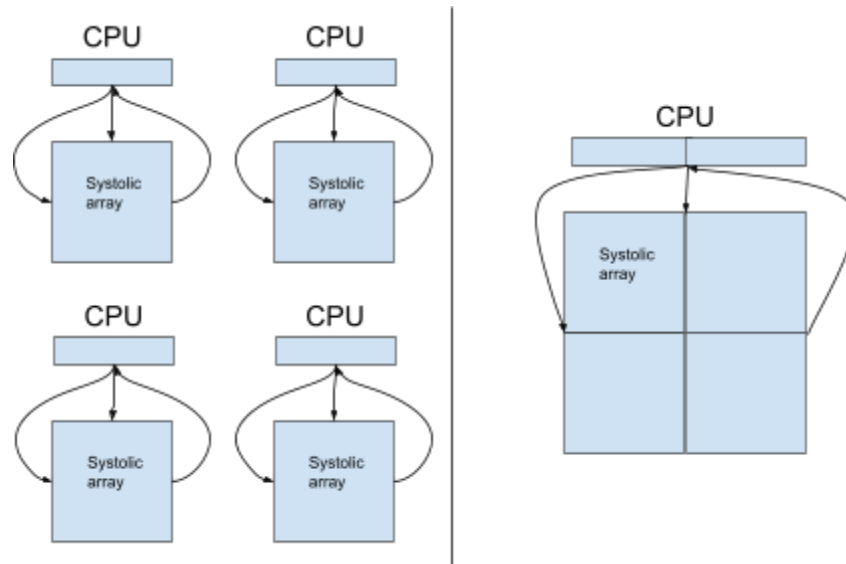


On every cycle, this systolic array takes in a vector from the left and a vector from the top and outputs a vector to the right. You can tell that this diagram is simplified, since it requires knowledge of the entire matrix B to produce a row of C, so you can't produce any row of C until you see all of B. That's why, in reality, the whole matrix B is fed into the systolic array and stored there and only then does it start to process rows of A. However, if you are doing many separate matrix multiplications one after the other, you can feed in the next B matrix while processing the current A matrix and outputting the prior C matrix, so in this way the left, top and right of the systolic array can be busy on every cycle, which is called 100% utilization. 100% is good. Systolic arrays also use something called skew/stagger, that complicates the picture further, but I'll leave that out of the discussion since it isn't relevant for this section.

The vector width of an $N \times N$ systolic array is N . What happens when we double the vector width from N to $2N$, creating a $2N \times 2N$ systolic array instead of an $N \times N$ one? It will look something like this:



The vectors are twice as wide and the systolic array is 4 times as big, so it can do 4 times as much math per cycle. To see what we have gained by doubling the vector width, we will have to expand the picture to include the CPU (or CPU-like structure like a GPU) that is associated with each systolic array and which tells the systolic array what to do and processes the rows (i.e. vectors) from A, B and C. You can see that in the diagram below.



So what have we gained by using this larger systolic array? Our larger $2N \times 2N$ systolic array is doing 4 times as much math per cycle as before, so the throughput-equivalent structure using smaller $N \times N$ systolic arrays will require 4 systolic arrays. It will also require 4 extra CPU cores, to tell the 4 systolic arrays what to do and to process their vectors. So by using a $2N \times 2N$ systolic array instead of a $n N \times N$ one, we have reduced the number of CPU cores/work by a factor of 4. However, the CPU on the right has to process vectors that are twice as wide, which makes it more expensive, but less than twice as expensive. So by going from the left picture to the right one, we have shrunk the rest of the chip by a factor in excess of 2. Yet we didn't lose any matrix multiplication capacity when we did this. So we have gained a factor of 2 on efficiency on the rest of the chip by doubling the vector width. Do it again, and you get a factor of 4. Do it again, and you get a factor of 8 and so on.

Couldn't you just use the same CPU core to drive all 4 systolic arrays on the left picture? You could, but then it needs to be 4 times as capable / fast, so that doesn't change the overall math.

These diagrams explain why we are seeing a march towards ever-larger systolic arrays for AI. Nvidia GPUs gained 4×4 systolic arrays in Volta (2017), got 16×16 systolic arrays in Turing (2018) and 64×64 systolic arrays in Blackwell (2025). Google TPUs have always used 128×128 or 256×256 systolic array sizes.

You might think this picture doesn't apply to GPUs because I put a systolic array and a CPU core on the diagram, but there isn't anything called a GPU on the diagram. For the purposes of

this particular point, a GPU is just a specialized form of a CPU, one that has a lot of vector compute, so this picture applies perfectly well to GPUs. Read “GPU” where it says “CPU” in the diagram and you will get the right idea.

Here’s a mystery to ponder Nvidia was already a very large and profitable company back in 2017 and GPUs were and are its main product. Meanwhile, while Google is also a very large company, the TPuv2 team within Google was small, something like 30 people in total across both hardware and software (5 people on the XLA team). Yet Google TPUs have continued to compete well with Nvidia GPUs from that time onwards. How is this possible? The answer will have in large part to do with the size of systolic arrays, I believe.

Let’s compare Volta to TPuv3 in terms of systolic array size. Volta had 4x4 systolic arrays while TPuv3 had 128x128 systolic arrays. To see what effect that has, let’s look at scalar compute and vector compute separately.

Scalar compute Scalar compute includes the processing to tell the rest of an AI accelerator what it should do. We saw that a doubling in vector width gives a factor of 4 reduction in scalar work to do matrix multiplication. To go from 4 to 128 is 5 doublings of vector width, so the benefit for scalar compute of going from a 4 x 4 systolic array to a 128 x 128 systolic array is:

$$4^5 = 4 * 4 * 4 * 4 * 4 = 1024x$$

An AI chip also has to use scalar compute for doing things other than matrix multiplication, so this isn’t the whole story, but it’s a big part of the story. The TPU had to do only 1/1024th as much scalar work to drive the systolic arrays to do matrix multiplication (it’s actually even more than this since TPU vector registers are 8x128 rather than just 128, so that’s $8 * 1024 = 8192x$). The scalar work is not the *main* cost on an AI chip, but it isn’t completely trivial and a factor of 1024 cannot be ignored.

Vector compute We saw that a doubling in vector width gives a factor of 2 reduction in vector work outside of the systolic array. With 5 doublings from 4 to 128, that becomes a factor of:

$$2^5 = 2 * 2 * 2 * 2 * 2 = 32x$$

This is very significant. You might argue that what happens inside a systolic array is itself a kind of vector work, so we haven’t eliminated this much vector work, we just moved it inside the systolic array instead of outside. That’s technically true, but a systolic array is much more efficient than a vector processor, including for the work to retrieve inputs and store results, so what has been achieved is to move this work from a less efficient place to a much more efficient place. So the net effect is not so much as 32x but it is still very significant.

If you ask a great hardware engineer to explain this more fully, they are going to give you a long explanation involving register files, picojoules and the efficiency of data transfers around a chip,

perhaps also including a digression into various alternative designs, but the overall conclusion is that much larger systolic arrays are much more efficient.

So you can start to see how a small team at Google could keep up with the might of Nvidia. There were other reasons, as well, but this is a big one. Nvidia GPUs have larger systolic arrays now, but they are still not as large as the ones on Google TPUs.

The obvious question here is then: why doesn't everyone use even larger systolic arrays, like 1024×1024 ? That should be even more efficient, right? That is the topic of the next section.

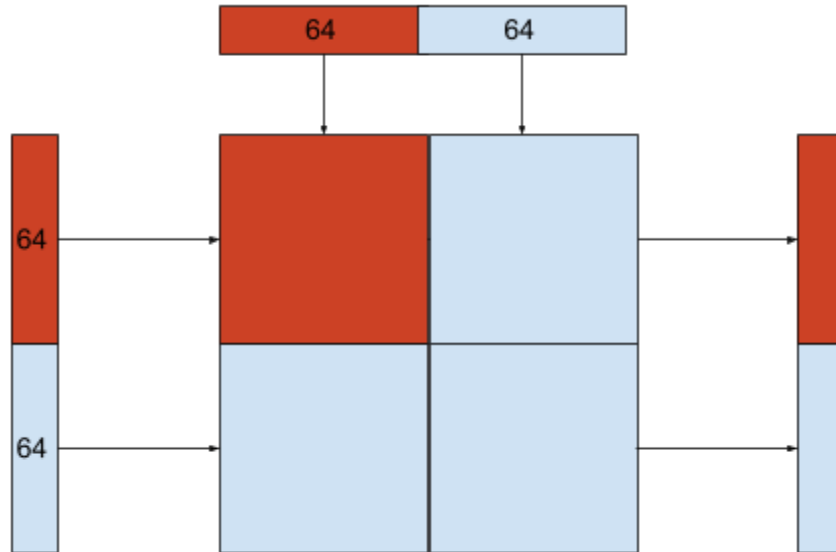
Forces limiting the size of systolic arrays

From our discussion so far, it appears that the most efficient AI chip will consist entirely of one very large systolic array (this is almost what TPUv1 was). The more of the area of a chip that consists of one (or several) large systolic array(s), the more efficient your chip will be. All other things being equal. So why not fill the entire chip with one large systolic array?

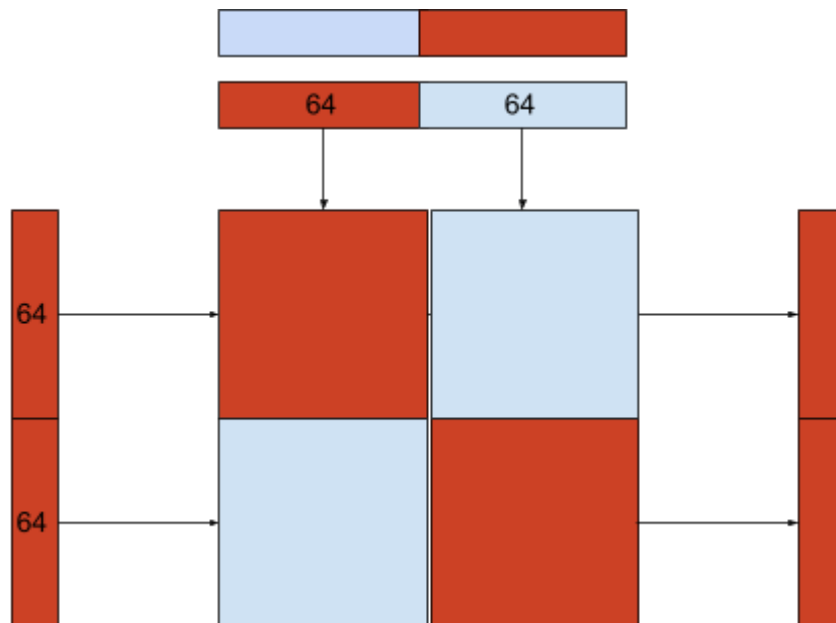
The most obvious reason is that the rest of the chip has to do *some* scalar and vector work, and you also need some space for caches, memory controllers and so on, so the *entire* chip can't be just one big systolic array. In fact these other elements are a significant fraction of an AI chip, though this will be less and less the case the larger the systolic arrays get.

[[[Technical detail for nerds TPUv1 was mostly one big systolic array. This requires driving it from a separate, external CPU, since there wasn't a CPU-like element on the chip. You can do it that way, but now you have to buy a CPU to drive the systolic array(s), so arguably you didn't save any chip area, you just put that area onto another chip that you still have to buy. **]]]**

There is another more restrictive problem with overly large systolic arrays for AI. An $N \times N$ systolic array cannot *efficiently* do a matrix multiplication that is smaller than $N \times N$. If the vectors in your AI model are 64 elements wide, and you have a 128×128 systolic array, your systolic array will not be used efficiently. Here is a diagram to visualize that. In the diagram, vectors are 128 elements wide but only the first 64 elements, in red, are actually being used:



You can see that only $1/4 = 25\%$ of the systolic array is being used. This is called 25% utilization, which is very bad. Thankfully, there is a way to improve on that picture, though it requires some complication in the software that drives the systolic array:



The idea is that the upper left and lower right subsets of the systolic array can be used independently, so that way 50% of the systolic array can be used. This is a consequence of the math of matrix multiplication, so you don't need a hardware feature to enable this optimization. This way, the systolic array can be 50% utilized. But 50% utilization is still bad. We want 100%.

So we can see that every time the systolic array is twice as big as the vector width of the AI model that it runs on, then efficiency is decreased by a factor of 2 (or 4, if the software does not

use the optimization mentioned above). This problem is the main reason that AI chips commonly contain many smaller systolic arrays instead of one huge one.

So the conclusion is that it is the dimensions of matrix multiplications in AI models that limits the size of systolic arrays in AI hardware. If your customer is using a small vector width in their AI models, you can't sell them a chip with a large vector width, not unless they change their AI.

[[[Three dimensions for nerds Matrix multiplication involves 3 dimensions instead of one. The diagram above is showing what happens if the middle one is smaller than the systolic array. Similar problems happen if the left or right dimensions are too small, though for those dimensions there is often some clever way to find more data to multiply so that you in effect increase those dimensions to reach 100% utilization. For example, the batch dimension in AI models serves this purpose. For the middle dimension, that we are focusing on here, "finding more data" doesn't help: if the dot products inside the matrix multiplication only sum up so many items, then that is a hard limitation that there isn't much you can do about. Except the optimization shown above. So that is why in this section I focus on the middle dimension, which corresponds to the vector width - it is the more fundamental limit.]]]

The vector width of AI models is the way it is for a reason. If your model has a certain number of parameters, then there is a specific vector width that best makes use of those parameters to create a clever AI. You can use a higher vector width than that, but then your AI will be dumber if given the same number of parameters.

Following industry rules of thumb, larger LLMs have larger vector widths, so that vector widths and therefore systolic arrays are going to keep growing for as long as LLMs keep growing. Which seems to be a trend that will continue.

Let us focus on the transformer architecture, which is the driver of the current LLM AI revolution. Transformers have two different kinds of matrix multiplications: attention and feed-forward (FF). I won't explain transformers here, but do study them if you want to know what's going on in AI. Attention matrix multiplications are, unfortunately, not that large. FF matrix multiplications can be very large. So it is the attention component of transformers that are the most limiting for the size of systolic arrays on AI chips. Otherwise we might be seeing 1024 x 1024 systolic arrays already now, not 64 x 64 (Nvidia) and 256 x 256 (Google).

[[[Technical details for nerds The full picture here is quite a bit more complex than this simplified view - isn't attention often reported to be waiting on memory more than it is waiting on the systolic array? So that sounds like it might be acceptable to get low utilization out of a systolic array during attention, since it doesn't need to be running very efficiently to keep up with memory when doing attention. That's *sometimes* true, but there are still limits to how inefficient this can be. This also gets into the difference between training, prefill and decode, where memory bandwidth demands during training and prefill are far lower than during decode, so for those you do need high utilization of the systolic arrays - see the later section on this. Techniques like speculative decoding and GQA make even decode much less memory

intensive, too (see the section on decode for more on this). Also, if you do it right, you can overlap attention and FF (requires deeper pipelines or, more efficiently, a model change to do them in parallel), in which case you do want high utilization of the systolic arrays always. Sparse attention, especially indexed attention, can also dramatically lower the bandwidth requirements of attention, and, in my opinion, all attention should be sparse (indexed) attention. There are also *many* techniques for making the KV cache smaller, and therefore lowering decode bandwidth requirements, that I don't even mention in this document. The simplified conclusion from all that is what it says above: FF can use very large systolic arrays, while attention, if done right, works better with smaller systolic arrays. So it really *ought* to be transformer attention matrix multiplications that are limiting the size of systolic arrays on AI chips, though it is true that in practice, if you or your customers don't put in the effort to make that happen, you may at times be limited on something else. **]]]**

It might be surprising to hear that attention matrix multiplications are not that large, given that LLMs spend a lot of time doing attention. Shouldn't attention be done quickly if it only uses a small matrix multiplication? The explanation is that the terminology so far has been a bit simplified. A matrix multiplication is of an $N \times K$ matrix times a $K \times M$ matrix, yielding an $N \times M$ result. So there are *three* independent dimensions, not just one N . Attention matrix multiplications are small in the sense that the K dimension is small, but the other two dimensions are often large and so the total computation is also large. The K dimension defines how many numbers are summed together to create one element of the resulting output matrix. If K is small, then a systolic array that is dimensioned to sum more numbers than that cannot get full utilization and this gets worse the larger the systolic array is compared to K . Even though K is small for attention, N and M can be large and also attention needs to do many matrix multiplications (if there are many attention heads), so that is how attention can take a long time even though K is small. In FF, all three dimensions, N , K and M , can usually be large enough to fill a systolic array.

[[[Detail on images and video and video for nerds If your AI is processing images or video, note that each pixel is only 3 values deep: red, green and blue. That can be a problem because this means that the matrix multiplications for processing an image has a K of only 3. That's extremely small, much smaller than attention. However, this isn't so bad as it may seem. As soon as AI starts to look at the data, we can and do immediately create a much larger K , so it's only the very initial processing where you have $K=3$. Also, that initial processing might be possible to place on a CPU before you hand the data to the AI chip, since it is light weight (due to $K=3$), in which case you might still get 100% utilization of your AI chip during image and video processing even for a large systolic array. **]]]**

[[[Detail on decomposable systolic arrays for nerds Etched's Sohu chip uses a *decomposable* systolic array, which can be configured in hardware as either many independent systolic arrays for attention or one large systolic array for FF. This is not as complete of a solution to the problem as it may at first appear. Given that the chip is capable of running in a mode with many decomposed small systolic arrays, the expensive on-chip bandwidth and scalar/vector compute required to service that mode has to be present on the chip. So you do

not get the chip area savings that a large systolic array would normally give you, in fact this approach instead costs additional area because now you also need chip area to implement the additional feature of the combined systolic array. There might still be power savings to be had from this approach, though. **]]]**

[[[Detail on batch, token latency and KV cache for nerds Larger systolic arrays can also lead to a need for a larger batch dimension, which is separate from the other issues pointed out in this section. Larger batch decreases memory bandwidth requirements but lead to increased token latency and KV cache storage. See the subsection on non-square systolic arrays for more on this. **]]]**

Numerics or how to make a cheap systolic array

Not all systolic arrays are made equal. We've seen that larger systolic arrays are more efficient. What else matters? Numerics. Numerics matter. But what does that mean?

A matrix multiplication consists of dot products. A dot product is a sum of products of numbers, e.g. the dot product of the vectors (a, b, c) and (x, y, z) is $a*x + b*y + c*z$. So a dot product involves N multiplications and $N - 1$ additions, but additions and multiplications of *what*? Well, of numbers, but on a computer, a number is not just a number. There are different kinds of numbers. That's what "numerics" refers to. But what are the kinds of numbers?

FP32 FP32 is what is called a "[float](#)" in C++. It is a 32 bit floating point number. The original breakthrough deep learning network, AlexNet, used FP32 for both additions and multiplications, using an Nvidia GPU. However, FP32 addition and especially multiplication are very expensive to implement on a computer chip in terms of chip area and power.

FP16 FP16 is a 16 bit floating point number. This was an innovation used on Nvidia GPUs where multiplications would be done in 16 bit, while additions are still done in 32 bits (the additions require higher precision than the multiplications). It turns out you can use a lower precision for the multiplications without degrading an AI at all, so this worked well. A systolic array using FP16 requires less than half of the area and power than one using FP32. Also, if you use FP16, you only require half as much memory bandwidth and memory capacity. So going from FP32 to FP16 was a big deal.

A drawback of FP16 is that it has a much lower dynamic range than FP32, i.e. it cannot represent as large a range of numbers. This means that when training an AI model with FP16, it is necessary to [use loss scaling](#). Loss scaling is not at all hard to do, but it does require modifying a model a little bit, so, because of this, FP16 is not a drop-in replacement for FP32 for training. For inference it *is* a drop-in replacement, since inference does not require loss scaling (losses are not calculated during AI inference). But this happened during a time where there was much AI research, but little AI use, so inference was not as important back then.

BF16 BF16 is also a 16 bit floating point number, but it preserves the dynamic range of FP32 at the cost of lower precision. It turns out that BF16 works just as well for AI as FP16 does, but it doesn't require loss scaling, so it *is* a drop-in replacement for FP16. In addition, BF16 requires even less power and area to implement in hardware than FP16 does.

BF16 was introduced by Google in TPUv2, which was a bit of a scoop over Nvidia GPUs at the time, which used FP16. Nvidia later added support for BF16 to their GPUs, due to the advantages of BF16, seemingly closing the gap. But is the gap really closed? No, not really, because Nvidia GPUs still support FP16. This means that Nvidia GPUs pay the higher chip area price for implementing FP16 while Google TPUs, to this day, don't have to do that since they don't support and don't need to support FP16.

FP8 FP8 refers to two different 8 bit floating point formats, namely E4M3 and E5M2. FP8 is not a drop-in replacement for FP32 during training or inference, though it can usually be used fairly easily during inference. FP8 is much cheaper to implement in a systolic array than BF16 is for multiplications. So Nvidia GPUs, and [now also Google TPUs](#), realize a 2x improvement in systolic array throughput when using FP8, though both still support BF16, which then runs at half speed. Also, FP8 of course takes up half as many bits as FP16, so you gain a factor of 2x on bandwidth, as well (or 4x compared to FP32).

BF16 marked a period of innovation advantage of Google over Nvidia on numerics and FP8 marked the end of that period. FP8 was invented as joint work between Nvidia, Arm and Intel and Nvidia introduced it to their GPUs *years* before Google introduced native FP8 support for TPUs. Google did have int8 support for TPUs somewhat earlier, but still behind.

FP4 FP4 is, as you might suspect at this point, a 4 bit floating point format. This is supported by Nvidia GPUs and possibly in the next version of Google TPUs¹. An FP4 systolic array *is* cheaper to implement than an FP8 systolic array, though since additions still have to be done in a much higher precision, and systolic arrays do have other overheads, the chip area and power advantage from using fewer bits is less and less - this benefit has diminishing returns. Still, Nvidia GPUs offer twice the systolic array throughput for FP4 than for FP8. The other big advantage of FP4, which is *not* subject to diminishing returns, is another doubling in effective memory bandwidth, since the number of bits to transfer is half as much as for FP8. Not all AI inference can use FP4, but some can and in that case it can be a significant optimization. FP4 for weights is challenging. FP4 for activations is *very* challenging.

[[[Integer summation for FP4 for nerds Multiplication is generally much more expensive than summation. However, as we use fewer and fewer bits for the multiplications in a systolic array, the summations start being a greater and greater proportion of the expense, as the summation bits do not reduce as much as the multiplication bits do. So for this reason, you can end up with situations like FP4 products being summed into FP32, as is done on Nvidia GPUs - at this point, the summation may be more expensive than the product. One of the benefits of integer types is

¹[This article](#) says a Google spokesperson said that the next TPU will support FP4, but [Google's docs](#) don't say anything about that, so I'm not sure if it's the case or not.

that integer summation is particularly cheap, and also you do not need as many bits for integer summation as you do for floating point summation. The product of two FP4's can be represented in an int8 using fixed point. So it might actually be cheaper and also more accurate to sum FP4 products in integer values instead of floating point. This does not apply to MXFP4, however, as it includes additional multipliers that make FP32 summation more fitting.]]]

Int8 Google TPUs and Nvidia GPUs also offer support for int8, which is an 8 bit signed integer. Floating point has some complications that are avoided by integers. Int8 works for all reasonable inference needs, though so does FP8, so if you've paid the hardware price for FP8, it's not clear why you'd need Int8. Still, the support is there.

MXFP8, MXFP6, MXFP4, NVFP4 These are additional formats supported by Nvidia GPUs that Google TPUs do not support. Nvidia has gone a bit crazy with supporting many different formats in their systolic arrays. These are complex formats that improve numerics somewhat, so that more inference workloads can use lower precisions. There is a modest bit cost to this, e.g. MXFP4 uses 4.25 bits per number instead of the 4 bits that FP4 uses. MXFP4 and NVFP4 are for models that *almost* work in 4 bits, but not quite - they might work with these formats. MXFP6 is what you can try if that didn't work - maybe 6 bits is enough. MXFP8 is the subject of the next nerd box. Nvidia has some customer lock-in benefits for supporting these formats, as these formats do not have as widespread support outside of Nvidia GPUs.

[[[The curious case of MXFP8 for nerds MXFP8 is a complex format that aims to improve upon FP8. However, if deployment (quantization) is done properly, then you will never need higher precision than 8 bits for inference (other than for accumulators, but that isn't what we're talking about here). In fact, 8 bits is more than you will ever need for inference. So why would MXFP8 exist? For two reasons. High-quality quantization is some trouble to do, so people sometimes use inferior approaches that might require MXFP8 to work. Also, people sometimes experiment with 8-bit training. You can do training in 8 bits, but it isn't easy and MXFP8 helps a bit with that.]]]

So both Google TPUs and Nvidia GPUs support many different formats in their systolic arrays. This has the benefit of being flexible for users, but it also leads to a very significant increase in hardware complexity inside the systolic arrays. This somewhat undermines the view of systolic arrays as simple and hyper efficient hardware structures. Not all systolic arrays are made equal. Nvidia is ahead of Google in terms of innovating in the area of numerics, and they have gained a large advantage from this. However, Nvidia now supports so many different formats that, surely, they must be paying a large price for it that Google doesn't have to pay as much. And even Google is paying what must be a significant price for the many formats TPUs do support.

TPUv1 was a pure Int8 systolic array (for multiplications). It received limited adoption within Google in part due to difficult software and in part because AI practitioners at the time did not commonly know how to prepare an AI model to run in 8 bits. That was a long time ago and this is a standard and easy thing to do now. Perhaps the market is now ready for a revival of this approach from TPUv1, leading to cheaper hardware for inference than the unnecessarily flexible

Google TPUs and Nvidia GPUs - unnecessarily flexible in terms of the numerics supported inside the large systolic arrays.

Why not a chip that only does FP4? Because FP4 is not always enough precision in the systolic array for all AI needs. It sometimes is, but not always. In the future perhaps improved AI methods will enable an FP4-only approach, but we are not there so far (and may never get there). So you have to support more than 4 bits for a serious AI chip at this time.

FP8 is always good enough if you do it right, so you could make a chip that only has that. However, Int8 is simpler and also good enough, so a pure Int8 chip is a real contender for a great chip to build in my opinion. The problem here is convincing your customers. They can always use 8 bits, but they might not know how to do it, even though it is pretty easy to do these days.

Here at the end, I'll point out that systolic array numerics do not need to be the same as scalar and vector numerics. The systolic arrays are the biggest, most expensive structures on a proper AI chip. So you want them to use cheap numerics. The vector and scalar structures on your chip are not as large, so you might support e.g. FP32 vector and scalar numerics even on a chip where the systolic arrays only support Int8. There are some AI ops like normalization and softmax that are not matmuls (and so will not run on the systolic array) that may be easier to implement that way. You don't really need FP32 for anything, I believe, but that is the easiest.

Structured sparsity for systolic arrays

Structured sparsity is another area where Nvidia has been innovating their systolic arrays ahead of Google's systolic arrays.

A *sparse* matrix is one where many of the numbers are zero. A sparse representation is, usually, one where you indicate the positions of just the non-zero entries and then say what just those numbers are. Such a representation is not a good fit for a systolic array, since systolic arrays are a large regular hardware pattern that requires a regular data pattern. Sparse representations can be highly irregular. There is no structure to it. So it doesn't work.

However, *structured* sparsity *does* work quite well for systolic arrays. In structured A-of-B sparsity, written A:B sparsity, every B entries in a row can have up to A non-zero entries. So in 2:4 sparsity, there can be 2 non-zero entries among the first 4 entries, another 2 non-zero entries among the next 4 entries and so on. So (2, 0, 0, 3, 4, 0, 1, 0) is 2:4 sparse while (2, 4, 0, 3, 0, 0, 1, 0) is not. It turns out that this kind of sparsity is a great fit for systolic arrays. [Nvidia GPUs, since Ampere, support 2:4 sparsity](#). This technique can double systolic array throughput while requiring only a very most increase in chip area and power (and a big decrease in power-per-operation). Structured sparsity at 2:4 also reduces memory bandwidth and storage requirements, since the zero entries do not have to be stored and transferred. The bandwidth and storage benefit is close to 2x, but not quite equal to 2x, since it is necessary to store the location of the non-zero entries, which requires 3 bits per 4 entries. The reduction in memory

bandwidth also usually would only apply to weights, not activations (i.e. vectors inside the AI that are not weights), since you usually should not sparsify activations, only weights. Google TPUs do not support this kind of sparsity as of 2025. I don't know why not.

[The way Nvidia suggests to use 2:4 sparsity](#) is to train a model densely (i.e. non-sparse) and then to later sparsify it so that it conforms to the 2:4 sparse pattern. They report that this leads to no or nearly no degradation in AI accuracy, i.e. the AI doesn't get dumber by doing this.

Is structured sparsity always a good idea? What if it leads to a reduced accuracy (i.e. a dumber AI), even if the reduction in accuracy is only slight? Perhaps that tradeoff isn't worth it sometimes? I will claim that you should normally always use structured sparsity if you are using an Nvidia GPU. If the possible slight reduction in accuracy concerns you, then instead train or download a model that is 50% bigger and then sparsify that instead. The result will be an increase in both speed (faster AI) and accuracy (smarter AI) at the same time. I don't know a case where you shouldn't use structured sparsity on an Nvidia GPU, unless you don't have access to modify and retrain the model (as required to sparsify it). If you are supplying models to others, consider giving them the option of a sparsified model, so that they won't have this problem.

Seems Nvidia got a factor of 2 over Google TPUs here and Google isn't so far looking to close the gap on structured sparsity in their TPUs. Not all systolic arrays are made equal. Google was initially ahead of Nvidia in systolic array innovation, bringing large systolic arrays and BF16 to market well ahead of Nvidia. Structured sparsity is another example showing that the period where Google was ahead of Nvidia in systolic array innovation is in the past. Google is innovating strongly in other areas of AI, but not on systolic arrays. Meanwhile, Nvidia is all-steam-ahead on the topic.

Is 2:4 sparsity the best approach? I'm not sure. If I were making a chip today, I would heavily investigate the possibility of 1:2 sparsity coupled with 7 bit integer arithmetic. This allows a particularly simple and appealing data format: an 8 bit format where the first bit indicates the position of the non-zero entry (out of the next two entries) and the remaining 7 bits are the bits of that integer entry. So you'd have a 7 bit multiplier systolic array that natively supports two formats: 7 bit sparse or dense integer (fixed point and integer are equivalent). The dense format would just set the upper 8th bit to zero (which aids compressibility). To have the format fit in a single vector width (instead of half of one, which is a nuisance for SIMD), you can pack together two adjacent rows into one row. If your vector registers are 32 bit, you could in this way pack together 8 rows to fill one 32 bit vector register. If a customer asks to have a matrix with a number of rows that is not a multiple of 8, I suggest defenestration as a method of resolution.

We normally talk of 4 bit and 8 bit numerics, where 8 bit is always sufficient for inference without loss of model accuracy (if quantization is done right) whereas 4 bit is just barely usable but sometimes good enough. We focus on 4 or 8 bits because those are powers of two and we in the industry are conditioned to prefer that (for good reasons? I'm not so sure). The truth is that 5 bits is usually enough and much better to work with than 4 bits. So it's very unfortunate that 5 is

not a power of 2. With 7 bits, you definitely have enough. Since we don't really need that 8th bit, this allows dedicating that bit to sparsity instead of arithmetic, so this format I'm proposing uses 7 bit arithmetic, though, if you want, you might consider 5 or 6 bits instead.

You may need some careful AI work to sparsify to 1:2 instead of 2:4 without much loss of model accuracy. You'll have to get some *good* AI people on this. If that works out, I think this simple Int7+1 sparse format could, potentially, be the end-state for AI inference numerics for many years to come if combined with the ideas on lossy and lossless compression described in a later section. I *do* recommend supporting dense operation where there is no sparsity, though then you can make that run at half the speed, just as Nvidia does for their structured sparsity support.

You might choose to support int8 for the dense format. Strictly speaking, this isn't necessary, since int7 can do pretty much any model int8 can, if quantization is done properly, but many models are quantized to int8 and not int7, because int7 is not a popular datatype, so your customers will have an easier time finding off-the-shelf quantized models if you do support int8.

Do note that while sparsity works well for weights, it is much less attractive for activations. This means that the 2x compute benefit does not apply to the matrix multiplication within attention that multiplies two matrices of activations. It also does not reduce KV cache storage and bandwidth requirements - unless you do choose to apply sparsity to activations.

[[[1:2 sparsity ideas The trivial approach to achieve 1:2 sparsity is to pair adjacent columns and set the smaller entry in each pair of entries to 0. You can achieve 2:4 sparsity in the same way, which [Nvidia recommends](#), followed by distilled training to recover accuracy. Nvidia also recommends combining quantization with this transformation, doing both at the same time. There are surely *many* improvements that can be made here and which *may* allow using 1:2 sparsity with no or little loss of accuracy. For example, you can likely get a better initial error by setting up a matching problem of columns so that columns with least co-error are matched instead of doing it randomly. Another approach you might try is to place a progressively stronger L1 loss on the smaller of the two entries in each pair instead of setting it to zero directly. After a while of that, collect the best, say, minimum 50% of column pairs that now have minimum error, preserve those column matches and rematch the other columns since we now have evidence that those were not as good matches (the training process pushed back too strongly) and then retry that. You might this way in the end have a difficult residue of a few columns that simply cannot find a good match. You might, if necessary, match those last few columns with a 0 column, which in effect completely preserves the last few difficult columns by disabling the optimization for those. There is also the option of using repeated lottery hypothesis training for distilled models (that aren't as expensive to train) - this ought to be the most powerful technique for this. You could also notice that sum of errors or sum of squares of errors in the weights is not quite the ideal matching / error metric, rather you'd measure the difference in dot product values across a representative set of input vectors. Surely there are many other ideas such as this that can push structured sparsity further, and, I suspect (but do not know), all the way to making 1:2 sparsity a technique that can always be used. If the proposed research project here fails, which it might, one can instead adopt Nvidia's (and AMD's) approach and use 2:4 sparsity instead,

which is already proven to work, but this format is more complex, not quite as bit efficient for weight storage, it's not so SIMD-friendly as 7+1 bit is, and it requires a bit more complication in the hardware. Note that 1:2 can be converted to 2:4 losslessly, but not the other way around. Another option you might consider is Int6+2 sparse 2:4. That's likely similar or a bit cheaper than Int7+1 sparse 1:2 in hardware. This all depends on proving these approaches via excellent quantization and training recipes that always work. If you think that most of your customers are too simple to follow a recipe for this, even if given an easy script, then this feature is less attractive. If you want to help humanity, though, then it is on you to figure out a situation where you *can* get customers to do this easily - easily enough that they will do it and do it well. One way to get there is to provide the quantized and sparsified models yourself or be your own customer.]]]

[[[Detail on TPU SparseCore for nerds You may think that Google TPUs support sparsity because they contain a so-called SparseCore. However, SparseCore has nothing to do with structured sparsity in systolic arrays. The SparseCore doesn't contain any systolic arrays and it doesn't do matrix multiplication. It's "sparse" in the sense of accessing the expensive HBM memory on the TPU at a lower granularity, so it can efficiently retrieve data in smaller pieces than the main TPU core(s) can, making it in that way more similar to a traditional CPU. It's supposed to be used for embedding table lookups, not matrix multiplications. TPUs can also do *block sparse* matrix multiplication, but that doesn't work *anywhere near* so well as structured sparsity (it is not a popular technique) and *block sparse* is not a hardware feature - it's something you implement in software. XLA:GPU can do structured 2:4 sparsity, but that won't work on XLA:TPU. TPUs really are missing a hardware feature here.

If you are thinking of replicating a SparseCore on your AI chip, then I'd suggest to put a regular CPU core on there instead. That'll serve as a "SparseCore" and also be more general.]]]

[[[Structured sparsity for training for nerds It would be perfectly possible to use structured sparsity also for training, but it is complicated by the need to transpose matrices during the backwards pass of training. Inference doesn't involve transposing matrices, but training does. A systolic array needs some additional special features aimed at training to get full advantage of structured sparsity during training. Also, it is necessary to select a sparsity pattern that remains A:B structured sparse even after transposition - not a big problem, but it has to be done. Nvidia's current structured sparsity hardware support is aimed at inference, not training, so it's not a technique that is used for inference currently. With some improvements to the hardware I believe everyone should use it also for training, and I predict that this will happen eventually, but I don't predict that "eventually" is going to be soon. People are still getting used to structured sparsity for inference.]]]

Mono-sized systolic arrays are unbalanced

Recall that transformers have two kinds of matrix multiplications: attention and FF. These are both important for the performance of transformer models. A complicating factor in designing AI hardware is that the matrix multiplications inside attention are small (the K dimension is small),

currently usually on the order of 16 to 128 elements. The matrix multiplications inside FF are much larger, determined by `d_model`, which can be e.g. 8,192 elements for a large model.

Google TPUs have large systolic arrays, now 256 x 256. This implies that they are very efficient for FF matrix multiplications but have lower utilization on attention, which does not fit as well with a vector size so large as 256. Nvidia GPUs have smaller systolic arrays (now 64 x 64 in Blackwell, 1/16th the size of 256 x 256), which implies that they have the potential for higher utilization on attention (depending on memory bandwidth, which is also a significant factor for attention) but are hardware-inefficient for FF since small systolic arrays are inherently inefficient compared to larger ones. So both Google TPUs and Nvidia GPUs have a problem here.

[[[Technical AI comment for nerds You can do attention that uses a higher vector width, thus increasing utilization numbers on TPUs. That's easy, assuming you are training your own model - just put in a bigger number for the attention vector width in your model definition. However, attention inherently is most cost effective with vectors that are not as large as what is optimal for FF. So increasing utilization by increasing the vector width is still inefficient, just in a different way than showing low utilization. An interested reader might read up on ideas such as [Grouped-Query Attention \(GQA\)](#), which reduces the issue by making larger vector widths optimal. While it reduces the issue, it still does not eliminate it entirely. More on this in a later section.]]]

As a chip designer, what should you do? FF works best with a wide vector width. Attention works best with a smaller one. My proposed solution is based on noticing that both Google TPUs and Nvidia GPUs contain multiple independent cores. All the cores on a given GPU have the same vector width and all the cores on a given TPU have the same vector width. What if you took half of an Nvidia GPU to do attention and half of a Google TPU to do FF and combined them on a single chip? Then the hardware would always be doing something that it is optimal for. Google and Nvidia are unlikely to agree to produce such a chip, though, but we can achieve this solution without mixing products from different companies. My suggestion is to have one FF core with a huge systolic array, to be used for FF, and also having several smaller attention cores, each with a smaller systolic array, to be used for attention. All on the same chip. This way attention can have high utilization (like on an Nvidia GPU) and FF can be implemented efficiently in hardware (like on a TPU).

This solution solves the mismatch between the optimal vector size for attention and FF. There are some challenges with this idea, though I will argue that they are not serious problems.

Problem: One more thing to understand This is yet another thing for people writing software for your chip to understand and design around. That's a genuine negative. However, at least for Google TPUs, the people programming these things are not naive users and this is the least of the complexity that they have to manage. For Nvidia, this is maybe a technique for expensive products like Blackwell, not necessarily something for a budget GPU.

Problem: Finding the right balance First, FF cores *can* do attention and attention cores *can* do FF, just not as efficiently (otherwise Nvidia GPUs could not do FF at all and TPUs could not do attention at all - but of course they can). So when you choose to dimension the area for FF and attention cores on your chip, you are choosing the ratio of FF operations to attention operations that AI practitioner should be aiming for when using your hardware, to keep all the cores doing what they are best at. So then AI practitioners now must go along with the ratio you have chosen between attention and FF or get reduced efficiency. Whatever ratio you choose as a chip designer, it will be suboptimal in some contexts. This is a problem, yes, but it's still an improvement even if you choose a bad ratio: currently, using the terminology being used here, TPUs are 100% FF cores and GPUs are 100% attention cores. 100% to either side is the most unbalanced you can be. So, if you, wisely, pick *any* ratio that isn't 100% to either side, this will be an improvement regardless of the context (no AI transformer does *only* attention or *only* FF - both are heavy computations and both are needed). So if you are terrorized by the idea of having to pick a balance, just do 50% / 50% by OPs (you can't just do half attention cores and half FF cores, since one attention core will be far smaller than one FF core, so that isn't balanced) . It will be better balanced than what is on Nvidia GPUs or Google TPUs, which are fully unbalanced due to using mono-sized MMUs.

[[[Very technical implementation aspect for nerds There is an issue here if you have an implementation where the same systolic array is first doing attention and then doing FF. In order for an attention core to be able to do *just* attention, and an FF core to be able to do *just* FF, which is ideal, you would need to pipeline the two. If you didn't already do that, you now need potentially twice as many stages in your pipeline as before, in order to partition the work, increasing KV cache memory demands and increasing latency. The easiest solution is to just still use all the cores for both purposes, even if it is not optimal, entirely solving the issue and still providing higher throughput compared to a mono-sized MMU architecture. However, the *true* solution, which can also halve your latency (instead of increasing it), is to modify the model so that attention and FF run in parallel. Do note that it is a benefit to overlap FF and attention also because attention requires more memory bandwidth than FF. So if you don't overlap them, you might be limited by memory bandwidth during attention and then have idle memory bandwidth that is wasted during FF. The net result is that you need more overall memory bandwidth if you don't overlap these two, leading to a more expensive chip. This is another example where AI chips are over-provisioned today due to poor software implementation.]]]

Problem: Creating the software and hardware It may at first appear that if you have two different kinds of cores on your system, that then you would need to implement everything twice, both in software and hardware: Once for a small vector width (attention cores) and once for a large vector width (FF cores). The important thing to notice here is that the cores differ only by the vector width, nothing else. So, if in your software and hardware, you never mention the vector width, other than by a constant named something like VECTOR_WIDTH, you can use the exact same code for both kinds of cores. You could even run the same binary, unmodified, since the ISA doesn't need to change depending on the vector width.

If you make the (vector) cache size proportional to the vector width, as you probably should, even cache hierarchy optimizations can be unchanged between cores of differing vector widths, since the number of vectors you can fit in the cache is the same.

There are reasons to avoid depending on the vector width in your code (in both hardware and software) beyond just being able to have differing vector widths on the same chip:

Reason 1: Vector widths already change between generations of the same hardware

Both Nvidia and Google have changed the vector width of their devices between generations. Nvidia have done it several times. So having a dependency on the vector width is bad business. It will cause unnecessary software and hardware (RTL) rewrites between generations.

Reason 2: Not doing this dramatically worsens your testing story in software

When you test your AI software for your AI chip, you need to provide each test with inputs and expected outputs to see if sw and hw does the calculation correctly. You will of course have to test the situation with multiple tiles, i.e. inputs larger than the systolic array size. So if your tile size is 256×256 , you necessarily have to write tests with more than $256 * 256 = 65536$ numbers in them. You will likely need 100s if not 1000s of tests (or even more than that - the TPUv2 test system would have been the fifth largest computer in the world by flops at the time, though this is primarily due to how fast TPUv2 was compared to other computers). You ideally have to manually understand and verify each number in each test. All 65536 of them. For each test. That's a bad time. If your software is parametrized on the vector width, you can write tests for 2×2 tiles and you can be confident that it will also work for 256×256 tiles (maybe have a *few* tests in that size). This may sound like a detail, but in practical terms, it is a *big* deal. How many in the industry are using this idea today? As far as I know, no one. It's a good idea. *Even if* the hardware that you are developing has mono-sized systolic arrays, you should still write your software tests as if the vector width was variable (you'll then need a simulator that supports varying the vector width).

You might think that testing doesn't sound important enough to drive a decision such as this, even in part. I'd suggest that you might be underestimating the importance of testing for AI software.

Reason 3: Not doing this worsens your testing story in hardware, too

Hardware testing has the same benefits here as software testing. Also, hardware designs are commonly tested on Field-Programmable Gate Arrays (FPGA - look it up if you don't know what it is). Many hardware designs will not fit on an FPGA in their entirety, which leads to a lot of trouble when testing designs on an FPGA. If you were so absolutely brilliant as to parametrize also your hardware design on the vector width, you can FPGA test a 2×2 systolic array and a 2-wide vector width and then maybe it *will* fit on your FPGA. So do that. Even if your final product doesn't support varying the vector width, it's still to your benefit to develop your chip as if it would. So *no*, you don't need to implement two different chips, just parametrize the vector width also in your hardware design. A given core on the chip must have a specific vector width, of course, but your RTL/Verilog/whatever definition of the core design can leave it as a symbolic

constant. Testing and debugging a 2-wide chip is easier than debugging a 256-wide chip. It's true in software and it's also true in hardware.

The summary is that having a chip with a varying vector width among its cores ends up forcing both software and hardware teams to do something that they should be doing anyway. If you are creating an AI chip, this idea isn't mandatory to be competitive, but I think it's something to consider carefully.

Chips with larger systolic arrays are easier to make

Larger systolic arrays has implications for the competitive landscape of AI chip products. The larger the systolic arrays can be, the easier it is to implement competitive AI chip hardware. Why is that? It's because the pieces of AI chips that keep getting more area are the systolic arrays and caches. The rest are shrinking away in terms of % of area. Or should be, if you are doing it right.

Systolic arrays and caches are not hard to make compared to e.g. the awesome complexity of an optimized traditional CPU. It is everything *around* the systolic array that gets to be complex, because that part around the systolic array resembles a traditional CPU - or actually *is* a traditional CPU, which I think is ideal.

If a traditional CPU is the most complex thing hardware engineers have ever made, which is true, why do I say that making a traditional CPU is easier than the alternative of making some bespoke and inflexible AI chip that doesn't very obviously resemble a CPU? What traditional CPUs have going for them in terms of hardware complexity is that, while they *are* complex, *all* educated hardware engineers already know exactly how to make them. A hardware engineering degree is pretty much a years-long education in precisely how to make traditional CPUs. They all know how. So if you are going to make something in hardware that is complex, you are way ahead of the game if the thing you're making is a traditional CPU. So CPUs are not simple, but, even so, they are easy to make for people with a hardware engineering degree (outcompeting Intel or AMD on making a CPU is not easy, of course, but just making an OK CPU is not hard for hardware engineers). They are also the easiest thing to program for people with a software engineering degree. So you can see that there is a benefit here.

If the systolic arrays are very large, then it doesn't even matter as much if you can do the CPU-like pieces around it very efficiently in terms of area and power, because that's then a smaller and smaller part of the chip the larger the systolic arrays get. Therefore, the minimum required sophistication in terms of creating competitive AI hardware progressively reduces the larger the systolic arrays get. There is a benefit to greater AI hardware complexity, but it is less and less required to harvest that benefit the larger the systolic arrays are.

The *software* is no easier to make regardless of the size of the systolic arrays, though (also no harder, once you are beyond a certain size), so the minimum required hardware complexity reduces with the size of systolic arrays but the software complexity is the same regardless. The

software complexity required to use an AI chip at high utilization is very high, even if the ideas later in this document do help to reduce it significantly. For this reason, it is a poor idea to create a hardware structure around your systolic array that is hard to program in order to save on the hardware. That saves you hardware cost on the progressively smaller hardware piece (the non-systolic array and non-cache piece), while costing you on software complexity, which is already too high. Not a good trade. This argument is false for small systolic arrays and becomes increasingly true the larger the systolic arrays are.

I therefore predict that the eventual future of AI hardware is just standard CPUs, fully and flexibly programmable as any standard CPU is, driving very large systolic arrays. Here “standard CPU” still includes high memory bandwidth, fast network, high vector compute etc.

There is nothing easier to program than a CPU. GPUs are programmable via CUDA, but CUDA is not an improvement on C++ (on which CUDA is based). The amazing thing about CUDA is that it makes GPUs generally programmable *at all* - that’s genuinely amazing. But GPUs are not as easily programmable as CPUs are because C++ is easier than CUDA. CPUs are the most programmable thing and that’s why CPUs are the *eventual* future of AI, not GPUs - because CPUs can be more convenient and more efficient drivers of systolic arrays than GPUs can be.

But hold on, using Nvidia software that runs on GPUs seems to work well for a lot of AI practitioners? Well, yes, that’s because Nvidia has invested massively in that CUDA software, and so has the entire AI community. It is definitely possible to build easy-to-use software for Nvidia GPUs - Nvidia does it anew with every new generation of GPUs. I’m saying it would have been easier to have built that same software for CPUs. That isn’t happening because the AI CPUs currently available are not sufficiently competitive.

Jensen is the CEO of Nvidia, and if you watch some of his interviews, one thing I’ve noticed is that where others speak of a CUDA moat, Jensen speaks more about the benefits of the CUDA *ecosystem*. I think Jensen is right to focus there. Far from CUDA being a benefit, I would say that Nvidia is stuck in a CUDA *swamp*, but [the fourth castle stood](#). It’s not CUDA that’s amazing, it’s what Nvidia and others have built and continue to build on top of it. Something that would have been easier to do without having to use CUDA, but, never the less, it was done with CUDA and now, unfortunately, we have a big CUDA ecosystem / swamp castle.

Why aren’t the AI CPUs currently available competitive against Nvidia GPUs and Google TPUs? I believe that this is purely because Nvidia and Google have been executing better on their AI visions than AMD and Intel have - so far. The potential for the AI CPU is great, but the premier CPU companies just haven’t been realizing that potential as a matter of execution.

Nvidia is making GPUs seem not just suitable but in fact *best* for AI because their GPUs are most excellent, their execution is fantastic and their software empire is vast, but, in the very end, the future of AI will be CPUs, not GPUs. Nvidia will not be able to outrun this future forever, but if they keep up their current excellent execution, they might be able to outrun it for a long time indeed. Also, it would certainly be possible for Nvidia to create an AI CPU (Nvidia’s current

CPUs are for *supporting* AI GPUs, but those CPUs aren't themselves AI devices with large systolic arrays). So I'm not predicting the demise of Nvidia, but I am predicting that, in the end, we won't use GPUs for AI.

Advanced section: Non-square systolic arrays

Square systolic arrays are the easiest to deal with in software. However, there are some advantages to non-square systolic arrays and this section concerns those. This section is more mathematically involved and uses many concepts explained further on in the doc, like decode workload, token latency versus throughput, KV cache, speculative decoding and batch, so you may need to postpone reading this section if you don't already know these concepts or you may prefer not to read this section at all if you are looking for a light read.

Suppose we are doing a matrix multiplication of tiles like this:

$N \times K$	Left Hand Side (LHS) activations, times
$K \times M$	Right hand side (RHS) weights, yielding a
$N \times M$	result

In this section we will look carefully at each of these dimensions N , K and M .

The N dimension (LHS rows)

The first thing to notice is that one row of the LHS $N \times K$ tile yields a corresponding row of the output. Rows of the LHS do not interact. So if LHS rows are passed in one at a time, a $K \times M$ systolic array is actually doing a series of $1 \times K$ times $K \times M$ matrix multiplications, yielding individual rows of size $1 \times M$, like this:

$1 \times K$	LHS activation row, times
$K \times M$	RHS weights, yielding a
$1 \times M$	result row

So it looks like N is not really a property of the systolic array at all, it's just something to do with how many times we pass in a row to the systolic array. However, this isn't how I suggest to look at it. The question is how many LHS rows N are required to get 100% utilization out of the systolic array. This is in a sense the true N of the systolic array, even if the $K \times M$ systolic array does not have a physical N dimension.

What determines N ? In short, it depends on the time it takes to load the RHS tile/matrix. It takes time to load a $K \times M$ RHS tile into the systolic array and for 100% utilization, we need to have enough LHS rows to process so that this takes as long as it takes to load the next tile into the RHS. The most obvious way to make a systolic array is so that the LHS and RHS of the systolic array have identical 1:1 bandwidth. So in this case the minimum required N to get full utilization requires the LHS ($N \times K$) and the RHS ($K \times M$) to be the same size, which is achieved if and only if $N=M$. In this case, a 128×128 ($K \times M$) systolic array will require $N=128$ rows to run at 100%

utilization. In this case, e.g. $N=1$ gets only $1/128 = 1\%$ utilization. So N is important even though it is not an obvious physical property of the systolic array.

[[[Terminology detail for nerds You can pass multiple rows to a systolic array at the same time for efficiency. E.g. TPUv3 vector registers are 8×128 and you pass a whole register to the systolic array at one time, leading to something you might call an 8×128 times 128×128 systolic array, and you could think of this as meaning $N=8$. That's also a sensible way to define N , in fact this is probably what people will assume you mean if you talk to them about the N of a systolic array as a third dimension, but that's different from what I'm talking about here. **]]]**

Suppose we provision the systolic array with twice as much bandwidth for the RHS as for the LHS. Then we only require $N=M/2$. We can generalize this to formula for the minimum required N for 100% utilization of a systolic array, namely

$$N = M * bw_ratio = M * (\text{LHS bandwidth}) / (\text{RHS bandwidth}) .$$

There are benefits to allowing a smaller N in this way. In decode attention, the N dimension corresponds to the number of query vectors, which can be low. So supporting lower N with high efficiency can increase systolic array utilization during decode attention.

In decode FF, the N dimension corresponds to batch times the number of speculated tokens. So allowing lower N can let you decrease batch, which directly leads to lower token latency (without changing network requirements) and KV cache storage requirements for decode overall - both important metrics. Do note that lowering batch does not at all improve throughput, though, so it doesn't help with tokens per dollar.

However, the picture is not so simple as increasing the RHS bandwidth into the systolic array to support lower N . The RHS weights (or key activations in attention) are typically stored in HBM and then in SRAM (cache) before being loaded into the systolic array. So you do not only have to improve the bandwidth into the systolic array, you also have to increase HBM and SRAM bandwidth to keep up. This is an expensive optimization, but perhaps worth it in some cases to lower token latency.

Why not provision bandwidth for $N=1$? Some chips do this! They are known as "batch=1" or "low batch" chips. Though this is very expensive. At this point what you have isn't really a systolic array anymore and it certainly does not capture the efficiencies that you expect from a systolic array. You will need M times as much bandwidth per operation if you do this, where e.g. M can be 128. So you need to provision $128 \times$ as much bandwidth. Expensive!

[[[Detail on low batch Groq LPU for nerds The Groq LPU is precisely this kind of low batch chip with a tiny N , perhaps as small as $N=1$ - I haven't been able to find a specific number for this, they might have a very small N that still isn't quite so small as $N=1$. So it makes sense that the LPU does not use HBM, since HBM would be too slow for this extreme case that has very high bandwidth requirements. Instead, the LPU stores all the weights in SRAM, with a cache

that must provide unusually high bandwidth. [Each LPU3 chip has a whopping 500 MB of SRAM](#). Such a large SRAM cache that also has to have higher bandwidth than regular SRAM caches requires a large overhead in terms of both chip power and area. Also, a 500 MB cache, big as it is for a cache, is still small compared to the size of models, so e.g. a 100B FP8 model requires at least 200 LPU3 chips just to store the weights.

I've written elsewhere that the LPU does not use a large systolic array. People have wondered: How can I know that, given that I don't have access to the internals of their chip? I say that because such a tiny N in effect means that what you have is not really a systolic array, not even if you tried to make it look like a systolic array (which you could - I don't know if they did). You are not capturing the area and power characteristics one would expect of a large systolic array.

Large systolic arrays are highly economical on a tokens per dollar basis and the LPU low-batch approach discards those benefits. So why would Nvidia license Groq technology in this case? Because all this expense leads to a valuable benefit: very low token latency. A batch of 64 means 64x the token latency, all other things being equal (which they aren't!). So you can see how a low batch helps there.

Nvidia GPUs, using large systolic arrays, are already getting excellent token latency, but they do require a certain sized batch to function well for decode FF. So a Groq LPU can offer even lower token latency than an Nvidia GPU can, but at a higher price. So Nvidia is using Groq technology to serve the niche of expensive but low latency tokens. Jensen explained this during his GTC 2026 keynote, that [LPUs are for expensive \("premium"\) low-latency tokens](#), though from coverage of the event not many commentators relayed this part of what he said.

The N dimension doesn't correspond to batch during attention, so you can do low-batch attention on a large systolic array with a large N, like on an Nvidia GPU, and still get high utilization. So if you buy LPUs, Nvidia still recommends to use GPUs for attention (and prefill), reserving the LPUs for decode FF. GPUs are more economical per token and have large HBMs to store KV cache, so this mixed strategy makes sense. Many commentators did relay the idea of this mixed strategy, but left out that this is only for high-priced low-latency tokens. Regular tokens with excellent but not extreme low latency requirements are more economically served with a pure GPU approach. In other words, with a large systolic array approach.

Groq marketing also heavily focuses on the merits of what they call static scheduling. This is sensible for them to draw attention to as a contrast to the chaos that is GPU warps, though it is not a unique thing for Groq chips. TPUs certainly also allow careful timing of things and, even on GPUs, with special techniques (that are not usually used) you can *roughly* predict when which warp will do what. Groq's marketing around static scheduling is correct as they state it, but they exaggerate the impact and how much this differs from what happens on other chips.]]]

The K dimension (the reduction dimension, aka "input feature")

This is the dimension that has been focused on in previous sections. In the previous sections, it has already been remarked that attention has low K while FF has high K. So attention limits how

large of a K that you want on your systolic array. It has also been remarked in a previous section that you may prefer to have cores on your chip for attention, with a lower K (e.g. 64) and cores on your chip for FF, with a higher K (e.g. 256). In these previous sections, it has been assumed we were talking about a square systolic array with $N=K=M$. But what if we allowed K to be larger than the other dimensions?

Larger K is not so helpful for attention, but it is for FF. During FF, you can have feature dimensions (i.e. K dimensions) in excess of 16000. So $K=1024$ or even larger can be used here. Note that this feature dimension is also used for within-layer parallelism for FF across several chips, so very large systolic array K dimensions does reduce the maximum degree of parallelism that you can extract. E.g. with model $K=16384$ and systolic array $K=1024$, you can parallelize at most 16 ways, since $1024 * 16 = 16384$.

Doubling K of the systolic array is roughly equivalent to having 2 systolic arrays of the previous size and using vector compute on the general scalar/vector core to add up the results. The benefit of a single systolic array with larger K is that this has fewer overheads for vector/systolic array bandwidth, control/scalar logic and vector compute on the vector core that is attached to the systolic array.

Do note that is it *very convenient* that square systolic arrays involve vectors that are all of the same vector width. If you introduce systolic arrays with a K that is not equal to M, then the vector width on the vector core is going to be different from K or from M (or both). You will have complications related to that in both hardware and software.

See [Wu et al.](#) for more on their high K approach.

The M dimension (aka “output feature”)

Maybe doubling this dimension also makes sense? It does, in a way, but it corresponds roughly to having 2 systolic arrays with shared LHS input. So you might prefer to just have 2 systolic arrays instead, though that sharing of the LHS input does save some on-chip bandwidth and control logic, so there is *some* benefit. I don't recommend this, but you have not done a thorough architectural exploration if you haven't considered this, too, carefully.

If you want to increase M, you may prefer to increase K and the vector core's vector width at the same time, because that way all the vectors are the same size, which is much simpler to work with. This just corresponds to making a larger square systolic array, so this is the standard idea that's been assumed the whole time previously. You might choose to increase M and K without increasing N - this is simply a square systolic array with lower N, which we discussed in the previous subsection on N.

The replication dimension

This isn't normally thought of as a dimension of a systolic array, but it has similar effects as the other dimensions, so I'm including it here for completeness. I've mentioned this idea a few times already: you can also scale up systolic array compute on a chip by having several systolic

arrays. Either on separate cores or by having several systolic arrays per core or both. Both Google TPUs and Nvidia GPUs have done both of these.

In general, replication by N times multiplies all costs and benefits by N . There are some exceptions to this where something can be shared. E.g. if you place two systolic arrays on the same core, you might be able to reuse some of the shared SRAM (cache) capacity.

Having many systolic arrays is normal because the size of the systolic arrays is limited by the various factors we've gone through in prior sections, yet one systolic array, even if it is 256×256 , is not nearly enough to use up all the area on a modern AI chip. So people use more than one. Those systolic arrays on one chip then can share some caches and can be connected to each other at bandwidths higher than what you can reasonably provision off-chip.

The tyranny of the cycles and systolic array DMAs

One of the things that make systolic arrays difficult to program is that they require a lot of attention from the core that drives them. This section is about that and what might be done about that issue in hardware.

Suppose you have a systolic array that takes in a LHS vector (row), a RHS vector (column) and gives out a result vector (row) every cycle. Then the core that controls the systolic array has to deal with these 3 vectors on *every cycle* just to keep the systolic array busy. You may also need to do two vector operations to read those two input vectors from somewhere and you may need to do other vector operations like softmax on the output, which can involve many vector operations. So we are up to maybe 10 vector operations per cycle and these vectors may be so wide as 256 elements. This is a lot of vector compute capacity to provision in hardware, but also in software this setup leaves very little slack to do anything else on this core and you have to set it up so that you are ready to do what you need to do on every single cycle without fail. This is challenging to deal with. You'll need an excellent compiler to make this work out. I call this issue *cycle tyranny* - the core has to be available to deal with the systolic array on every cycle all the time.

[[[Cycle tyranny disaster for nerds Why do we live under cycle tyranny? Because the alternative is that the systolic array is idle. I have personal knowledge of a chip project (not TPUs) that met disaster from not taking the issue of cycle tyranny seriously enough - they made a chip that could not ever run at full systolic array capacity no matter how you programmed it because the controlling core could not keep up with the tyranny. The degree of the tyranny of the cycles for servicing a systolic array sneaks up on people. You really must think carefully about how you will handle this.]]]

[[[Unrolling to alleviate cycle tyranny for nerds The previous story was about doing too little about the cycle tyranny. This one is about doing too much - which is much better!

When scalar compute was provisioned for TPUv2, engineers looked at example inner loops that might run on TPUv2 (which was completely different from TPUv1), and realized that these loops required a high rate of scalar compute to compute memory addresses to keep up with the systolic array. In fact, the amount of address computation they observed was so high that they added a special hardware unit just to compute addresses for loops. This unit offered very high address computation capacity, but it was somewhat difficult to use. So we never used it.

[Loop unrolling](#) is an ancient and powerful compiler optimization. It turns out that unrolling loops at least a little bit makes living under cycle tyranny much easier, so for TPUs loops are always unrolled at least a little bit. As any compiler developer can tell you, unrolling loops can greatly optimize address calculations, since many addresses become constant offsets, which are cheap to compute, and that's why we never needed the address calculating unit. The example inner loops that were used to derive the need for an address computation accelerator had not been unrolled at all, so that's why this unit appeared to be necessary even though it wasn't. **]]]**

The issue of cycle tyranny is of course not improved if you have several systolic arrays per core. Also training is worse for this than inference since training has additional vector work to be done for updating the weights using optimizers like Adam.

There are a number of ways that you can reduce cycle tyranny. [One is 2D vector registers, e.g. TPUv2 vector registers are 8x128.](#) So in this way you can provide a vector register as an input to the systolic array and that will provide 8 whole vectors (rows), so you won't have to worry about that again until the systolic array is done with those 8 rows. That already reduces the cycle tyranny by a factor of 8. You could also have even more than 8 rows. Be aware that introducing 2D tiling can lead to very significant software and hardware complication. You might especially run into problems if the amount of data to process is not a multiple of 8, or however many rows your vector registers have.

Another approach is to have hardware queues before and after the systolic array, so that you can provide a bunch of rows in quick succession and then have the vector core do something else for a while. This helps a bit, though deep queues are not free in hardware.

You can also run the systolic array at a lower clock rate so that it is easier for the scalar/vector core that drives it to keep up - both in hardware and software. This approach was taken in early TPUs. Though if you are down to 8 bit or lower arithmetic in your systolic arrays, you are going to need a large number of such systolic arrays to fill a reasonably-sized chip with compute. If you will use slow-clocked systolic arrays, then you are going to need *even more* of them. This choice may limit available parallelism between chips and/or increase token latency.

Here's a different idea I haven't seen used anywhere (though it starts to be similar to dataflow designs): Systolic array DMAs. This is a DMA that delivers potentially 1000s of rows/columns from some place into (or out of) the systolic array - could be from another systolic array (maybe with a vector addition), could be SRAM, could be HBM, could be over the network from another chip. This way you much reduce the tyranny of the cycles - the core doesn't have to service the

systolic array very often in cases where what is happening is just a direct transfer from or to somewhere. During inference, the weights are always just being transferred, and so is the KV cache usually when you read it. Except for (de)compression, but in the section on that I advice adding this as a feature to DMAs. If the data being transferred in this way will not be reused, then you also avoid incurring bandwidth to read and write this data into registers or SRAM before the data goes into (or out of) the systolic array. This also allows you to stream data from your systolic array directly into (or from) some kind of networked operation (like all-reduce) or onto another core. Whether this is a good idea depends (also) on the overhead in hardware of implementing this, but it seems a promising idea to me.

The main problem with a systolic array DMA is that you may need to transform the data in some custom way before or after the systolic array. In this case, the data then has to go through the scalar/vector core in the usual way. If you have reduced the throughput of the scalar/vector core on the argument that systolic array DMAs are available, you may then in this case be unable to reach 100% systolic array utilization, being instead bottlenecked on scalar/vector compute. So you will have to keep the potential for this kind of scenario in mind if you go this way.

Systolic array DMAs also go well with my suggestion to have multiple cores, some of which do not have systolic arrays attached. These can then be used for other purposes, like input pipeline processing, running network algorithms for all-reduce, doing retrieval (for RAG or within-layer) or running the OS that I also recommend to run on the device. So these cores can do various other work without wasting capacity on the systolic arrays. In cases where the vector or scalar capacity on the core directly attached to a systolic array is exceeded, you might then be able to use systolic array DMAs to offload some of this excess work to these additional cores in a way that is particularly light on the primary systolic array scalar/vector core.

Anecdote: Systolic arrays are hot

This section contains a long and rambling tale about managing unexpected systolic array heat production in TPUv3.

Systolic arrays contain such an incredible density of computation that the area of the chip that contains a systolic array may be far hotter than other areas of the chip. This is a problem as it could damage (melt) that one spot on your chip - which is why it is preferable for heat to be produced evenly across a chip, not all in one spot. There are ways around this, such as making a larger systolic array at a slower clock than the rest of the chip, placing systolic arrays far from each other on the chip or, in extreme cases, perhaps (perhaps not) even separating pieces of one systolic array across the chip, putting other (less hot) logic in between the pieces. This is a real concern but it is something that can be managed and it shouldn't limit the size of systolic arrays.

Do note that systolic arrays produce far *less* heat per mathematical operation that they perform than other hardware structures. The reason that systolic arrays still produce high heat is that

they are so incredibly dense with computation that the total heat gets to be high, because so much computation is achieved in a small area, even if the heat *per computation* is low.

In TPUv3, the heat production of the systolic arrays on the chip was underestimated during the design phase, which led to a nasty surprise when the chip was produced and arrived at Google headquarters. Such things can be corrected if you modify the chip and do a whole new production run, but this would delay TPUv3 and that was not acceptable. So what then?

The hardware team did a lot of work and put in place a long list of mitigations for this issue, which meant that we were able to release TPUv3 without redoing the chip. Part of this issue was also corrected in software, using a compiler-based static analysis technique to avoid ever driving the systolic arrays in a pattern that would produce dangerous amounts of heat, based on an empirically verified mathematical model of heat production inside the chip. The result was that TPUv3 could safely run at a 25% increased clock rate due to this software solution, significantly reducing the impact of the problem. I personally came up with and wrote this software solution. I was the technical lead for the TPUv3 software.

Internally, I called this feature “compiler throttling”, which later led my manager’s manager to politely inform me that I suck at marketing. Compiler throttling sounds like something that makes the chip go slower, doesn’t it? But the actual result of the feature is to produce up to 25% faster TPUv3s and some important workloads did realize the full 25% speedup. So if it makes things go faster, why not call it Turbo Mode instead? Now that’s a nice name. As soon as he told me this, I knew that he was right, but it was too late to change the name at that point. :(

Compiler throttling, uhm, I mean, “Turbo Mode”, was an interesting (and very challenging) project for me, but if *you* are producing a chip containing a systolic array, do make sure to carefully consider the heat production of the systolic array. You may in particular want to consider including a way for the chip to notice in hardware that it is using too much power and quickly throttle itself down in that case. TPUv3 did have a hardware throttling mechanism, but it was based on measuring temperatures after the fact, not how much electricity was flowing into the chip at any given moment. That would have been fine normally, but this response was too slow given the higher than expected heat spikes that could occur inside the systolic arrays. In other words, you might damage the chip on the inside before the higher temperatures reached the outside of the chip and caused throttling to occur. Turbo Mode was a software solution that ensured that the compiler couldn’t produce an instruction pattern that could cause that result, which made it safe to increase the maximum clock rate.

You might ask why you wouldn’t just create a cooling solution to handle the worst case without throttling. Why do chips need throttling at all? Just make it work without it. That is what I was at first wondering back then. The reason is that the worst case data patterns and instruction patterns are far, far, far more heat producing than the average case and also far worse than what will ever actually happen when the chip is used. The *really* bad data and instruction patterns cannot happen by chance, they have to be intentional to ever occur - i.e. someone is messing with your chip, not trying to do anything useful with it. So you need throttling because

handling the worst case without it is far too expensive and unlikely to be of any benefit to real customers. This is true for all high performance chips, not just TPUs. That's part of why they all have a hardware throttling feature that reduces the clock rate in response to high temperatures.

There is also another reason. If your chip is capable of briefly running at a higher speed, until it throttles down, then you can report in your marketing that your devices are capable of running at this higher speed. You don't have to mention that they cannot sustain this speed. Nvidia GPUs are known to throttle down significantly, so Nvidia has been getting a marketing benefit from this factor. Google TPUs aren't like that - they may throttle, but not as much. This is likely in part due to the fact that Google TPUs were initially built for internal Google use, and in this scenario marketing is not as relevant, just actual delivered performance. A conclusion from this is that you cannot directly compare ops / second numbers between Google TPUs and Nvidia GPUs, you instead have to look at numbers for long-running benchmarks that resemble your actual workload.

Even though throttling in some cases may only occur for bad workloads, you can't just ignore the worst case, assuming or hoping that no one will be so foolish as to trigger it, since then a curious experimenter, a virus on your computer or a malicious user in Cloud could destroy all of your expensive AI hardware in a moment. In particular, what might happen in a cloud environment in this scenario is that the malicious program would destroy the hardware it was running on and then be restarted on fresh new hardware due to the datacenter automatically detecting that the hardware had failed ("oh, sorry about that, let's get you a new machine right away"). So, without throttling, a malicious user could destroy potentially all free TPU capacity across Google's global fleet of datacenters in this manner. So AI chips, and chips in general, do need throttling features. You might from this story also conclude that... Turbo Mode... had to be developed in a very careful manner that was probably a lot of effort to get approved for release to Google datacenters. Yep, it sure was. If you get it right, big win. If you get it wrong, completely unacceptable.

Why is 25% a big win? Well, if you invest \$X in TPUs, and you get 25% more flops for free from this technique, you've arguably realized an economic value of 25% of \$X. So if X is a big number, and it sure was, then that's a lot of money. Much more than 100x my annual salary, in fact. I think that's a relevant calculation since this would not have happened if I had not been at Google at the time. Unfortunately, despite this brilliant argument, I didn't get a commission based on that calculation. Oh well.

You may have heard that salaries for higher level engineers in Silicon Valley are high, and they are. Even so, I just told you that Google could have hired much more than 100 of me for what they got out of this one project that I did and that only happened because I was there. Sometimes that happens. So this story might begin to explain why large companies are so keen to hire great engineers even at very high prices.

Another bad pattern that requires throttling is if your datacenter has a hot day, for whatever reason, leading to increased power usage - yes, unfortunately, a hotter chip draws more power,

causing yet more heat. In this case, without throttling, all the AI hardware might shut itself down to protect itself all at the same time, leaving the entire data center all of a sudden without any AI capacity. If the higher heat would be due to some kind of global systemic issue, like a cooling software update that was rolled out globally, you might lose all AI capacity in your entire fleet across all datacenters all at the same time in this way. Or you might lose only your AI capacity on the currently sunny side of the Earth. Such issues are why you will find throttling features on all high performance computer chips, despite the problems throttling can cause - e.g. it causes noisy benchmarks and also you may have less AI capacity on very hot days. We had to and did make sure to avoid the potential for any such scenarios in TPUs. If you are making AI accelerator chips, you will have to consider such scenarios as well.

Compression for activations and weights

The deep learning revolution started out with 32 bits per number, then moved down to 16 bits, then 8 bits and now even sometimes 4 bits are used with little loss of accuracy - i.e. without making the AI much dumber. This is *lossy compression* for both weights and activations and it is a standard technique in AI.

However, what about lossless compression? This might seem like an odd question, since we are already using lossy compression, and isn't lossy compression more powerful than lossless compression? This is a misunderstanding. Lossy compression works by removing unnecessary precision. Lossless compression works by exploiting a non-uniform pattern in the data. These are different things. This is why the JPEG image format uses *both* lossy compression and lossless compression. In particular, JPEG applies [Huffman coding](#) to the data after it has been lossily compressed. Why don't we do that for AI, too? I don't know why not.

There has been [some investigation](#) of this topic, but not much. Nvidia GPUs have hardware support for *texture* (de)compression, but that is a graphics thing, not an AI thing. There is also [a more general decompression feature in more recent Nvidia GPUs](#), but it cannot at all keep up with memory speeds (~1/10th), so you'd actually get much less bw with it than without it if you compress in HBM - though you might still get some boost if you compress some of your data but not all of it. AMD has recently announced "Universal compression" for their future GPUs, which sounds interesting, but there is no detail on it so far, so it's unknown what it is about so far.

You can see some results for Huffman encoding as applied to AI [in this paper](#), though in that paper they did something specific to BF16 and broke it into 5 bit pieces, which isn't going to be optimal. I'd much rather see results for 8 bit integers or FP8. [This paper](#), which doesn't contain much information, claims 20% reduction in data size for real-world AI workloads. This is using just lossless compression. In the paper, they want to do decompression in one cycle, which seems unnecessary, since cycle latency is not so critical for AI due to software pipelining (cycle latency is not irrelevant, but not critically important, either).

Something that has stood in the way of lossless compression for AI is that the results are variable - not all vectors compress equally well. This leads to some trouble, though this can be

resolved by having high and low priority tokens. If a given batch of tokens is taking more time or space than usual, simply drop some low priority tokens and rerun them later. With this technique, mild amounts of variation, like you'd see from compression, is not a problem.

In this section, I would like to raise awareness of the usefulness of huffman encoding and decoding tables also for *lossy* compression. Such tables are more useful than it might at first seem. I think it's something to consider including at memory and network speed in future AI accelerators.

To huffman encode e.g. an 8 bit integer, you create a table with $2^8 = 256$ entries. Each entry has a bit pattern of some variable length. So you take an 8 bit integer, you look it up in the table and you output that many bits in that bit pattern. It's that simple. The idea is that more common values receive shorter bit patterns, while rare bit patterns receive longer bit patterns, so that the net effect is to compress a vector or matrix. Suppose the longest supported bit pattern is 10 bits long. To decompress, you also have a table, now with $2^{10}=1024$ entries. The value in the table is the original 8 bit pattern that encoded to that bit pattern, along with the encoded length (so you know how far to skip forward when decoding). So encode and decode consist of two tables that are inverse of each other. Since the decoding table is larger than the encoding table, not all entries in the decoding table will be used.

How can this be used also for *lossy* compression? When you create the encoding lookup table, you can map multiple values to the same bit pattern. In this case, that table encodes a lossy compression on top of the lossless huffman encoding. All FP4 values times 2 (to avoid halves) can be encoded as 8 bit integers, so there is literally an encoding table that encodes Int8 to FP4 and back (the times 2 just creates a fixed point representation). This doesn't change the internals or throughput of the systolic array, which would then remain at e.g. 8 bit, but what it means is that, by varying the encoding and decoding tables, you can dynamically craft numeric formats like FP4 or any bespoke custom format for the purpose of compression and therefore saving memory storage and bandwidth. And you get the benefit of lossless compression on top of the lossy compression. So for each layer, you can run an automated optimization that finds an optimal combination of lossy and lossless compression for that one layer.

In the paper mentioned above, compression is applied only to weights and it is used right at the systolic array. That is one option. However, compression can also be applied to activations (non-weight vectors inside the AI), which means it can be used also for e.g. KV caches, not just weights. This leads to mild variability, but this can be resolved as mentioned above.

This kind of compression requires hardware support to be able to run at the speed of memory and network, both for encoding and decoding. Huffman encoding is serial, though it can be parallelized by having multiple independent streams of data within the compressed file format. If this feature would be added to the DMA feature of a chip, and software would use DMAs to access data, then it could be quite transparent to the software - it's just a question of configuring each DMA to do (de)compression. This requires high throughput but not necessarily low latency

given software that is software pipelined - which it should always be. DMAs can already have high-ish latency anyway.

I have not empirically quantified the impact of this optimization against the area and power costs, so certainly an AI chip startup ought to do that before deciding whether or not to spend chip area on this. As far as I know, there is no public research thoroughly investigating this - the above two papers is the closest thing that I've found and they don't give the necessary picture to make a decision.

I believe that the Int7+1 structurally sparse format described in a previous section, if coupled with this lossy + lossless compression approach, could be quite attractive. The compression means that you can support 7 bits, but seamlessly scale down to smaller non-integer bit widths (e.g. 4.3 bits) if that is right for a given layer.

[[[What about TurboQuant for nerds [TurboQuant](#) is a KV cache compression technology from Google. It supposedly gives ~8x reduction in KV cache size without loss of accuracy. This existing paper was introduced to the public by a Google blog post and was at that time covered widely in the press. Both the blog post and the press coverage was highly misleading. The baseline for that ~8x is 32 bits per element, which is not a serious comparison here, so the number is meaningless. In this case 8x means down to 4 bits, which has been done before, so that isn't a new thing. In other words, the press coverage focused on something that wasn't even improved! The actual TurboQuant advance, if the rest of the blog post is to be taken at face value, is doing what it does without retraining, without much loss of accuracy and even without calibration data. That is indeed a significant advance. So it's not necessarily a bad result, but the true story has little resemblance to what the press reported. I'm skeptical, but perhaps TurboQuant is how you should compress KV cache. You'd have to go find that out. If it is, you'll need to ensure that your chip is able to perform the non-trivial overhead in TurboQuant at memory speed.]]]

Kinds of AI workload - training, decode, prefill

This chapter explains the primary three kinds of AI load that AI chips have to support: training, prefill and decode. These are all significantly different in terms of the requirements for AI hardware and software and someone making an AI chip needs to be aware of which of these workloads they are targeting their chip at.

Training This is where the weights in an AI model are updated to make the AI cleverer. It's AI school. The AI isn't doing anything useful here, just like students don't do anything useful, it's about learning to do, not doing. Training is the most challenging workload in terms of features required on the chip and in terms of the software that is required to implement training. Building a good training chip, with accompanying software, is the most challenging AI chip project. If you did the hardware and software right, your chip ought to be compute-bound during training, i.e. the bottleneck is the systolic array, not memory bandwidth or anything else.

Inference This is where the AI graduates from training school and potentially starts doing useful work. The weights are no longer updated during inference. Inference is less hard to make a chip for in terms of hardware and software complexity. For a transformer model, there are two kinds of inference: prefill and decode.

Inference: Prefill This is a specific kind of transformer inference where the AI reads text (or other data) that is supplied to it, but it isn't saying anything about it, it's just noting and remembering what is written. Prefill is less complex to implement in hardware and software than training. If you did the hardware and software right, your chip ought to be compute-bound during prefill, i.e. the bottleneck is the systolic array, not memory bandwidth or anything else.

Inference: Decode This is the other kind of transformer inference, where the AI speaks in the sense of creating output like text, audio, images or video. Decode tends to be more complex to implement than prefill, but not as complex as training.

Depending on how you implement it, decode requires more memory bandwidth than prefill and training. If your software is suboptimal and you do not have much memory bandwidth on your chip, you will be memory-bound on decode. You should aim to be compute bound also for decode, but you may not be able to achieve it. Even if memory bandwidth is not an issue, decode can create unbalanced matrix multiplications that do not get as high utilization on systolic arrays. For this reason, the current price in the market to buy a decode (output) token is much higher than for a prefill (input) token, even though the amount of mathematical operations to compute per token is almost the same between prefill and decode.

You will probably have noticed that AI assistants speak to you much more than you speak to them, so AI assistants have been heavy on decode and light on prefill on average. This is even more the case for reasoning AIs that "think" to themselves before responding - generating that "thinking" involves speaking to itself, which is decode. However, AI assistants are starting to do more and more prefill. That is because AI assistants are now searching the web (and other databases) and reading large amounts of information in order to figure out how to respond, an approach called Retrieval Augmented Generation (RAG). Due to RAG, which isn't going away, most tokens in the future are probably going to be prefill tokens rather than decode tokens. This has significant consequences for how to build AI chips. It means that systolic arrays are as important as ever.

So an AI chip is not just an AI chip. A **training chip** requires fast systolic arrays and advanced features that aren't needed for inference. A **prefill chip** requires fast systolic arrays. A **decode chip**, on the other hand, requires smaller systolic arrays and higher memory bandwidth than training and prefill do (at least for now - I believe that sparse (indexed) attention will greatly reduce memory bandwidth requirements for decode).

You will sometimes see chips marketed as specifically for training or inference, like Amazon Inferentia (for inference) and Amazon Trainium (for training). For inference, the industry standard as of 2026 is to do prefill and decode on the same chips and chips are not marketed

as being specifically for one or the other. So this means that systolic arrays commonly lie significantly idle during decode and memory bandwidth lies significantly idle during prefill.

Training

If you do an AI chip startup, should your first chip be a training chip? I would suggest no. Let other companies have this market. At least at first.

Training may seem to be a bigger market right now, reflecting there being currently more AI investment dollars (that go to training) than there are customer dollars (that go to inference) for AI services. As the field matures, clearly this will have to invert. There will be more money for inference, but also more competition - everyone else also started with an inference chip.

Training is a large market because it requires a lot of hardware to train very large LLMs. Many well-funded AI application startups buy vast amounts of training chips to develop their AI. They probably won't buy many inference chips because, at first, they won't have any customers for their AI to talk to. So the early phases of AI development involves buying a lot of training chips and no inference chips. Training chips can also do inference, just not quite as cost-effectively, so if you sell training chips, you may be able to keep a customer on your training chip even as they (hopefully) transition into having customers and therefore needing to do inference. Or you can transition them to your inference chip if you also have one of those. So it sounds smart to be a training chip company, doesn't it? Both Google TPUs and Nvidia GPUs can do both training and inference. So follow the industry leaders and make a training chip, right?

That's not how Google did it. TPUv1 was an inference chip, so Google didn't start out making a training chip, either.

The problem is that training requires significantly more complexity in your software and hardware than inference does. Training chips are also more expensive to produce per chip since they require additional hardware features that are not needed for inference. If you are a startup that may have to compete on price or go bankrupt, this will matter more to you than it does for Nvidia and Google - they probably have higher amortized profit margins.

What is needed for a training chip that you don't need for an inference chip?

High-throughput transposition The backwards pass of training requires transposing weights, which you will need hardware to do, otherwise it's too slow. This is usually not required for inference. You *can* use matrix multiplication to do transposition, so it *can* be done on existing inference systolic arrays, but this is not efficient - transposition units require *far* less power and area than a big systolic array.

It might seem that transposition hardware is required also during inference since the attention formula involves computing $Q \cdot K^T$, but you can get around that by having your systolic array do this operation directly instead of $Q \cdot K$, i.e. always multiply together rows instead of rows times

columns. You can still do all the other matrix multiplications in a transformer (and most models), since those are activations * weights ($A*W$), and you can transpose the weights offline instead of during inference, using that $A*W = A*(W^T)^T$.

Is there precedent for a systolic array that does $A*B^T$? Yes, that's how it is on TPUs, where the systolic arrays can optionally do either one of $A*B$ and $A*B^T$ (do read [the TPUv2 hardware paper](#)).

You may also be able to avoid permutation units for inference, but watch out for the data movement within multihead attention on this - if each head feature dimension is not a multiple of (e.g. equal to) your vector dimension, then you may need some way to do a permutation here. Convolutions may also be easier to implement on your hardware with some permute (and cross-lane sum) capacity. Convolution is difficult to implement well, harder than matmul. You may or may not need to implement convolution.

You may prefer to ensure that you won't have a problem regardless of the model by provisioning significant transpose and permute capacity also for inference, but it may not be *strictly* necessary if you keep to standard transformer models with vector-multiple tensor dimensions. I think I recommend having a permute unit anyway. Not sure about transpose. I do know you'll sleep better if you do have it.

Additional vector compute Training requires support for a variety of optimizers [like Adam](#). Such optimizers involve a significant amount of vector computation. This is in addition to what happens during inference, so you will need a higher rate of vector processing for training than for inference.

Expensive numerics All inference can be done using 8 or fewer bits for the multiplications inside the systolic arrays (the additions require more but are cheaper), which makes the systolic arrays cheaper. Even so, your product will be easier to sell if it also supports 16 bit numerics, even for inference, because some inference customers just can't be bothered to reduce to 8 bits. So it's a choice that you have as a company for an inference chip - 8 bit or 16 bit. For training, you're not likely to sell your chip to anyone if it only has 8 bit numerics, so it has to be 16 bits (even though I actually think 8 bits can be and will be enough for training, but that's not industry standard and you can't expect your customers to know how to deal with this currently).

Additional training ops and fusions Much of the work to write the software for an AI inference chip is to implement a large variety of ops being used for inference. This is already more work than most vendors would like to do, but it's necessary. There are even more ops being used for training and they tend to be more complicated than for inference.

Increased software and hardware flexibility The people training really large LLMs, who are the people you most want to sell a training chip to, are AI researchers. Whole teams of AI researchers coming up with new ideas every day. They are going to be constantly trying out new ideas. This requires a lot of flexibility in the software and hardware that you supply to a

customer who is looking to do large-scale training. Inference customers are unlikely to have as much of a need for flexibility of your software and hardware, even if they may still appreciate some of that. Also, some of those AI researchers aren't necessarily very interested in the computer science of using your product. It needs to just work even while they use it in strange ways. Large inference customers tend to staff engineers who are more interested in computer science (as opposed to AI research) and with a higher tolerance for software that isn't as easy to use but which provides their companies with tangible benefits in terms like tokens per dollar.

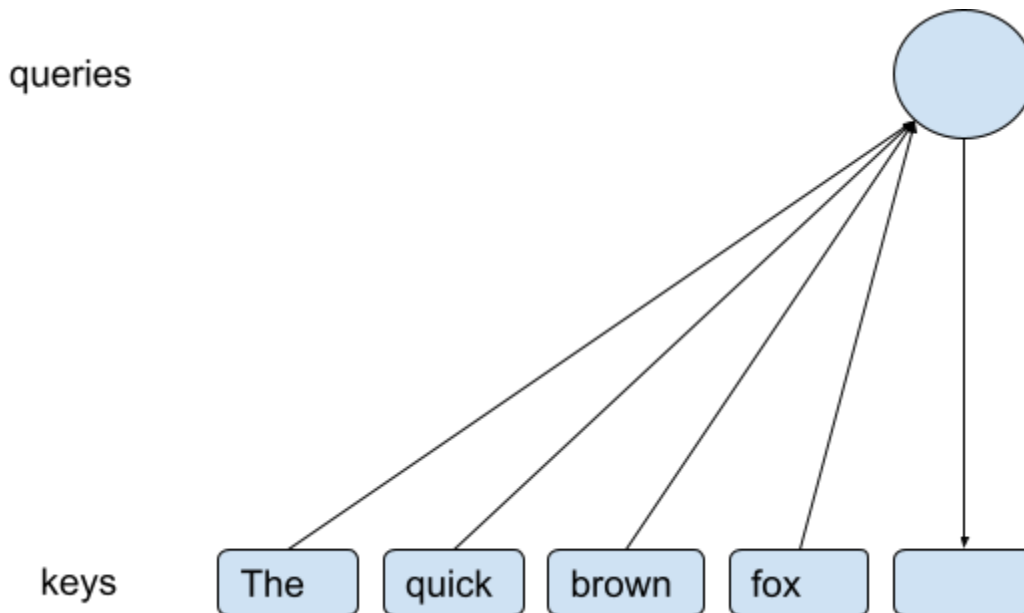
It's not that you can't do a training chip as your first chip. But do you want to? I'd suggest probably not. So, for this reason, I'm not going to go into as much detail on training in this document.

Decode

Decode is when a model has to say something - write text, generate an image, anything where there is so-called autoregressive output. Decode is the more troublesome kind of inference to support. It doesn't naturally fit well on a systolic array, but with enough AI research and software heroics, you can make it happen - so that's what you will have to do.

Recall that transformers contain two kinds of layers: attention and FF. Both are problematic for decode, but, of the two, decode attention is the more problematic.

The trouble with decode is that it, if done in the straightforward way, only generates one output token at a time. This is similar to how, when you write, you only write one word at a time. This is called autoregression, meaning that the model cannot start on the next token ("word", though a token is not always one word) until it has decided what the previous token is going to be. So it has to see ("regress on") its own ("auto" = self) output before it can proceed. To see why this is a problem, consider this diagram of how attention works during autoregressive decode:

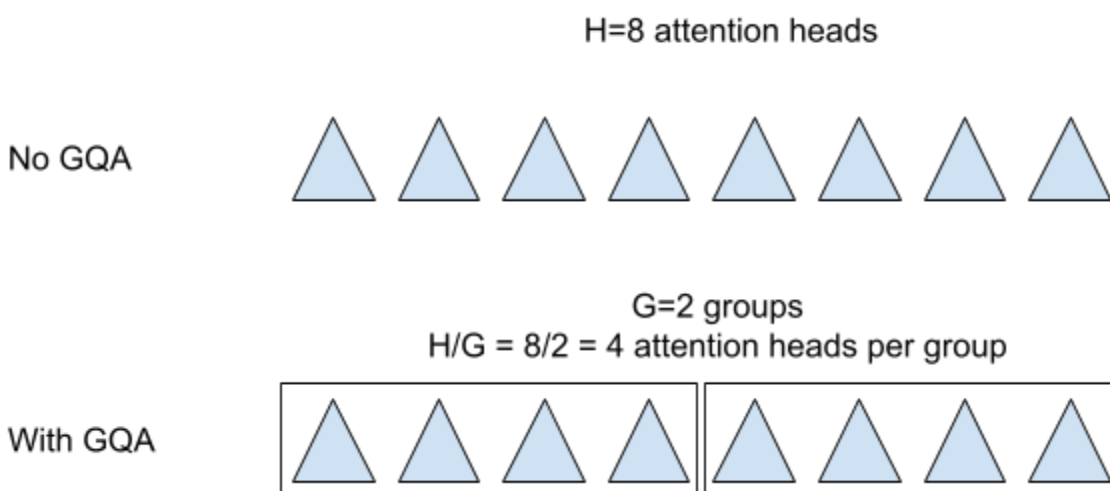


Each previous token publishes a *key* that is a vector that describes what that word is about. This was already done previously, so we are pulling these keys from what is called a KV cache (“key-value” cache - never mind what the values are). When we need to process a new token (looks like it is going to be “jumped”), we create a *query*, also a vector, that is compared to (dot-producted with) each of all the keys. Go read a proper introduction to transformers on the web if you want to know more of the details on transformer attention. The problem here is that this process corresponds to a matrix multiplication and, because we are processing only a single token, there is only a single query, which implies that the matrix multiplication has a dimension that isn’t just small, it is in fact equal to *one*. This leads to almost *no* utilization of the systolic array. So decode, if done naively, has very poor systolic array utilization, less than 1%.

Also, there can be many keys. Suppose you’ve been talking to your favorite AI for the past 2 years. If you are using an AI with what is called a long context window, and your AI is keeping track of your entire chat history, it might have to pull in as many as a million keys, just to compare each to one query vector - even if all that history is rarely relevant. This is going to require a lot of memory bandwidth, but meanwhile there is so little to do with the data that is read in from memory that the whole chip is just sitting there waiting for the memory to deliver a million keys. Worse, a transformer has multiple layers, e.g. 32 layers, so your poor AI might have to do all of this 32 times just to figure out that it should now say “jumped” as the next word. Then the *entire process* starts over from the beginning to figure out that the next word is going to be “over”. And so on, one token at a time.

Hold on, you might think. I heard somewhere that transformers use something called “batch”, i.e. a chip will process multiple conversations at the same time, and this is supposed to help with systolic array utilization. Unfortunately, batch is important for FF layers, especially during decode, but it doesn’t help much with attention, so that isn’t going to help us here. Batch might let you reuse some weights between conversations, but loading the weights isn’t the problem

that we are looking at here for attention - loading the keys is, and different conversations have different keys, so there is no reuse of keys from that.



Maybe you heard that transformers use multiple attention heads. That means that this entire process doesn't happen once per layer, it actually happens up to 128 times per layer, once for each separate attention head, each with separate key and query vectors that each reflect different properties of their tokens. By itself, this doesn't help the situation of low efficiency at all. However, it can help a lot in combination with a technique called [Grouped Query Attention \(GQA\)](#). The idea in GQA is to reuse the same key vectors across several attention heads in G groups of size $128/G$ (assuming there are 128 attention heads). Each attention head has only a single query vector, but now that we've combined the heads into G groups of $128/G$ attention heads each, that use the same keys within each group, we suddenly have $128/G$ queries across the same keys. So this means that the matrix multiplication dimension that we are looking at increases from 1 to $128/G$. This leads to much higher utilization of the systolic array. This has also reduced memory bandwidth consumption by a factor of $128/G$, since we have G sets of keys to read instead of 128. On top of that, since we are now reusing the keys $128/G$ times, it becomes sensible to increase the size of the key (and query) vectors, to improve their quality (they contain more information), which it turns out corresponds to increasing the K dimension of the matrix multiplication, which you might recall from an earlier section is yet another thing that will increase systolic array utilization during attention. As long as we increase the size of the key vectors by less than a factor of $128/G$, we've still gained a net benefit on memory bandwidth requirements. So GQA is a *very* important optimization.

We can do even better. It turns out that we don't really need to do just 1 token at a time, due to a technique called [speculative decoding](#). The idea is to make a cheap and informed *guess* about what the next few tokens are going to be, e.g. you might try to guess the next 3 tokens. In the case above, our guess about what comes next might be "jumped over the". Since the model now has a guess about what the words are going to be, it can start processing what will come after "the", even while it is also processing and making sure that "jumped", "over" and "the" are actually what it wanted to say. This can all be done in a single step, so now we are processing 4

output tokens at a time instead of just 1. This improves key memory bandwidth requirements and systolic array utilization by an additional factor of 4. The guess in question might instead have been “jumped over pineapple”. In this case “pineapple” is wrong, which means that we wasted time processing “pineapple” and also we wasted time figuring out what the fourth token should be, since that calculation assumed that the previous token was going to be “pineapple”, but that turned out to be wrong. So we are taking a risk here. As long as the quick guesses are often correct, this is still a benefit for both memory bandwidth requirements and systolic array utilization.

Why exactly is speculative decoding a benefit? Well since we are now processing 4 tokens at a time, and we are using G groups of $128/G$ attention heads each (assuming there are 128 attention heads in total), we end up with a total of $4 * 128/G$ queries that look across the same keys. So we have improved memory consumption and systolic array utilization by an additional factor of 4 and a total factor of $4 * 128/G$. If G is 8, then this is a total improvement of 64x.

The conclusion is that decode can still have poor systolic array utilization and that memory requirements can still be high, but decode is not nearly so bad as it seems at first sight because you can get this improvement by a factor of e.g. 64x by combining speculative decoding with GQA. If you have a 256×256 systolic array, you’d ideally get all the way to a factor of 256x, since we started at just 1, but such a large factor might not be available, so you can see how larger systolic arrays are harder to get good utilization from during attention and, especially, during attention decode.

Why don’t we pick $G=1$ to get a supremely fast decode (assuming there are 128 attention heads)? The trouble is that in this case there is only one set of keys per token, that all the attention heads have to share, and that isn’t very good for model quality. In other words, it tends to make the AI too much dumber, so that’s no good. Therefore we need enough G to preserve the AI’s intelligence but not so much that decode becomes too slow. This is a simplified description, but it gets the right idea.

Why not speculatively decode 16 tokens instead of just 4? That’s 4 times better, right? That is because the process by which we make these guesses is a very dumb AI that is much cheaper and faster to run than the proper AI - otherwise, the dumb AI would not be fast enough to be many tokens ahead. It is unlikely that it will be able to correctly guess so many tokens of what the much smarter proper AI is going to say. The dumb AI is going to blurt out “pineapple” (or something else wrong) long before we get to 16 tokens, and then the computation that we did for those tokens is wasted. So again there is a balance to be struck. Again, the full details here are highly complex (e.g. you can vary the number of speculated tokens depending on confidence in the guess), but this simplified view gives the right idea.

[[[Detail on systolic arrays for decode for nerds There is a third optimization that might get you that last factor of 4 to get to 256 and therefore potentially 100% utilization of even a 256×256 systolic array. You can do this by decreasing the required N dimension - see the section on

non-square systolic arrays. This is only relevant if you didn't use the idea to have smaller systolic arrays for attention and you also aren't memory bound. **]]**

So it seems that, with difficulty, we can probably get high utilization of a large systolic array also during decode. Many of these techniques also help memory bandwidth requirements. In general, anything that decreases the KV cache storage requirements have a similar effect on decode memory requirements, and there are *many* techniques that do this - too many to list here. The received wisdom in the field is that decode is highly memory bandwidth bound, even if you provision a lot of memory bandwidth, but the truth is that this may or may not be the case depending on how well you execute all of these optimizations. E.g. sparse/indexed attention can completely change this math - to the point that research into this may allow even SSD bandwidth to be enough for decode (!!!). The trouble is that if you do not control your customers' software and models, then they may not be doing an optimal job here. So then you will need incredible amounts of memory bandwidth to keep them happy, which is what Nvidia GPUs and Google TPUs have and this is one of the reasons for it. So how much memory bandwidth you need to provision is a difficult question. It depends on the software.

You should be noticing a trend here - pretty much everything about how to provision an AI chip depends on the software and you can end up with answers that are 10x or 100x different depending on how well the software and the AI model itself is set up to run efficiently.

Unlike for storage, parallelization does not change decode memory bandwidth requirements for reading KV cache (unless you could then fit it all in SRAM). Parallelizing across C chips may require each chip to pull in only $1/C$ as many bytes from memory per token, but one hopes that this results in processing C times as many tokens in the same time, so then it cancels out to be neutral for required per-chip memory bandwidth since $C * 1/C = 1$.

If you are going to sell an AI chip, you have to make sure that all of these techniques work well with your hardware and that you have the right software to support your customers in doing this - otherwise your chip will be off by a factor of e.g. 64x compared to your competition. This description is highly simplified compared to the software that actually has to be written on this, and this is just one aspect of many aspects that you have to offer software support for so that people can use your AI hardware. The software side of an AI chip product is quite challenging.

[[[Agent multiplication for nerds If you try using Grok's AI assistant (note that Grok and Groq are completely different AI companies!), you may notice that it says "agents thinking", plural, not "agent thinking", singular. The idea is to have multiple chain-of-thought agents that "see" (attend to) the keys for previous tokens and each other's tokens. This is a further multiplier, in addition (multiplication) to speculative decode, GQA and non-square systolic arrays. With for example 4 such chains of thought, you get yet another 4x on key reuse for attention. You also gain a 4x on the batch that can be profitably applied to a single user.

Doesn't agent multiplication only apply to parallel chain-of-thought, so we can't say it's a straight speedup, since we didn't speed up the part where the AI sends you a response? That's true, but

big chains of thought are where you really need a lot of tokens, driving the need for much higher per-user token throughput than you'd otherwise need, so improving "just" that part is still a big deal.]]]

[[[What about batch=1 for nerds Suppose you use 4x speculative decoding and 4x agent multiplication. That's 16 batch for a single user. This might suggest that one would never need a batch=1 product. However, while these factors certainly make higher-batch products more attractive, I don't think this completely eliminates the case for low batch.

Recall that Groq LPUs are for expensive low-latency tokens at low batch. The primary point of low token latency is to get high per-user token throughput. Now it appears that we've gotten that high per-user throughput anyway, or atleast 16x on that, without the overheads of batch=1 operation. So LPUs are useless? I've heard this argument brought up, but it isn't quite right. If you have 16x batch for a single user, you can buy 16x as many LPUs and also get a 16x on per-user token throughput for LPUs. So LPUs are still ahead for expensive tokens at high per-user throughput.

This can still change the attractiveness of LPUs, though. For one, you now need to buy even more of them to harvest this benefit, and you already needed to buy many to run a model at all. Also, it is often the case that there is a certain rate of per-user token generation that is fast enough. A 16x on this has the potential to move a more economical approach into the fast enough range, at which point it becomes unnecessary to buy an LPU. Also, the economic efficiency of alternatives increases with higher per-user batch, since you don't need as much bandwidth, so those expensive LPU tokens get relatively even more expensive this way.

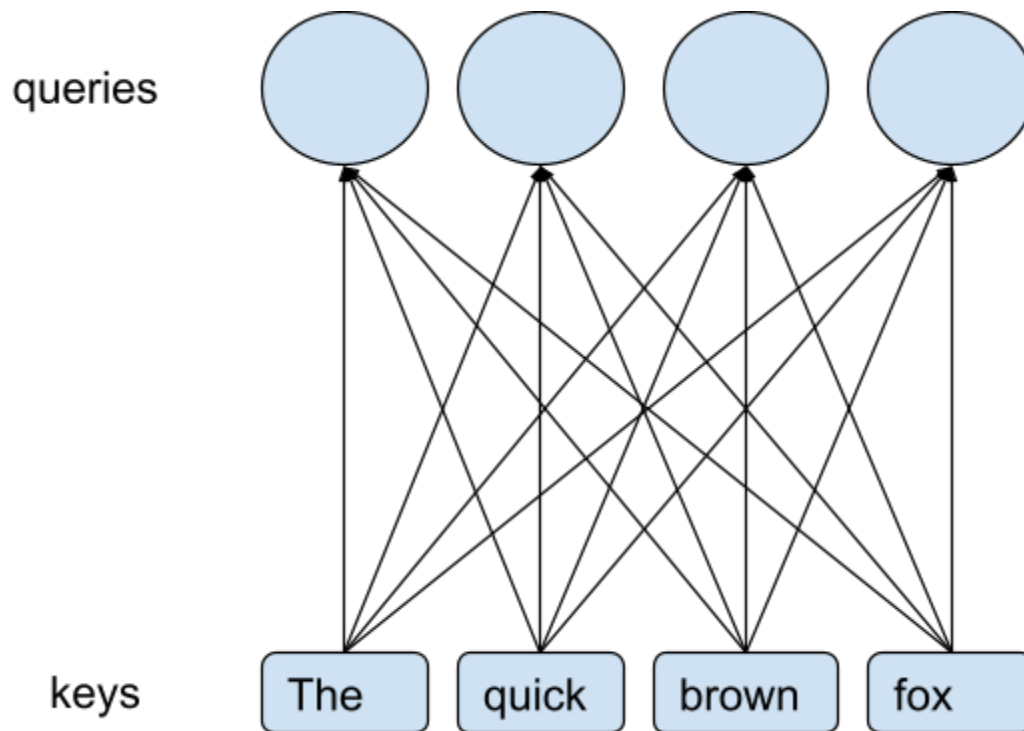
These points also apply to on-device / embedded use cases, where people often cite the need for batch=1, since an embedded device tends to only have 1 user at a time. This is unlike a datacenter application that services many users simultaneously and can find batch that way. Given the techniques in this section, we can see that you might have more batch available than you think, also for embedded use-cases. Embedded is still probably low batch for text decode, but even just getting to batch=4 is much more efficient than batch=1.

Another point to keep in mind is that this is for text decode. For images, typically diffusion models are used, which have plenty of batch. I won't explain diffusion models here, but you can look it up. There are also diffusion models for text, though they tend not to work quite as well. Still, if you find that you can use a text diffusion model, then that is another way to solve low-batch text decode woes.]]]

Prefill

Prefill does essentially the same thing as decode, except we are reading tokens that have been already written. So the AI is understanding text (or other data) that already exists ("(pre)filling the KV cache"), it isn't generating the text. That's important because that means we can process

all the words in the text at the same time (up to the limit of the chip's memory) instead of going one word by one word, so the picture becomes instead something like this:



Here you can see that where before we had only one query vector to process, now we can process all the tokens at the same time, giving us as many query vectors as there are tokens. It's much like speculative decode except we already know for sure what the next tokens will be, so there is nothing speculative about it. The input could be a whole webpage with thousands of tokens. So this dramatically improves memory bandwidth requirements, since every key that is read in from memory (or generated for the first time) is now being used potentially thousands of times every time we read it in. So we don't have the big problem of too few query vectors that we had during decode.

Note that the same benefit of plenty of query vectors applies also to training, since training happens also on text (or other data) that is already generated. The problems with decode is an inference-specific thing.

We can still benefit from GQA during prefill. For prefill, GQA will not help with the N dimension of the matrix multiplication that we were most worried about for decode, since that dimension is already large enough (we have enough query vectors) to get full systolic array utilization in that dimension during prefill. GQA *does* help with memory bandwidth requirements also during prefill, since there will be fewer keys to read with GQA, but prefill is less likely to have a need for any improvement on memory bandwidth compared to decode, so this is less important for prefill than for decode. However, attention also has a K dimension, as mentioned in a previous section, and that dimension also needs to be large enough to fill the systolic array to get full

utilization. This is where GQA can help the most during prefill. We mentioned in the section on decode that GQA is frequently combined with an increase in the width of the key and query vectors. This is another way of saying that we are increasing the vector width of the attention operation, i.e. we are directly increasing the K dimension, helping to get high (possibly 100%) utilization from systolic arrays during prefill attention. So GQA is important also for prefill.

Decode-prefill (managed) aggregation vs disaggregation

Default aggregation The default for decode and prefill is when you just do the obvious thing: do prefill and decode on the same chip using the same kernels. Mathematically, there is little difference between prefill and decode, they just have a dramatically different number for how many tokens are processed at once per conversation (e.g. 1-4 vs 256), so they can be combined and done simultaneously. In this way, the ratio of decode to prefill is variable and unmanaged - one chip might arbitrarily be doing mostly prefill while another does mostly decode.

Disaggregation Decode-prefill *disaggregation* is when prefill and decode are done on separate chips. It could even be (but doesn't have to be) different kinds of chips, since decode may require higher memory bandwidth. This requires transferring the KV cache from where it was prefilled to where it will be used for decode. The benefit of this is that it becomes possible to use software and even hardware that is specialized for just prefill or just decode.

Another issue that disaggregation can help with is that mixing prefill and decode can end up with the chip doing mostly prefill at times, since a prefill conversation/sequence can take much longer to process per step since it has many more tokens to process in one go. This can lead to increased token latency for decode, as the single decode token waits on potentially 1000s of prefill tokens to get done before it can run again. I've seen this cited as a benefit of disaggregation, though this would only really be a problem if the system that puts prefill and decode together in batches isn't set up to intelligently take this into account when creating batches.

Managed aggregation Decode-prefill *managed* aggregation is when you mix decode and prefill in a *managed* ratio. This is beneficial when decode is memory bound (i.e. waiting on memory) and prefill is compute-bound (i.e. waiting on systolic arrays). If you intentionally combine two such workloads together in the right ratio, you can arrive at a perfectly balanced scenario where both memory bandwidth and systolic arrays are fully utilized and neither is waiting on the other. This improves both latency (for prioritized tokens that go first) and throughput (for all tokens) compared to doing first just one and then just the other.

Managed aggregation is a term I just now made up, though in implementation this is very similar or even the same to something evidently called "chunked prefill". The implication of managed aggregation is that you do not need as much memory bandwidth on your chip if you expect this technique to be used. It's not just about preventing a chip from being swamped with prefill work.

Reaching balance The overall ratio of decode to prefill might not reach perfect balance across your AI system, in which case it's not possible for all chips to receive a perfectly balanced workload. However, managed aggregation can still avoid cases such as having one chip doing mostly prefill (unbalanced one way) while the chip next to it does mostly decode (unbalanced the other way). A company could set their pricing in a way that roughly achieves balance, at least for offline tokens (offline as in "you get the answer when you get it, not necessarily right now") - many companies already price prefill (input tokens) at far lower prices than decode (output tokens). Also, if you have an infinite source of decode or prefill work that you generate yourself (automatically generated work could involve e.g. mining for inconsistent answers from your AI - e.g. answers should flip on question negation), you could use that to balance your system, doing more or less of either one depending on the balance at that moment. This also helps to ensure that your system is fully utilized at all times. Another option is to combine decode with training, in addition to prefill, since training also tends to be compute-bound (though not necessarily as much as prefill is). The important thing to keep in mind is that managed aggregation is beneficial even if it isn't possible to reach perfect balance.

Managed aggregation leads to a minimum required ratio of prefill to decode to ensure that you are always compute bound instead of memory bound. You can make this ratio requirement more forgiving by also overlapping attention with FF - FF likely has some idle memory bandwidth that you can make available for attention by overlapping them. This works best if there are 2 or more cores per chip (for some reason called SMs on a GPU). If there is only 1 chip, and you try to multiplex attention and FF onto it simultaneously, then these two will compete for cache space which may not be a benefit in that case. To enable this overlap, you will either need pipelining with attention and FF in separate stages of the pipeline or a model change that makes attention and FF occur in parallel instead of one after the other. The latter technique also halves token latency.

A hybrid system is also possible - in this case your main server farm would be set up for near-perfect balance, but you also have a smaller set of prefill or decode-only chips (whichever you sometimes have more of) that handles unbalanced overflow.

If you built a chip that was intended for managed aggregation in this way, you could even provision it with less memory bandwidth than you'd otherwise need, lowering production cost. This is a bold (read: risky) move, since maybe not all your customers will be using managed aggregation, and if so they might see worse performance due to this choice. This is yet another example where the product that an unsophisticated customer needs is different and more expensive from what is right for a more sophisticated customer.

Expensive HBM storage capacity

AI chips ship with large amounts of expensive RAM called HBM. Why is that? Is that really necessary? That's the topic of this chapter.

Weights and KV caches as reasons for large expensive HBM memories

HBM is a *very expensive* kind of RAM, i.e. computer memory. Estimates vary (companies won't tell you what they pay for HBM), but HBM might be 3x as expensive per GB of storage compared to regular RAM. Regular RAM is already itself expensive in large quantities. HBM offers high bandwidth, and HBM is *not* so expensive in terms of \$ per GB/s of bandwidth. It is a GB of HBM *storage capacity* that is very expensive. So you might think that AI chips would ship with small HBM memory capacities that still offer high bandwidth. Is that happening today? Not *at all*. Instead, the huge expensive HBM storage capacities on modern AI chips are getting larger and larger. Nvidia has announced that their Blackwell Ultra GPU will launch with an amazing 288 GB of *particularly* expensive HBM (namely HBM3e - the "e" doesn't stand for "expensive", but it's fine to think of it that way²). As a customer, you can maybe see how you could get excited about 288 GB of memory - until you remember that you're the one paying for every GB of that.

If expensive HBM is so expensive, why do AI chips ship with so much of it? It's mostly to store large KV caches and model weights. What is a KV cache?

In the previous sections on decode and prefill, we were looking at many key vectors. These key vectors are stored in a data structure called a KV cache, short for key-value cache. Never mind what the values are - a value vector is some vector, associated to each key vector, that is used for purposes that we won't get into here. We will continue to focus on the *keys*.

Part of the trouble with the KV cache is, as we saw in the section on decode, that it can take a lot of memory bandwidth to inspect it (i.e. pull it into memory). Another related issue with the KV cache is that it can take a lot of memory capacity to store it. Why would that be?

The question is how many keys (and values) we have to store and how much space that will take up. Here is a formula for how much space a KV cache takes up, where each quantity in the formula is explained below.

$$2 * \text{batch} * \text{layers} * \text{seq_length} * \text{attention_head_groups} * \\ \text{vector_width} * \text{element_size} * \text{idle_magnification} * \text{pipeline_factor}$$

This will turn out to be potentially a *very large* amount of storage, driving the need for large expensive HBM memories.

Value vectors x2 The formula starts with a multiplication by 2 to account for value vectors. This is because there are as many value vectors as key vectors and value vectors take up the same amount of space as key vectors do (you can do GQA in a way so that this isn't true, but

²HBM3e stands for "High Bandwidth Memory 3 Extended". Maybe it's "extended price tag", because it requires extra digits?

normally it would be the case). I didn't explain what value vectors are, and it doesn't matter for our present purposes, but they do have to be stored, so we have to multiply by 2.

Well OK, I can explain value vectors a little bit: Value vectors are the data that keys are used to find. So if a KV cache were a phone book (which it isn't), the key vectors would be people's names and the value vectors would be phone numbers - a phone book is only useful if you have both. So a KV cache contains keyed (searchable) information (values) about past tokens.

Layers $L=32$ You might recall that a transformer has multiple layers L , e.g. $L=32$. Very large models can have significantly more layers than 32, e.g. ~ 100 , but we're going to go with 32 for this calculation. 32 layers is already not a small number of layers. Each layer has its own keys, so we need to multiply the number of keys by $L=32$.

Sequence length $N=1,000,000$ How far back can our AI remember? This is called the sequence length N or attention window length N . This can be as high as a million, so to understand the impact of what kv value storage can be, let's say that $N = 1,000,000$.

Note that one may let some attention heads use a much smaller N , which is then called local attention. In this case, it is the average sequence length N that matters, not the maximum. People also at times use entirely different attention approaches for some or all of the attention, e.g. linear attention. In that case, the math ends up being different from the formula here.

Batch $B=64$ A batch of B means that that the AI is handling B independent conversations at the same time. Batch is required to get full utilization of the systolic arrays during decode for the FF layers of a transformer - we've been focusing on attention layers here instead of FF, but FF is also needed. For a 256×256 systolic array, you are going to want a batch of 256 to keep the systolic arrays busy during decode. However, if you are using speculative decoding with, say, 4 tokens (i.e. 3 tokens speculated), then you can reduce that to $B=64$, getting that factor of 4 from speculative decoding instead of batch. So we need to multiply the number of keys by e.g. $B=64$. Note that during prefill and training, we may not need to use any batch, or only very little, since, during prefill and training (but not decode), the sequence length can serve all the purposes that batch otherwise serves for FF layers. However, an AI assistant that does only prefill or only training will never do or say anything, i.e. it will output no tokens, so we need decode and therefore we need batch. Training and prefill may also need batch, but only if the sequence length is small.

Attention head groups $G=8$ How many attention heads are there? For a large model there can be e.g. 128. Without GQA, each head will have its own set of keys, so we would need to multiply by 128. However, with GQA, these heads are grouped into groups that share keys. There might be as few as 8 groups, so that we only need to multiply by $G=8$. So you can see how GQA is important also for reducing the storage required to store KV caches. GQA can be done so that the values are still distinct, leading to increased KV cache storage - for this calculation, we'll assume that the values are shared within a group.

Vector width W=128 How wide is a key vector, i.e. how many elements (individual numbers in the vector) does it have? For a large model you might have W=128. Let's go with that.

Bytes S=1 How many bytes S does it take to store *one* element (number) in a key vector? Many LLMs use 16 bit KV caches, since this is easiest, but let's say that the creators and/or deployment team for our LLM put in the effort to get it down to 8 bits (as they *really* ought to do for anything serious). So that is S=1 byte per element (there are 8 bits in a byte). Using 4 bits is also possible, and there is research on using even fewer bits, but this can degrade model accuracy, i.e. make the AI dumber. It might be that the march of improvements to AI methods will eventually get us down to 4 bits here as an industry standard practice, but that isn't true today. Though see the section on compression - this can help here.

Idle magnification M=1.0 Idle magnification (a term I just invented, I don't know of a standard term for it) accounts for the fact that there could be many more conversations in expensive HBM memory than we are currently doing calculations with. So they are currently idle. This can happen e.g. when you are spending time reading the response from an AI assistant. During this time, the AI is waiting for your response. The conversation is not being processed during the wait, since nothing is happening. However, once you respond, you will prefer an immediate or at least a quick response from the AI. If the conversation and its KV cache is ready in memory, then the AI can pick up processing it quickly. If it is stored on an SSD somewhere, then it will take a bit of time to read it back into memory before the AI can get started on it, which slows things down. I'd recommend to get faster SSDs instead if this is a real problem, but storing idle conversations in HBM is a technique that an AI assistant might use. If only half the conversations in memory are currently being processed, that represents a KV cache idle magnification by a factor of M=2. We will assume M=1, so no magnification. To be clear, when your AI assistant waits for a minute before it responds, saying it is "thinking", that isn't likely due to reading KV caches into memory, which shouldn't take that long - what it is doing is speaking to itself (generating tokens, i.e. decode) and looking at data like websites (prefill) to figure out how best to respond to your query.

You may also have an effective reduction in capacity due to fragmentation of the HBM. You can view this as also a kind of idle magnification, though it's hard to put a specific number on that.

Full pipelining P=1.0 Full pipelining can multiply KV cache memory usage by up to 4x - see the section on parallelism. For this calculation, we will assume that this has not been done, but probably you should do that (it's not good for KV cache storage requirements, but it's great for compute utilization and network bandwidth requirements).

So how large is the KV cache with these numbers?

$$2 * B * L * N * G * W * S * M * P = 2 * 64 * 32 * 1,000,000 * 8 * 128 * 1 \text{ byte} = 4,194 \text{ GB}$$

That is a *lot* of expensive HBM! Large KV caches like this are a big driver of putting a lot of expensive HBM memory on AI chips. To be fair, a sequence length of N=1,000,000 is large, but

this *is* a product that AI assistant companies provide, so this calculation is yielding a real number.

Recent Nvidia GPUs and Google TPUs have on the order of 100 to 200 GB of expensive HBM memory. They have so much expensive HBM memory in part due to needing to store large KV caches. However, we can see that we cannot store 4,194 GB in just 200 GB of memory, so how is this going to work? The answer is that LLMs are parallelized across many AI chips, and the KV cache is distributed across many chips, so it's not about how much storage you can find on one chip, it's about how much expensive HBM memory there is all together across many chips. You could distribute within the layers of a large LLM across e.g. 32 chips, in which case you need $4194 \text{ GB} / 32 = 131 \text{ GB}$ per chip for the KV cache. So you can see how you might end up needing 200 GB of expensive HBM per chip. This is a simplified view, but overall this gives the right idea.

What about weights, don't LLMs have a lot of weights, so presumably this requires a lot of storage, too? It does, but if there are 1000 GB of weights, you can distribute that across all the chips, so if you parallelize by 32 within layers and pipeline parallelize across all of 32 layers, then you are using $32 \times 32 = 1024$ chips. If you distribute 1000 GB of weights across 1024 chips, then you only need to store 1 GB of weights per chip. So that's not much. Weight storage becomes a problem if you aren't parallelizing very much, but not otherwise. Note that KV cache storage is improved by parallelizing within layers but *not* by parallelizing between layers (pipelining). In contrast, weight storage requirements are reduced by both kinds of parallelism. See the chapter on parallelization for more detail on how parallelization changes the per chip requirements for KV cache and weight storage.

Reducing the need for large memories on AI chips

The previous section arrived at the need for large HBM memories on AI inference chips due to needing to store large KV caches and, also, due to needing to store large weight matrices for large LLMs.

Something that I want to complain about here is that we are using $N=1,000,000$ for this calculation. AI assistants with such a long attention window is a real product that is being sold, so it needs to be supported, but couldn't this be done in some more economical way? There are in fact *many* ways to resolve this problem with minimum loss of model accuracy, by turning attention into retrieval - here is [one paper for that](#), you can find others. I also [posted on LinkedIn](#) about this. Ultimately I believe that we need solutions that may involve dense attention for the recent past, e.g. the past 1024 tokens, but then turns to sparse retrieval for the rest of attention history, which can then be stored on an attached SSD or perhaps even further away out in the datacenter. In this way you can reduce memory requirements from $N=1,000,000$ to $N=1,000$, an improvement by a factor of 1000. You still need to buy a large SSD, but SSDs are much less expensive than HBM per GB. So we go from 4,194 GB to 4 GB by reducing to $N=1000$. There are other ways of optimizing KV caches, so if you apply such ideas, too, then you can get even

smaller KV caches than that, and this is *before* distributing the KV cache via within-layer parallelism. So why isn't this happening?

Well why do I think this isn't happening? Wouldn't companies use these ideas of sparse retrieval for attention? Perhaps some are, but the fact is that recent Nvidia GPUs and Google TPUs are being sold with extremely large HBMs and the story is that this is to store large KV caches. So I think that the AI industry is causing the current world-wide ram shortage unnecessarily. We don't really need all that expensive HBM after all.

If you are an AI chip startup, this seems like a relevant topic to figure out. My strong suspicion is that you can do a 32 bit chip that just has 4 GB of memory (32 bit pointers can address 2^{32} B = 4 GB values), instead of the 100-300 GB that Google and Nvidia are currently selling, and if you get the software exactly right and parallelize massively then your cheap chip will be just as capable as these super expensive industry leading chips. "All you need to do" is to use per-layer retrieval for attention (past recent history) and to parallelize enough so that you can store the weights of a very large LLM in a distributed way. Current-gen HBM cannot be bought in such small quantities per stack as 4 GB (at least it looks this way from public sources, though if you ask maybe it's possible). Once you take interposer and other costs into account, HBM is not necessarily the most economical memory technology on a GB/s per dollar basis (!), so this isn't necessarily such a problem as you might think. Though moving away from HBM is likely going to be a better fit for smaller chips in a fast on-chip network, which will require greater discipline of per-chip cost minimization.

I do see two problems here. The first problem is that this way the minimum number of chips you need to have in order to load a big model is going to be large. A 1 TB model (which is a huge model) is going to require parallelization across *at least* 256 chips in order to run if they only have 4 GB of memory each. You can see how that's inconvenient, especially if each of those 256 chips is expensive individually (though if you follow my ideas, they won't be). But a 1 TB model is very large and you'll want to parallelize within layers for speed (token latency) in any case. For serious purposes of e.g. prominent AI assistants, having a large minimum parallelization requirement is not such a big deal, since prominent AI assistants require a lot of throughput anyway, leading to a large installation regardless. You can of course go with 16 GB or 32 GB per chip instead of my extreme 4 GB - I'm putting the idea to its limit to highlight what is possible.

There is also a second problem. Above, I silently assumed that it is *me, or someone like me*, who is writing the software and deploying the model and ensuring that an SSD is used and setting everything up so that, [just precisely, everything works out](#) even with small amounts of HBM. Customers who are highly sophisticated about deploying an LLM may be able to do that for themselves. However, it is quite likely that *many* of your customers will *not* be highly sophisticated about deploying AI models, but they might still be willing to pay you many millions of dollars to buy your chips. So you end up doing very expensive things with your hardware to serve customers that wouldn't really need all that expense if they were doing things more efficiently in software, *but they aren't going to do things more efficiently in software, so it doesn't*

matter how it would be if they did. I suspect that this is the real explanation of the huge expensive HBMs on recent Google TPUs and Nvidia GPUs. I suspect it's an effort to make things easier for less sophisticated customers (internal or external), even though it isn't completely necessary. And I think this is in turn happening in part because of the high profit margins on these products. It doesn't make that much of a difference to the bottom line to make a high-profit AI chip more expensive to produce, but it does in that case make a difference to acquire more customers.

So you can see that there might be an opportunity for a chip company that is more hands-on in helping their customers to arrive at efficient solutions in a standardized way that they can roll out across all of their customers. There may also be an opportunity for a chip startup to pivot into directly providing AI assistant services at low prices, where they can be their own customer for their own chips and drive down costs using their own excellent software. This seems to be the strategy that Groq is pursuing with their LPU chips (at least the use-their-own-hardware part). Both roads seem difficult, but I think interesting.

An objection here might be that while HBM is very expensive, the chips that Google and Nvidia are selling/renting out cost a king's ransom as well. Very, very expensive. So if HBM would be e.g. 20% of the cost that leads to whatever the poor³ customer paid (this is a guess - expensive HBM prices paid by large companies are not public, though I've seen estimates as high as expensive HBM being 50% of the cost), then it might not make much sense to make a less convenient product with less HBM to save just 20% on production costs. So maybe the HBM cost isn't that important and not a place to focus currently?

I think that this kind of argument can often be in error in a non-obvious way. Why? Because I believe that you will find this kind of bloat in many places in these products, so that if you remove it all at the same time, the impact is far greater than a measly 20%, but if you remove each type of bloat individually, it will look like it is "only" 20%. And this is how all the bloat stays where it is - you can't decide to remove one kind of bloat because it gets justified as being only a smallish percent (like 20%) of all the other bloat that you also can't decide to remove *for the same reason*. Bloat protects bloat. For example, why do TPUs (or GPUs) need a host, a separate computer into which you connect the TPU/GPU. Why don't TPUs just have some on-chip CPU cores ([Google already makes excellent CPUs](#)) and then the TPU *is* the computer. I happen to know that these host computers are pretty expensive. But perhaps the host is only 10% (I don't know the number) of the overall cost, so why change it? And so on.

I suspect that Google could make great inroads in the AI hardware market in a year or two by getting extreme and creative with lowering their TCO (total cost of ownership) at a realized tokens per dollar basis, while offering excellent software, and then also lowering their prices. I don't think they will do this, [in part because they don't have to](#), and I definitely don't think Nvidia

³ Wouldn't the poor customer have to be rich to be able to pay these price? Well, yes, the customer was rich, but they might not be rich any more after paying these prices. So I guess it should say "now poor" instead of "poor".

will, either, so that leaves opportunities in the market for other players to become the biggest company in the world instead of Nvidia.

[[[Per chip performance matters for HBM economics for nerds Something to be aware of here is that you should not compare HBM capacities directly between two chips to understand cost effectiveness of HBM, instead you should compare something like the ratio of per-chip HBM capacity to the per-chip realized performance. To see why this is, suppose you made a chip that is half as fast as Blackwell and also it has half as much HBM. Did you improve the economics of the HBM memory in this case? No, not at all, it's exactly the same, because you'll need to buy twice as many chips to get something equivalent to a Blackwell, so you ended up buying exactly as much HBM as for Blackwell to get a comparable system. The same is true for other capacities on a chip than HBM. This factor is likely one of the forces leading to ever-larger chips that we see in AI accelerators. Though, in the other direction, if you start having multiple separate memory interfaces that make your memory spaces highly non-uniform, then you've essentially taken separate independent components and simply labeled them as one thing, but the label doesn't change the math of what you did, though you may have a fast enough network on there that it isn't *just* a label - in this case the math of the actual economic impact of this arrangement becomes complex to evaluate.]]]

[[[Cautionary tale: The attitude that cost a trillion dollars Back when I worked at Google on TPUs, in the TPU v1-v4 era, there existed a subtly strange situation in the company the entire time from what I was able to read between the lines over the years. From the outside, it might appear that Google was prescient in developing TPUs as a reasonably well funded effort so early as they did. Yet, on the other hand, there was internally a subtle yet clear and ever-present sense of what I'm going to describe as "neglect by unambitiousness and non-shareholder-aligned interests." I cannot know if that persisted after I left the company, but my impression looking in from the outside is that it did.

What happened with TPUs is not so much leadership that asked for the moon and received what they asked for. In fact, it was not like that at all. TPUv1 was motivated by "if Google will ever speak to you, as an assistant or otherwise, it's going to be too expensive to run the speech synthesis on current hardware, so let's do something to make it less expensive for us." TPUv1 was not rolled out widely within Google, though it did succeed significantly beyond its originally intended aims. TPUv2 was intended to deliver on precisely two internal training applications having to do with search and ads, respectively, and there was *explicitly* no ambition for anything beyond that. Once TPUs ended up becoming available for general internal use anyway, there was *explicitly* no ambition for making it available to anyone outside Google in Cloud or in any other way. Once it got into Google Cloud anyway much later, the effort for this was very small and with an unbelievably poor result if you care about Google shareholder interests - there was no ambition to make TPUs a drop-in replacement for GPUs, there was only the will to make TPUs available at all, and the investment into that effort itself was very meager. I didn't look too closely, but it appeared to be handled overall by one person who seemed personally capable but also seemed tasked with "just get it working somehow" (that's not a quote, it's just my impression). I got the sense that this was something that someone somewhere decided to have

happen, but not really something the company overall cared for, so it was made to happen, but not well. The result was that the way you start and run a TPU in Cloud was *different* from how you do it with a GPU, so even though XLA itself made it often possible to switch between GPU and TPU by simply changing one line that says “GPU” to instead say “TPU”, it was a somewhat more involved story for a customer to make that happen in Google Cloud, requiring more changes at a technical level. An unforced error that is hard to explain. That’s not what you would have done if you wanted to sell TPUs in Cloud *in earnest*. Since Google wasn’t selling TPUs, Google Cloud was the only way for external customers to access them.

Meanwhile, it is obvious today that Google was sitting on a trillion dollar gold mine (and may still be), and this is also what I believed back then, which was very strange for me at the time. Google as a company viewed TPUs as a way to save some money internally on AI hardware, which was a success, but Google has *never* been “all steam ahead” on making money from TPUs and I don’t see that it is today, either. There presumably is a reason for that, but I don’t know what it could reasonably be. It certainly was never explained to me when I worked there. If TPUs will not be a big part of the future of Google’s profit, it’s because Google chose that outcome as a management decision. Nvidia is the top company in the world by market cap and that could easily have been Google up there if there had simply been the will for that, I believe.

So how come Google is making a fair bit of money from TPUs these days despite all this? Google hires strong engineers and, as far as I can explain from where I was in the company, what happened with TPUs is that the bottom level of the company delivered beyond expectations for an extended period, leading Google to have a success on their hands that was never expected nor requested. Google never wanted to be an AI hardware company, it primarily wanted to minimize its own expenses on AI hardware, so what do you do in a company like that if you suddenly are sitting on potentially the most profitable project in human history? (Nvidia is the #1 company in the world by market cap, so this potential was/is factual) Evidently, you handle it with lukewarm enthusiasm - that’s what it was and, it appears to me from the outside, still is.

I once pressed a very high up manager on this, in my (rather long) reporting chain, asking why we weren’t all steam ahead on TPUs the way Nvidia was for GPUs. The initial response was “but that’s Nvidia’s main product”, I said “yes, that’s what I mean” (I could have found a better response here, I didn’t mean that Google should abandon search as our main product, although in hindsight that might have made more money for shareholders) and then I got a surprising response delivered in a *highly hostile manner*: “**are you for real with this question?**” shutting down the conversation. That’s verbatim - I will never forget that sentence. This reinforced my impression that the unambitious approach didn’t originate from my close-by reporting chain, but rather somewhere much higher up.

I was a very prominent software engineer on TPUs, I led TPUv3 software at Google and was closely involved in TPU hardware/software co-design for an extended period, yet I was *never once* asked a single question that had to do with how to be more ambitious with TPUs. I also was never once presented with any kind of statement along the lines of “if you/someone/the

team can deliver X, then maybe we will do more with TPUs,” it was always just “no, we’re not doing more with TPUs.” The ambition and desire to make real money from TPUs just wasn’t there when I was there. Looking in from the outside now, I still don’t see it.

This is in contrast to Nvidia - *they* are full steam ahead. I know because I also worked at Nvidia for a time. Google doesn’t appear to be - which is wonderful news for you if you are looking to make a competing AI chip. I was never a Google director or vice president and I was not in those kinds of meetings, so I can’t say for sure what happened in those meetings, I can only report what things seemed like to me from the bottom.

I still find all this unfortunate because I believe that Google’s unambitiousness on this wasn’t only bad for Google shareholders but also bad for humanity. We need affordable and high quality AI and that has to start with AI hardware that has much better economics than what is available in the market today. Therefore, we need full-steam-ahead competition to Nvidia that Google doesn’t appear to want to provide. Other companies will need to pick up the slack and indeed there are many AI hardware companies by now. I hope to have helped with aiding that competition by writing this document to the benefit of the public.

I reached out to press@google.com for any comments they may have on this cautionary tale in particular. The automated response I got said: “If you are not a member of the press or a Google employee, you should not expect to receive a response beyond this email,” which, indeed, I haven’t.]]]

Parallelization of an AI assistant

This chapter gives some big picture ideas for how to parallelize an AI assistant (i.e. an LLM). This section is already somewhat complicated, and I can assure you that actually implementing these ideas at high performance is more complex than what this section makes it seem like.

If you are just looking for some light reading, you may prefer to skip straight to the co-design chapter - that one is much lighter reading than this section and the next one on AI chip networks.

To be clear, “parallelization” refers to distributing the calculations of an LLM across multiple chips and/or multiple cores within a chip. So if you have two chips, ideally things should go twice as fast. That’s parallelization. Parallelization is the reason why per-chip performance is irrelevant on its own, despite much (most?) AI chip advertising focusing entirely on this irrelevant metric. What matters is what your system can deliver at what price, not what an individual chip can deliver.

For an AI chip startup, it is critical to understand the precise impact of parallelization, which in my opinion is a highly tricky and *unintuitive* subject. Here are three facts about AI parallelism:

- 1) Network bandwidth is a limit on within-layer parallelization. This is why AI inference needs a fast network. Between-layers parallelization does not require nearly as much network bandwidth.
- 2) If you double the network bandwidth, then you may be able to do twice as much within-layer parallelization, which can as much as halve token latency. So if you have a problem with high token latency for your AI chip, as you might have if your chip requires high batch due to using large systolic arrays, then you can improve that by increasing the network bandwidth. So large systolic arrays are a force for faster networking.
- 3) Between-layers parallelization, i.e. pipelining, helps with weight storage requirements, but it *doesn't* help with KV cache storage requirements. By contrast, within-layer parallelization *does* help with KV cache storage requirements, which, again, is in part limited by network bandwidth. So if you double network bw, you might be able to roughly halve per-chip HBM storage requirements if you've already parallelized enough to make weight storage negligible. So network bandwidth is related to how much HBM you need.

If you use Mixture-of-Experts, as is commonly done, then all this becomes even more complex. MoE is half-way a kind of within-layer parallelism, but it uses the network differently.

Another fact is that if you double the speed of your chip, then you also need a twice as fast network to keep up. That's both obvious and intuitive, but it's important to keep in mind.

In this chapter, I explain, among other things, why the above 3 facts are true. They are true for non-obvious reasons. I personally find these 3 facts highly surprising. If you didn't already know these 3 facts, and you are involved in making or buying AI chips, then I suggest paying close attention to this section. Whatever calculations you make that involve parallelization, I suggest to double, triple, quadruple check that you actually understood the topic correctly. Otherwise your math will be meaningless. *Many* highly intelligent people have gotten this wrong - you wouldn't be the first. For example, initial TPU networking was grossly overprovisioned due to getting parallelization wrong.

You can't dimension the HBM or network bandwidth or consider the token latency without understanding parallelism, so it's not optional to get into in detail when designing an AI chip. That is why this section is rather long and gets into quite a few details.

Parallelization concepts

There are a variety of concepts related to parallelization. These concepts make it possible to understand the pros and cons of various parallelization methods. It's a bit of a list, but the effect of parallelization will not be clear without these concepts, so here's the list:

Throughput (tokens per second) This is the most obvious benefit of parallelization.

Throughput for an LLM can be measured in tokens per second. If one chip can generate 1000 tokens per second, then two chips can generate 2000 tokens per second.

First token latency (seconds to first token)

First token latency is the time it takes an AI to generate the first token after it starts to consider your request. First token latency can be high because the AI might have to retrieve data from disk in order to remember what the conversation is about, it might need to silently (i.e. in a way that you can't see) talk to itself ("think") at length, and read many web sites and other data, to figure out how it wants to respond. The AI can't start talking to you until it knows roughly what it wants to say. So when you wait while the AI is "thinking" before saying anything, that's first token latency. First token latency is not a parallelization-specific concept, but it can easily be confused with token latency, which is an important concept in parallelization, so I've defined it here separately just so I can say that this is NOT what "token latency" is.

Token latency (ms per token) Token latency is the time (latency) it takes to generate one token in a steady state. It's not to be confused with first token latency, since the first token can take much longer than other tokens. So if your AI's response to you starts going, and then you get words one at a time pretty slowly, e.g. one token per second, that's high token latency.

Token latency and throughput are not so closely related as it might seem. If throughput is 1000 tokens per second, then I can expect to get a 1000 token response in a second, right? No, that's not how it works *at all*. The reason it doesn't work that way is that throughput is counted across conversations, so if there are 1000 conversations going on simultaneously, with 1000 different customers waiting for the AI to get done responding, then they might each be receiving 1 token per second. That is still 1000 tokens per second from the AI, even if you personally don't receive tokens that quickly.

Well, that makes sense, but what if I get all the other users to stop using the AI, so it's just me, *then* I can get 1000 tokens per second if the throughput is rated at 1000 tokens per second, right? That sounds like it makes sense, but it's still wrong. The AI might only be able to generate 1000 tokens per second if they are all from independent conversations. You are only offering one conversation, so you will in this case not likely be able to receive the full throughput within that one conversation. In fact your token latency might be the same regardless.

What if token latency is 1 ms (1/1000 second), then surely throughput must be 1000 tokens per second. Right? Not quite right. Throughput will be *at least* 1000 tokens per second if token latency is 1 ms, but it could be much higher. Token latency is still 1 ms regardless of whether the AI generates 1 token per 1 ms or 2 tokens (from unrelated conversations) per 1 ms. In the latter case throughput is 2000 tokens per second, not 1000 tokens per second.

The conclusion here is that latency and throughput are individually simple concepts, but it can be difficult to get used to the idea that the two are not related in any simple way in general. It is not the case that higher throughput necessarily leads to lower latency or that lower latency necessarily leads to higher throughput. In fact it is often the opposite.

I may have just now given you a frustration that you may not have had until now: being frustrated with material that talks about one out of throughput and token latency, but not the

other one. This makes the numbers meaningless since you cannot possibly know whether a system is both economical (throughput) and useful (token latency) without knowing something about both. The sad thing is that almost ALL material talks about only one of these and not the other. So welcome to this frustration, now that you know. You're welcome.

Variability Overall throughput and latency are *averages*. So a given token may take much more time or much less time than another one. In general, variability is highly undesirable when doing parallelism but it cannot always be avoided. Mixture-of-Experts is a prime cause of variability.

Storage Storage may not seem like it would be closely tied to parallelism, but it turns out that it is a very important concept in AI Transformer parallelism because parallelism can dramatically increase or decrease per-chip storage requirements, depending on how it is done. In later sections in this chapter, we will go into the details on the effect of parallelism on the requirements for expensive HBM memory storage.

Local vs global requirements In parallelism, it makes a big difference whether a number is local (per-chip) or global (overall sum). E.g. a 1 TB model requires 1 TB of expensive HBM to store. If your chip only has 100 GB of expensive HBM storage, then you can't run the model. Right? No, because this confuses local (per-chip) and global (overall sum) capacity. The local per-chip number is 100 GB capacity of expensive HBM, but if you have 20 chips, then the global capacity of expensive HBM is $20 * 100 \text{ GB} = 2 \text{ TB}$. You bought 2 TB of expensive HBM overall and so therefore you can fit a 1 TB model in there via parallelism.

Minimum installation size The minimum installation is the number of chips and other infrastructure that you must have before your AI can function. We just discussed a notion of fitting a 1 TB model across 20 chips when it doesn't fit on a single chip. This means that we must always have 20 chips, otherwise we cannot run the model, so the minimum installation is 20 chips (or somewhere between 10 and 20, depending on expensive HBM requirements for other things than weights). You still need 20 chips even if 20 chips generate a throughput that is far in excess of what you need. So large minimum installation sizes are inconvenient.

Reliability Computers sometimes fail. Cables disconnect, fans stop turning, chips melt, many things can happen. If your AI parallelization scheme involves 1024 chips working together, then it may be that if just one of those chips fail, then your whole AI will not function and the capacity of the other 1023 chips is wasted until that one chip is identified and fixed or replaced. So parallelization magnifies failure rates by in this case a factor of 1000.

In early TPUs we had this exact problem: TPUs can function together in pods of 1024 chips, but the cables connecting them would sometimes fail, leaving the entire pod unusable until a human employee in the datacenter would walk over and identify and replace the failing cable. An individual cable would rarely fail, but among 1000s of them, needed to connect all these chips in any given pod together, it would not be so rare for *one* of them to fail. We couldn't change the failure rate of the cables, so instead [Sameer Kumar](#) changed the parallelization software so that

it still functioned well even with a few failed/unusable/missing cables. This was pretty tricky given the torus network topology that TPU pods use. The cost of this solution was to halve the effective network bandwidth of the torus, but the bandwidth of the TPUv3 torus network was overprovisioned in the first place - this is an example where overprovisioning, even unintentional overprovisioning, leads to flexibility in resolving issues that were not predicted when a chip was designed.

Failure domain The failure domain of a component is the subset of a system that will cease to function if the component fails. So if a pod of 1024 TPUs stops working if a single cable breaks, then the failure domain of that cable includes all those 1024 chips. So the way that parallelization tends to increase failure rates is that it increases the size of the failure domain. If two chips are collaborating somehow, there is the potential that one of them being broken will affect the work of the other one. Minimum installation sizes tend to also be minimum failure domains, since if 20 chips are required, then the system cannot function if any of them break.

Parallelization using duplication

The simplest way to parallelize an LLM is to have several separate instances that each handle a subset of the conversations that the assistant engages in. This is the simplest kind of parallelization to implement in software, but even this isn't so trivial. For example, if all the customers assigned to one instance of the system start speaking to the AI at the same time, it may be necessary to load balance by quickly moving some of those conversations from one instance of the system to another one.

Here is the effect of having D duplicate instances of your system:

Duplication by D

Throughput: Multiplied by D

Latency: No effect

Expensive HBM per chip: No effect

Reliability: Improved (if one instance fails, the others are still available)

Network bandwidth: Little change - but may need to be able to move KV caches quickly

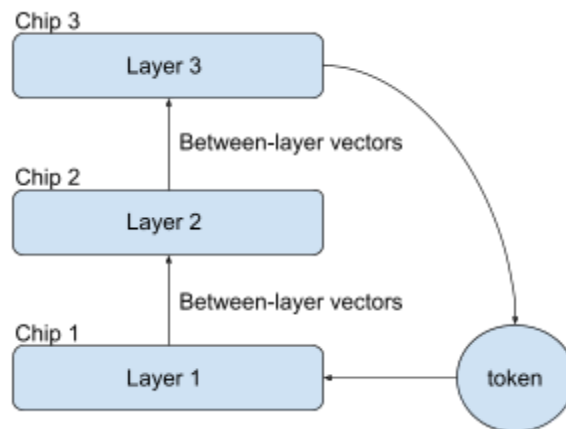
Systolic array utilization: Unchanged (can still be 100%)

Duplication may have a negative effect on first token latency and variability if the KV cache of your conversation has to be moved from one instance to another one because the instance holding your KV cache was overloaded.

Duplication is the only form of parallelism that we will discuss that improves reliability.

Between layers parallelization of decode using pipelining

If you have e.g. 32 layers in your model, then you can distribute those layers across 32 chips. To see how this works, this is an (extremely) simplified view of the structure of a transformer:



Here each transformer layer consists of an attention and an FF (sub)layer. Each layer has been placed on its own chip and the chips can exchange vectors over some kind of network. The output of layer 1 is a batch of vectors (activations) that are the input of layer 2. So layer 2 cannot get started until layer 1 is done. This is a challenge for parallelization, since this would mean that chips 1 and 3 are idle while chip 2 processes layer 2. The solution to this problem is pipelining. How this works exactly depends on whether we are doing training, prefill or decode. We'll discuss decode in this section and look at prefill in the next section.

For decode, the objective is for layer 3 to figure out what the next token will be (or next few tokens if using speculative decode). Suppose we have 3 batches (sets) of conversations X1, X2 and X3 that we need to process (respond to / produce output tokens for).

This is the sequence of events of what occurs during decode with pipelining:

1. Chip 1 processes X1.
2. Chip 1 processes X2, Chip 2 processes X1.
3. Chip 1 processes X3, Chip 2 processes X2, Chip 3 processes X1
4. Chip 1 processes X1, Chip 2 processes X3, Chip 3 processes X2
5. Chip 1 processes X2, Chip 2 processes X1, Chip 3 processes X3
6. Chip 1 processes X3, Chip 2 processes X2, Chip 3 processes X1
7. ...

As you can see, all the chips end up being busy all at the same time. This is because we have three sets of independent conversations for the layers to process, so Chip 1 can process Layer 1 for one set of conversations while Chip 2 processes Layer 2 for another set of conversations and so on. The conversations will then flow in a circle through the layers, where the circle is closed at the final layer (Layer 3 in this diagram), where the next token is decided and then communicated back to the first layer (Layer 1) for it to start the process anew on the next token. This kind of parallelization solution where independent elements are processed at different stages simultaneously is called pipelining.

So if there were 32 layers, we would be parallelizing across 32 chips. Suppose we only have 4 chips. We can still pipeline, we just put 8 layers on each chip, creating a 4 stage pipeline to run on the 4 chips. In general, an S stage pipeline involves S chips and multiplies the throughput by S , assuming all the layers take the same amount of time to process.

The most obvious effect of pipelining across S chips in this way is to multiply throughput by S . Pipelining in this way does nothing to reduce token latency, since each individual conversation/token is only ever processed by one chip at a time. In fact, compared to using 1 chip by itself, we now have to spend time to transfer the data between chips at every stage, which will somewhat *increase* latency and reduce throughput.

Recall that parallelization using duplication gave a straight throughput increase without an increase in latency, which is better than what we are seeing here for pipelining. So why would one ever use pipelining? Pipelining has the strong benefit of reducing weight storage requirements, so that we do not need to buy as much expensive HBM. A given chip/stage only has to store the weights for its own layer(s), so if there is e.g. one layer per chip, that chip only has to store the weights of that one layer. So if there are 32 layers, we can reduce weight storage requirements *per chip* by a factor of 32 by using pipelining. Duplication does not have this important benefit.

How about KV cache storage requirements? It turns out that pipelining does nothing to reduce the weight storage requirements for KV cache. The reason for this is non-obvious. At first sight, it seems as if it should help. Each layer has its own keys, so, as for weights, wouldn't it be true that the chip that does a given layer only has to store the keys for that layer? Yes, that is in fact true. So then haven't we reduced the storage requirements for the KV cache *per chip* by a factor of 32, like we did for weights, if there are 32 layers? No, it doesn't work out that way. Recall that every layer is doing separate batches of conversations. So due to pipelining we have 32 times as many batches being processed at any given time. A given conversation is going to come around the circle and back again to the same layer to produce also the token after this one (unless that was the last token), so you need to keep all the conversations in the KV cache, even the ones *currently* being processed by other layers. So we have reduced storage requirements *per chip per conversation* by 32x, yes, but we also need to process 32x as many conversations across the pipeline at any given time to keep all the chips busy (get high utilization). These two factors of 32 cancel each other out, so in the end pipelining doesn't make any difference for KV cache storage.

Let's look more closely at the problem of the network transfers between the chips. After processing a batch of conversations, every chip has to transfer its output to the next chip in the circle (which loops around to the first layer, via a token, at the final layer). No chip can start processing on the next batch until it has received the next batch. This means that all the chips are idle while the network transfers the data between them, and this happens every time the pipeline moves on to the next stage. This can be a long time or a short time depending on how much time it takes a chip to process a layer versus how long it takes the network to transfer the data that is the output of that layer. One solution is to accept that there will be a wait and to

ensure that the network is so fast that the wait will be short. This way utilization of the systolic arrays will always be lower than 100% since we are leaving them idle during the transfers, but it might not be that much lower than 100% if you have a really fast network.

Another solution that enables 100% systolic array utilization is to double the number of pipeline stages to hide the transfer time. The way this works is that we introduce twice as many batches of conversation into the pipeline. One half of conversations are being processed by the chips, while the other half are having their data transferred over the network. At every step of the pipeline, the two halves swap places. Assuming that the network is fast enough to transfer the data in the time it takes to process an entire layer (which it should be), then the transfer will be done before the next chip is done processing its layer, which means whenever any chip is done processing a layer, it will be able to immediately start processing an input that has already been transferred to it. So none of the chips ever wait on network transfers (assuming a network transfer can process concurrently without bothering the systolic array). This does lead to lower than 100% utilization of the network, unless network bandwidth and systolic array throughput have been perfectly balanced (I wouldn't do that - better to overprovision the network a bit).

This modified pipelining approach, that we might call full pipelining, ensures that we get a full throughput speed-up of S if we use S chips. It also *doubles* token latency. Why would that be? A given conversation now has to go through twice as many steps, and all the steps take the same amount of time as a layer, so it's going to take twice as long to produce a token. Kv cache requirements have also doubled, since we doubled the number of conversations to store KV caches for. Weight storage is unchanged, since it doesn't depend on the number of stages in the pipeline. A more subtle but very important thing that we have achieved is that the network now doesn't have to be anywhere near as fast as before, since we are not waiting on it any longer. It just has to be able to complete its transfer in the (significant) time it takes to do a layer - if that is true, then it causes no slowdown at all. For this reason, when using full pipelining, the network connection between layers doesn't have to be as fast as one might think.

To sum up, the benefit of between-layers pipelining is to greatly reduce weight storage requirements. This comes at either the cost of needing a very fast network and a small drop in throughput if we use partial pipelining or the cost will be a doubling of latency and KV cache storage requirements if we use full pipelining (but network requirements are less).

How bad is a doubling of latency? Different applications have different latency requirements. Sometimes it's irrelevant because the customer just needs an answer sometime, could be tomorrow. For an AI assistant with a person on the other end waiting, token latency should be low, but if it is already so fast as, say, 100 tokens per second, then 50 tokens per second isn't much worse - it's a faster speed than people can read. It's fast enough already. Though if you are using a reasoning model, that silently speaks to itself ("thinks"), and thus needs to generate *many* tokens that the user never sees, then perhaps token latency is more of a concern even at fast speeds and so you may (or may not) prefer to avoid full pipelining.

How bad is a doubling of KV cache storage? Well it depends on how large the KV cache is *per chip* and how much expensive HBM storage capacity you have per chip. If you are doing something reasonable with your KV cache, like using retrieval for ancient (past 1000 token) history, then a doubling might not matter at all.

Note that you can pipeline e.g. only every 4th layer instead of every layer. This means it takes 4 times as long until you have to transmit the same amount of data over the network that you otherwise had to transmit between every layer. So this drops bandwidth requirements by a factor of 4 compared to pipelining every layer. It also reduces the increase in the KV cache storage requirements, as you now have an extra network transfer pipeline stage between every 4th layer, instead of between every layer.

Decode partial pipeline by S

Throughput: Multiplied by a bit less than S (how much less depends on network bw and latency)

Token latency: Increased, depending on network bandwidth (and latency)

Expensive HBM per chip: Divide weight storage by S, KV cache storage unchanged

Reliability: Worse. Multiply failure domain by S.

Network bandwidth: Critically important - on the critical path

Systolic array utilization: Reduced (can never reach 100%)

Decode full pipeline by S

Throughput: Multiplied by S (assuming reasonable network bandwidth)

Token latency: Doubled (assuming reasonable network bandwidth, otherwise more)

Expensive HBM per chip: Divide weight storage by S, KV cache storage doubled

Reliability: Worse. Multiply failure domain by S.

Network bandwidth: Has no effect, as long as fast enough

Systolic array utilization: Unchanged (can still be 100%)

This was for decode. The next section concerns pipelining of prefill.

Between layers parallelization of prefill using pipelining

Pipelining is more attractive for prefill than for decode. The reason for this is that it is not necessary to introduce extra batches of conversations in order to pipeline for prefill (and training). Steady state token latency is still increased regardless, but this way the KV cache storage requirement for S-way parallelization is reduced by a factor of S, unlike for decode. For a very large document, it is possible for all the chips in the whole system to be working on reading that same document (or conversation). The only reason to introduce a new document or conversation into the system is if the current document or conversation has been fully processed ("read") already.

During decode, the first chip would process Layer 1 for one batch of conversations, send its output to the next chip and then proceed with processing Layer 1 for another batch of conversations and so on. The pattern for prefill is similar as for decode, but we will be looking at

spans/subsets of a single document/conversation instead of batches of different conversations. So e.g. the first chip might process Layer 1 for the first 256 tokens of a document, send its output to the next chip and then proceed with processing Layer 1 for the following 256 tokens of a document and so on. Since we are looking at only the same document, full pipelining does not significantly (or at all, depending on how you do it) increase KV cache storage requirements. Also, KV cache requirements for prefill are not high in the first place anyway.

Prefill partial pipeline by S

Throughput: Multiplied by a bit less than S (how much less depends on network bw and latency)

Token latency: Increased, depending on network bandwidth (and latency)

Expensive HBM per chip: Divide weight *and* KV cache storage by S

Reliability: Worse. Multiply failure domain by S

Network bandwidth: Critically important - on the critical path

Systolic array utilization: Reduced (can never reach 100%)

Prefill full pipeline by S

Throughput: Multiplied by S

Token latency: Doubled

Expensive HBM per chip: Divide weight *and* KV cache storage by S

Reliability: Multiply failure domain by S

Network bandwidth: Has no effect, as long as fast enough

Systolic array utilization: Unchanged (can still be 100%)

I won't explain much about how this works for training, since the topic is more complex, but there is a little information about it in the nerd box that is the next paragraph.

[[[Details on pipelined training for nerds In training, there is initially the same forward flow of data as during prefill, followed by an opposite flow of *gradients*. The direction of the flow of data turns at the final layer, making its way from there back through all the layers until it ends up back at the first layer. The gradients are a kind of feedback about what happened during the forward pass - each layer is being told by its following layer "you told me that X, but I would have preferred you to say Y instead." Each layer learns from this feedback and that is what happens during training. Each chip will process each text (mini- or microbatch) twice - once forward and, later, once backward. Training requires storing forward information (activations) for later use when gradients for the same text come back around. With L forward layers, it takes 2(L-1) total layers traversed (forward+backward) for gradients to make their way back to the first layer. Given that we are using a pipeline with many texts in flight at the same time, the first chip thus needs to store 2(L-1) instances of forward data (activations). This amount of storage is manageable with proper optimizations like checkpointing and also because sequence lengths during training are not so large as they can be during inference. You might train with a sequence length of 4k or 16k tokens, not 1m (except perhaps during a very brief long-sequence-length training run at the very end of training, but these are done differently and do not have to be fast, so e.g. you might store activations to SSD, which would otherwise be unacceptably slow).]]]

Within layers parallelization

In the section on pipelining, we assumed that each layer runs on a single chip (or several layers on one chip). It is also possible to parallelize a single layer across several chips, which is the topic of this section. This is more desirable than between-layers pipelining for reasons including that it reduces both KV cache requirements and token latency. A big cost here is that this requires much more network bandwidth than pipelining.

You may recall that a transformer layer consists of two sublayers: Attention and Feed-Forward (FF). These are parallelized in separate ways, so we will discuss each separately.

Attention is quite easy to parallelize. If there are 128 attention heads, then you can have one chip do each, so you'd be able to parallelize across 128 chips that way. This works because attention heads operate independently of each other, except a sum at the end.⁴ However, you are probably using GQA, which changes the picture a bit. If GQA puts two attention heads into the same group, then you may prefer to put those two attention heads on the same chip, since then that chip will have more query vectors during decode - otherwise GQA will not have the benefits (which are important) for decode that we claimed in the section on decode. So you might only get a factor of 8, or however many GQA groups that you have, this way. You may well wish to parallelize within layers far beyond 8 chips, so we need some additional ways to parallelize beyond this.

There are other ways to parallelize attention within layers. One idea is to split the KV cache across 2 or several chips. The results of attention on two subsets can be combined at low cost across chips, so this works well. For long-context attention, you can probably get a factor of 4 to 8, maybe more, this way, so in total we are up to 64 (8 GQA groups times 8-way split of KV cache), which is pretty good. You can push it further if you like, especially if you are doing prefill (can potentially split queries), but the techniques mentioned here work for both decode and prefill.

FF works differently but also parallelizes well. FF consists primarily of two large matrix multiplications, increasing the (hidden, intermediate) vector width substantially (typically x4). You can parallelize so that each chip is responsible for a subset of the wider middle vector width. If you have a 256 x 256 systolic array and a 16k model vector width, you can potentially push this all the way to parallelizing by $16k * 4 / 256 = 256$ chips while still getting 100% systolic array utilization. From this you can see that while FF requires tremendous computational power, it also parallelizes well and is a systolic array's greatest friend. There is no difference between prefill and decode for FF, both work equally well.

⁴ Don't attention heads *concatenate* (as opposed to summing) and then do a matrix multiplication? Well, yes, that's more efficient, but this is equivalent to multiplying up to width `d_model` and then summing. So I'm describing it that way because it's simpler, but yes, you might get more efficient use of the network if you concatenate in a distributed fashion (all-scatter) instead of summing. That's an optimization.

Both attention and FF require an across-chips sum at the end to combine results between parallel chips. An across-chips sum is called *all-reduce*. I won't describe all-reduce algorithms here as they are highly complex (I did the initial all-reduce software for TPUs), but there is some detail on this in the next chapter on network topologies. Per-chip bandwidth requirements for the within-layer network are a bit more than doubled when you parallelize across twice as many chips using within layer parallelization, so your network bandwidth may limit parallelization of this kind. See the nerd box if you want to know what "a bit more than doubled" means.

[[[Quantifying "a bit more than doubled" network bandwidth requirements for nerds To do a distributed sum (all-reduce), the data is divided into equal $1/P$ chunks if there are P chips involved. Each chip is responsible for one of the chunks and so needs to receive that chunk from all the other chips. It will then sum those chunks and send back the summed result to everyone (this is a simplified view, but the math works). That is two transfers, so if the data on each chip has size X , then the total amount of data transferred from each chip is $2X$. Except you don't need to send your own chunk to yourself, so the actual amount is $2X * (1 - 1/P)$. The factor of $1 - 1/P$ is the source of "a bit more" in "a bit more than doubled". If you go from parallelizing 2 ways to 4 ways, then the multiplication by $1 - 1/P$ goes from 50% to 75%, which is significant. If you go from parallelizing 64 ways to 128 ways, this is 98% to 99%, which is not significant. So this factor is mostly important when parallelizing across few chips. $1 - 1/P$ is especially significant for $P=1$, where you don't need any bandwidth because there is no parallelization.

From this you might observe that the network transfer amount of $2X * (1 - 1/P)$ is roughly equal to $2X$ for large P and you might further observe that $2X$ does not depend on P . So no matter how large P gets, we never need to transfer more than $2X$ data per chip per step. So why would doubling P require double the network bandwidth per chip? This is because the rate of steps has doubled, so you need to do twice as many distributed sums in the same time. This is a big driver of massive bandwidth requirements for AI chips, so I figured it was worth going into some detail on.

Assume we balanced the network so it can transfer $2X$ bytes in the time it takes one chip to compute one batch on its own. Then with P -way within layer parallelization, we now need P times as much bandwidth to still be able to transfer $2X$ bytes in the time it takes to compute one batch (using P chips, which is P times as fast). And then the math is a bit more complicated than that since you need to take the factor of $1 - 1/P$ into account, as well. So the simplified conclusion is that doubling P requires a bit more than double the network bandwidth.]]]

[[[Per-chip between-layers bandwidth requirements are neutral to within-layer parallelization If we do within-layer parallelization by a factor of P , then we now process P times as many batches in the same time. Doesn't this mean that the networking requirements increase between layers by a factor of P ? Not necessarily. If you have each of the P chips within a layer send only $1/P$ of the data forward to the next layer, then that will improve speed by a factor of P , cancelling out the fact that things now happen P times as fast. So it is neutral on a per-chip bandwidth basis if you do it this way. Since within-layer bandwidth requirements per chip increases by a factor of P here, while between-layers bandwidth is neutral per chip, you

can see that you need much more bandwidth within-layer than between-layer for high degrees of within-layer parallelization. This is a big driver of high network bandwidth requirements for highly within-layer parallelized AI transformer model inference - which you need to get low token latency for large models. It also means that you might be able to put between-layer transfers on a much slower network. Another factor that facilitates this is to do several layers on the same set of chips, which reduces between-layers network requirements proportionally. **]]]**

As for pipelining across layers, we now have a choice: When we need to do the network transfer for within layer parallelization, we have the option of leaving the chip idle until the network transfer completes. The other option is a 2-stage pipeline, where we double the amount of conversations in flight and one half is being processed in a layer while the other half is being transferred (all-reduced) over the network. The two halves swap places at each stage of the pipeline. As before, pipelining by 2x increases kv-cache storage requirements and token latency by the same factor (2x) but reduce network bandwidth requirements (very important!) and also ensures that we don't wait on the network at all as long as it can do the transfer in the time it takes to do a layer.

Note that there is a communication step between attention and FF, so for full pipelining, this is 4 stages per overall transformer layer, not 2. I suggest to do a model change so that you do attention and FF independently, in parallel, so that there is no dependency between them inside an attention layer. This halves token latency and the number of stages and therefore KV cache requirements. The cost is a slight degradation in converged model accuracy. However, public transformer models that you can download are usually not done like this, so you'll have to deal with the serial approach to use any of those - this is something that requires training a new model, you can't easily retrofit it into an already-trained model.

Suppose you are doing full pipelining both within layers and between layers. Then the output of each layer have two network transfers back-to-back: one within layer and one between layers. Assuming the network is fast enough, you can combine those two stages into one, thereby reducing the number of stages and proportionally reducing token latency and KV cache storage.

If you combine between-layers and within-layer parallelization, the weight storage benefits *multiply*, so this is very significant. This is how you achieve parallelization by large numbers of chips, like 1024, and get down to tiny weight storage requirements per chip even for huge models. The number of chips required also multiply, of course. So does the failure domain.

[[[Example of the benefits of full pipelining Suppose you have parallelized 8 ways within layer without full pipelining, and you observe an extra 25% of the time is waiting on the network. You are concerned about this overhead, so you don't want to parallelize further. You also do not want to use full pipelining, since then your token latency will double.

Did you spot the error in thinking here? If networking is 25% extra, then if you fully pipeline, you can parallelize 4 times as much without waiting on the network at all, so that is 32x in total. You did double token latency, but now you also divided it by 4, so in total you have halved the token

latency, not doubled it. And it's better than that, because you can subtract the original 25% overhead since you no longer wait on the network. **]]]**

If you are designing an AI chip, you might want to consider running a simulation of how you will parallelize across chips, rather than just using math in a spreadsheet. Ideally this would be a full implementation running in simulation (of some kind), if that is feasible given your time table and simulation resources (it might not be). The reason I give this suggestion is that it is both complex and unintuitive how network bandwidth requirements and other requirements respond to a given parallelization scheme. If you get it wrong, you may have built a chip that can't be used in the way that you intended to use it. I am loath to admit it, but earlier in my career, I misderived (there was no one around to explain it, so I had to figure it out myself) some of this math for an embarrassingly long time. TPUv2 network bandwidths were set by someone else earlier *who also got it wrong*, leading to the network being overprovisioned (many things on TPUv2 were overprovisioned). Maybe you are sure you got it right because your engineers are more reliable at using math than I am. That's possible, though I do have a PhD in computer algebra. Doing a simulation much reduces the risk that you got something fundamentally wrong.

[[[Even simulation doesn't catch everything When TPUv3 arrived at Google HQ, there was a problem with networking. The problem was that the TPUv3 chip wasn't capable of driving the cables in a way that allowed communication to occur between two TPUv3 chips. You might see how this was a problem given that massive scaling is one of the selling points of TPUs. Turns out the cable communication component of the chip wasn't functioning as expected. The team even *had* done simulations of networked chips, but they abstracted away some of the physical details of communication over a cable. A heroic and intense weeklong collaboration between the firmware team and the hardware team (that I wasn't involved in and can take no credit for) resulted in the discovery of a way to operate the chip through software so that TPUv3 in the end did have fully functional networking without having to redo the chip, so TPUv3's launch was not postponed. Even detailed simulations are not a guarantee, though they are useful to catch many issues. **]]]**

[[[Cheap networking for AI chip nerds The sum at the end of attention/FF is an all-reduce, which can be decomposed into separate reduce-scatter and all-gather operations. You can do the between-layers data transfer in-between these two operations, having all chips send their $1/P$ subset of the data across to the next layer of chips. So the data sent between layers per chip can be quite a bit less than that sent per chip inside layers if P is non-trivial. You may want to consider whether you can use this to produce a more economical chip. The networking solutions commonly used by AI chips are very expensive. More on this in the next chapter on network topologies.

[[[Bits on the network for nerds Your bandwidth requirements also depend on features like compression and lower-bit numerics on your chip. Do be aware that network transfers are sums and so can (very unfortunately) possibly require higher bit precisions than what you are using for multiplications (inside systolic arrays), leading to increased network bandwidth requirements compared to what you might think if you assume that you can use 8 bit transfers for sums just

because your systolic arrays are using 8 bits for multiplications (your systolic arrays will also be using more than 8 bits for additions). This then gets into routers versus tori, because tori need to transfer wider partial sums than routers do, during all-reduce/reduce-scatter, so that can have an effect on the required precision, also depending on the precise size of the torus. Ask your parallelization engineer to figure this out in collaboration with an AI researcher who is interested in such topics. You may not be able to find an AI researcher who was already interested in this topic (this is likely), so then you need to get someone trained up on this. This can save you from having to use 32 bits for additions on the network like many people do. 8 bits is probably not enough, 32 is too many. 16 bits is probably OK, but this can require special care and it depends on your numerics (this is perhaps the only place where FP16 can be better than BF16, and possibly int16 is best). Maybe your customers are not sophisticated enough to use anything less than 32 bits for sums (i.e. within-layer parallelization), in which case you need to be aware of that when you dimension your network bandwidth (this requires TWICE as much bandwidth). You and your customers will have an easier life with 32 bits, but you'll need to buy a network that is twice as expensive.]]]

Mixture of Experts (MoE)

MoE is a complication of FF where instead of one big matrix you have N different smaller matrices called *experts*. Each matrix/expert has knowledge of different areas, so the idea is that each token is routed to only the expert(s) that have knowledge that is relevant to that token. This is a speed-up since that token then does not have to be considered in relation to weights (experts) with irrelevant knowledge, saving a lot of matrix multiplication. MoE is a powerful technique to speed up transformer models, but it causes a lot of complication in terms of deployment if you want to reach high utilization.

MoE is not necessarily a parallelization technique, you can do it without parallelization (put all the experts on one chip), but one way you parallelize this is to place each expert on a different chip or set of chips. Individually, each expert is just a regular FF layer, so it can itself be parallelized within-layer just like we saw for any other FF layer (including the implication for within-layer versus between-layer bandwidth).

The transfer to the expert and back to where the token came from can optionally be pipelined to avoid waiting on the network, with the same storage and performance implications as explained in previous sections for pipelining.

The number of experts varies wildly between models. E.g. one model like Mixtral 8×7B might have 8 experts (2 experts active, so 25% active) while another like Llama 4 “Maverick” has 128 experts (17B weights active versus total 400B, so 4% active). Google even once published a model called Switch-C 1.6T with 2048 experts, though that is not the norm today.

A big problem with mixture of experts models is that experts are not necessarily evenly used. E.g. if expert A knows about English nouns, and expert B knows about hats worn in Mongolia in 405 BC, you can see that expert A is going to be consulted more often than expert B. The result

is that you need to provision many more chips to handle expert A than expert B. If suddenly people start caring about hats worn in Mongolia in 405 BC, then you need to add capacity to expert B at that time at a moment's notice. None of what we've discussed so far in this document has required this kind of dynamic variability during deployment and that is the trouble with MoE.

Training of an MoE is often done in a way so that one tries to avoid uneven load among the experts by giving them a distribution of knowledge that is intended to make all experts roughly evenly in demand. Training MoE models in this way improves the evenness of demand, but it does not guarantee perfect evenness.

One of the problems with MoE models with many experts is that you may have so few conversations that consult a given expert that it doesn't have enough conversations to process to fill a systolic array. So then it will have to operate in an inefficient way with less than 100% utilization, perhaps much less. The same chip could serve other experts while waiting to accumulate enough conversations with the less popular topic to fill a systolic array, though that will increase token latency and KV cache storage requirements.

This same issue applies even at higher loads. If you have load for 120% of an expert, and you don't have a way to queue waiting tokens, you need to duplicate that expert so that you have two experts at 60% load. So even though you have significant load on the expert, both experts of that kind are running at only 60% systolic array utilization. So you may prefer to have *some* queueing for a MoE model.

The more massive your installation size is, the higher MoE utilization you can get even without queues, assuming you can fill the whole system with work. This is because then you will have higher multiples of the experts. We saw that 120% load leads to 60% utilization. However, 1020% load on an expert leads to assigning 11 chips to that expert (or sets of chips if parallelizing within layers), each with $1020\% / 11 = 93\%$ utilization. As you can see, the rounding effect of "one more expert" is reduced at high loads. You can approach 100% utilization on average in this way at high multiples, but it won't reach all the way to 100% on average without queues.

You aren't going to get 100% utilization for a MoE model without letting some tokens wait, increasing token latency and KV cache storage requirements. The simplest solution to this issue is to accept lower utilization. From listening to industry chatter, MoE as commonly implemented usually leads to MUCH lower utilization than 100%, though I think this can be improved with better software.

A more advanced system is to have a system with high-priority tokens (e.g. AI assistant tokens with a live customer waiting on the other end) and low-priority tokens (batch offline work, no or only mild token latency requirements). Then experts prioritize serving high-priority tokens immediately and low-priority tokens wait in queues that are used to fill the experts, to increase utilization. If the low-priority tokens have low attention length (you might be able to arrange that),

then the KV cache storage requirements for having them wait in queues is reduced. Combining such techniques makes it possible to reach high utilization also for MoE models. You might look at projects like [vLLM](#) for deployment of MoE models.

The difficulties of deployment increase with the number of experts, which is likely why some MoE models have few experts. Other than deployment woes (which is a serious issue!), MoE works better with many experts.

I'm going to make a prediction here: LLMs contain massive amounts of knowledge in their weights, most of which is irrelevant most of the time. That is why MoE with many experts is so effective - it is discarding huge amounts of irrelevant weights. The problem here is that we stored so much knowledge in weights in the first place. That's wrong. We should instead use within-layer retrieval from a vector database (not the same as RAG, RAG is not within-layer) to store most knowledge, dramatically reducing the number of weights while improving model accuracy. This will also dramatically reduce the positive impact of MoE, though I think we'll still use it, but only with a few experts, like 4 or 8. Instead, the irrelevant knowledge will have been shipped out to a database with lots of rows that were not retrieved and were therefore free. I don't know if it'll go that way, but I suspect that it will. We may also have much more retrieval of small amounts of weights, like QLoRAs, in the future, which again off-loads some of what MoE achieves today. MoE is better than nothing, but we need something better still.

[[[Detail on MoE implications for tori versus routers One of the great strengths of torus topologies is that in everything we've discussed so far, a chip only ever needs to communicate with its neighbors in the torus, making torus topologies supremely economical and efficient. This isn't true for MoE models, since you might have to send tokens several hops away to get to an expert over there. In a routed topology, you can jump straight over there assuming its on the same router (which it might not be). In a torus the data has to be transferred through several hops in this case. This reduces the effectiveness of torus topologies, but perhaps not so much that routers become preferable. Google TPUs still use a 3D torus even though MoE models are quite popular. You can read much more about this in the next chapter on network topologies.]]]

[[[Detail on forced even distribution for nerds There is a technique, well suited to training, that leads to perfectly even load also during inference. The technique is to force the AI to route tokens evenly no matter what. This way an underutilized expert will receive tokens that are a better fit for another expert... but that's just too bad. So this way your system can still get 100% systolic array utilization, since everything is even. However, if this is used during inference, it also means that the responses you get from the AI might be randomly worse sometimes since the quality of your response will depend on what topics other people asked about at the same time. If it's the same topic that you asked about as they did, maybe that expert was busy and that's just too bad for you. For example, if you ask a question in Danish and only one of the experts know Danish, and that one wasn't available right then, then you might get some strange answer back since the AI normally knows Danish but now suddenly it doesn't. My understanding is that some AI assistants actually do this. Yikes! Though presumably in this case they've put a lot of effort into removing or reducing the worst case results for this.]]]

Network topologies for AI in a MoE world

In this chapter I explain router and torus topologies and the algorithms that AI workloads require to run on these networks. I will assume that you already read the previous chapter on how to parallelize AI workloads. I also explain Nvidia's and Google's differing approaches to AI networking and give a different proposal of my own that I think is more economical.

You may have noticed that the previous chapter on AI assistant parallelization is the most complex chapter so far. I am giving you a simplified view here, but, even so, this chapter is also complex. If you are just looking for some light reading, perhaps you'd prefer to go right to the next chapter on co-design. That one is much lighter reading.

In designing an AI chip network, you need to choose between router and torus topologies, you need to minimize cost and you need to figure out how much bandwidth you need. Figuring that out is a complex question that is the subject of this chapter. Google TPUs use very large 3D toruses with a fancy optical approach that allows reconfiguring the size of the torus. Nvidia used to offer a small NVLink 1D topology (ring topology), but in more recent times has moved to more expensive routed topologies that are still called NVLink.

I will end up recommending neither of these approaches. It's not possible to determine what Nvidia's cost is to produce NVLink, but it is very expensive to customers. Google's wide torus approach is a poor fit for MoE workloads and they have used expensive optical connections. Both approaches are overly expensive for what they provide. AI chips simply have not been optimized very much for cost because customers are willing to pay high prices.

Google has sunk enormous sums into buying TPUs for its own use. I don't know the numbers any longer, but Gemini (Google's own AI assistant) suggests ~\$100B as an all time expense to Google for all the TPUs it has produced. In that case 1% is worth \$1B, and that's only so far. So you might think this would motivate heroic cost optimization efforts, but I didn't see that at Google when I worked there and I still don't see it now from the outside. While minimizing cost may serve Google shareholder interests, the social reality of how this worked, at least while I worked at Google, is that the bar was to significantly beat Nvidia on cost. Not minimize cost, just significantly beat Nvidia on cost. So we had to beat Nvidia. And we did. But we did not move heaven and earth to minimize cost.

You will have to do your own math on this, but my proposal now is cheap lower-bandwidth routers on top of a cheap higher-bandwidth slim torus. No expensive cables and still a good fit for MoE workloads. Seeing why this works requires a simultaneous view across AI workloads, AI software and AI hardware. A view I aim to provide in this chapter.

Networking concepts

Just as for parallelism, there are some concepts related to networking that need to be introduced before it is possible to discuss networking topology.

Node

A node in a network is something that needs to transfer data to another node. Your AI chip will be a node and, if you have a router, that router is also a node. That the router is a node is a critical piece of the picture that is usually missed - it is critical because the fact that the router is a node introduces a 2x overhead on routing that is usually not noticed (more on this later). The host computer that houses your AI chip is also a separate node and this brings its own overheads.

Link

A link is a physical connection that carries data directly between two nodes. A single link may be composed of many different components, like CPU pins, on-motherboard copper traces, networking ports and copper or optical cables. There are many ways to make a link that may omit many of these components. E.g. an on-motherboard connection omits the networking ports and the cables, which is part of why on-motherboard copper trace links are cheap. In general, the more of these components you can omit, and the shorter the distance that is traversed, the faster and cheaper a link can be in terms of both latency and bandwidth. Routing and air cooling leads to the need for longer (and therefore more expensive) links, while toruses and water cooling makes it possible to use shorter (and therefore cheaper) links.

Link latency

Link latency is how much time it takes for one bit to traverse from one node to another across a link. There is not just a single link latency number - there are several versions of link latency depending on how much of the processing inside the chip for sending/receiving and routing you include. Link latency does not include the time it takes to transfer a large amount of data, just the time it takes to get the transfer started and, once it's done, to notice that it is done.

I can think of a few situations where link latency becomes important for AI workloads:

1. You didn't software pipeline. Software pipelining is important for AI, that's why this doc has a whole chapter on that. The effect of software pipelining is to ensure that you have something else to do while waiting on latencies, so then latency becomes less important.
2. You are using an extremely large torus or mesh topology. In this case, you may have to traverse a large number of links to get from one node to another and then the latencies can add up.
3. You didn't optimize it. Link latency is less critical for AI workloads than you might think, but if you didn't optimize for it at all, then you might still have a problem.

If you avoid these 3 mistakes, then link latency should not be a major concern for an AI chip. Bandwidth is more important.

If the intended niche for your chip is expensive low-latency tokens, perhaps you will choose the uneconomical approach of abandoning software pipelining, since that does decrease token latency. In this case link latency may be more important for your chip than it otherwise is.

[[[Dojo link latency for nerds Tesla's now-discontinued Dojo training chip was optimized for link latency. E.g. they mention a link latency of 1 cycle. For comparison, TPUv3 link latencies could be in the 100s of cycles, which is pretty high, yet that didn't come up as a big problem, so it was OK. So the Dojo focus on low link latency has puzzled me. Looking at [page 25 of these slides](#) might explain some of it. That slide shows a very wide 2D mesh, leading to having to traverse a great many links to communicate between far-apart nodes. If they also did not aggressively software pipeline, and used small transfers, which seems possible given the low amount of SRAM on each chip (software pipelining increases SRAM requirements and so does large transfers), then that would explain the focus on link latency. **]]]**

Link bandwidth

This is how much data can be transferred across a link per unit of time. Sounds simple? Well, suppose someone says that a link has "10 gigs of bandwidth". What do they mean?

1. Bits or bytes? B is bytes and b is bits, so 10 GB/s is 8 times faster than 10 Gb/s. Software engineers think in bytes and therefore need GB/s numbers. Hardware networking engineers think in bits and talk about Gb/s. People mix this up.
2. Bidirectional? A link can usually transfer data in both directions. Usually, 10 gigs means that you can transfer 10 gigs in both directions at the same time without one transfer competing with the other. But if you put your marketing hat on, you can see that it sounds more impressive to say that your cable is $10 + 10 = 20$ gigs in this case, since that is how much data it transfers combined. So sometimes people do that.
3. Giga or Gibi? Giga is 10^9 while Gibi 2^{30} . It is completely normal to say Giga to mean 2^{30} , even though that is wrong. A GiB is 7% more than a GB, since $2^{30}/10^9 = 1.07$.
4. Goodput or line rate? The link's bandwidth of payload data that you care about transferring is referred to as goodput. The rate of bits the link can transfer is referred to as the line rate. These are not the same, as some of the bits transferred are about the line protocol and therefore not useful data. So if someone tells you 10 gigs, that ought to mean goodput, but it might be the line rate instead.
5. Theoretical or realized? Even if the goodput is some rate, that might be some kind of theoretical calculation that doesn't take some practical factors into account, so the bandwidth that can actually be realized might be even less.

So you might have to ask several questions in order to figure out what someone means when they try to tell you a bandwidth number. And you have to consider the potential that they got this number from someone else and they misunderstood what this other person meant. Or they just got a number, and they have no idea about 1-5 above. The result is that communication is so difficult to do reliably on bandwidth numbers that you might prefer to run your own benchmark to figure out the ground truth for yourself. Which then everyone has to do, even though you already did it, because they can't be sure that they are understanding you correctly, either. Isn't this an amazing situation?

If you are involved in chip design, and someone tries to tell you *one* bandwidth number, be suspicious. It's odd not to mention both theoretical and realized bandwidth, so what you're being told is probably just the theoretical number. If you do planning based on that number, you might have a surprise later.

"The theoretical bandwidth is 10 GiB/s bidirectional, with realized bandwidth around 9 GiB/s." If someone talks like this, it *probably* means that what they are saying means what it sounds like it means. But you can never be sure. It really is true: everyone has to do their own benchmarks. But you can't do that in co-design, where the chip hasn't been delivered yet. So then you need to bother people with a conversation about 1-5 above. If you have some way of simulating the chip, absolutely try running your own bandwidth benchmark using that.

Transfer latency

Transfer latency is the time it takes to transfer data from one place to another. Transfer latency is at least the amount of data to transfer divided by the bandwidth. You might also include time spent waiting behind other transfers. You might also include time for occasionally resending corrupted packets. You might also include the link latency. You might also include several link latencies if this is a multi-node route. Transfer latency can mean many different things.

When people just say "latency", you don't know if they mean transfer latency or link latency. They might even be talking about *token* latency, which is completely different. And there are several different versions of each of those concepts, depending on what is included. To avoid some of these ambiguities, I would recommend not talking about "transfer latency", but instead talking about "how long does the transfer take?"

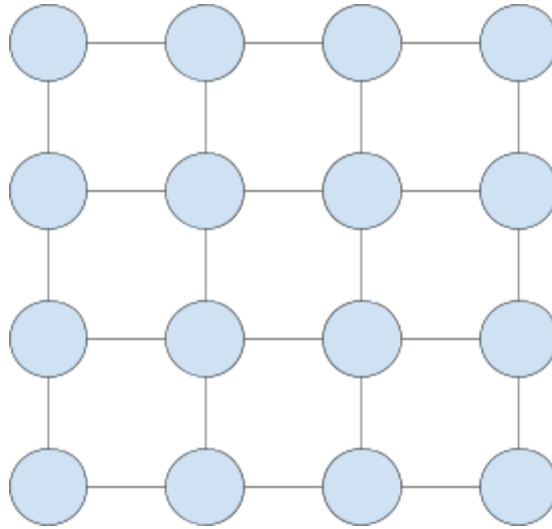
So you might need to ask some questions if people start talking about "latency". Yet from personal experience, I can tell you that people may not approve of being asked such questions, for a variety of reasons I get more into in the co-design section (many people hate admitting it if they don't know something). An example: the employee was asked to do some kind of latency measurement or calculation, and they did, and now you're asking them questions that they don't understand, so the employee becomes annoyed because some people hate saying "I don't understand what you're saying."

Mesh topology

A 1D mesh is a line. This is a 4-wide 1D mesh, which has 4 nodes and 3 links:



A 2D mesh adds another dimension. This is a 4x4 2D mesh, which has 16 nodes and 24 links:



You can also have 3D or even higher-dimensional meshes. The now-discontinued Tesla Dojo training chip used a very large 2D mesh. I recommend against using a mesh topology because a torus topology is usually a better choice.

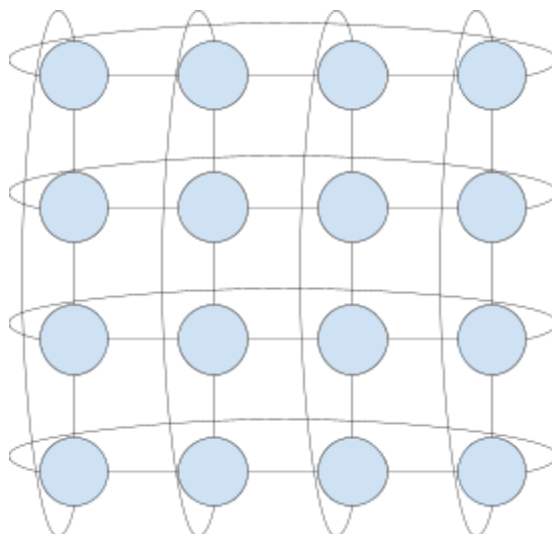
Torus topology

A 1D torus is a circle/ring. This is a 4-wide 1D torus, which has 4 nodes and 4 links:



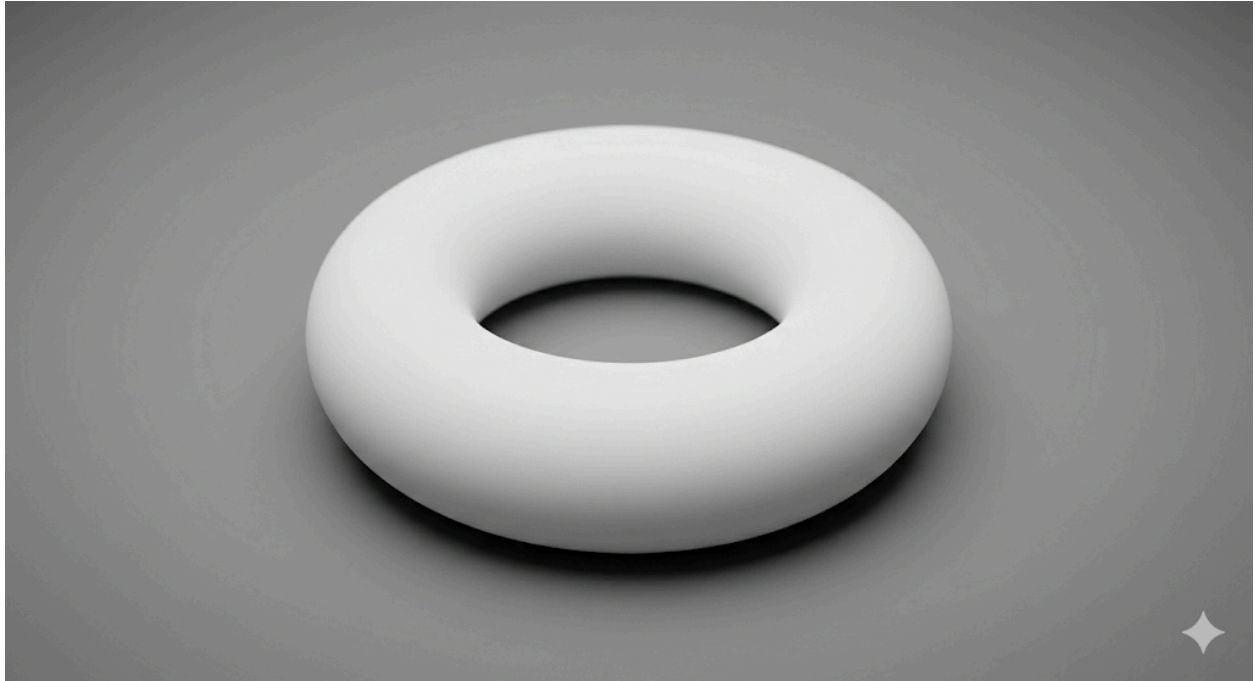
In olden times, Nvidia GPUs would be connected via point-to-point NVLink in a circle topology such as this, though these days Nvidia instead sells a more expensive routed solution.

A 2D torus adds another dimension. This is a 4x4 2D torus, which has 16 nodes and 32 links:



You can see that a torus is like a mesh, but with a few extra long links that wrap around, creating a circular connectivity in each dimension. You can think of a 2D torus topology as the cartesian product of two 1D tori topologies.

In mathematics, a 2D torus refers to a donut shape like this:



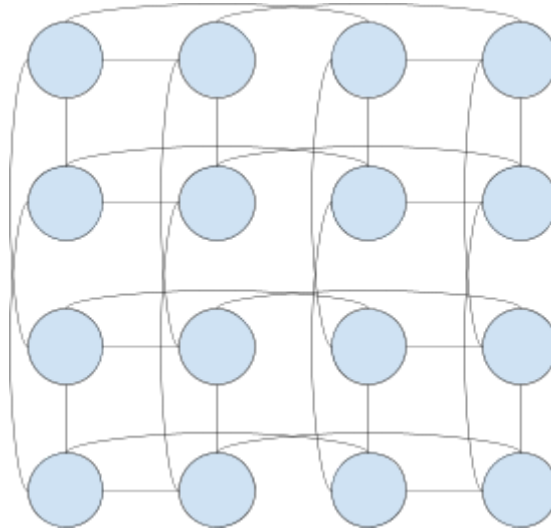
It might not be immediately obvious what this kind of donut shape has to do with a torus network topology. In fact, I once joined a team where everyone had agreed that the label of “torus” was misleading because they knew it was a donut shape and they did not see the connection from that to a torus network topology. If you imagine placing nodes on the surface of the torus in a regular pattern and connect neighbors with links, then you will notice that you can go around the surface of the donut in the picture in two separate directions: around the hole in the middle or around the center of the donut itself. This is precisely the connectivity of a 2D torus topology in networking - there are two separate ways to go around. That’s the connection - the connectivity of a 2D torus network is the same as the connectivity of the surface of a physical 2D torus as in the picture.

Folded torus

The long wrap-around links in a torus can be a problem. This is because longer links are more expensive and may need to use a different technology than the short neighbor links in a torus can. One solution is to use a mesh, which avoids the wrap-around links. But this halves throughput for many AI workloads, so that’s not great either.

For TPUv3, this was solved using very long optical cables from one side of the torus all the way around to the other side.

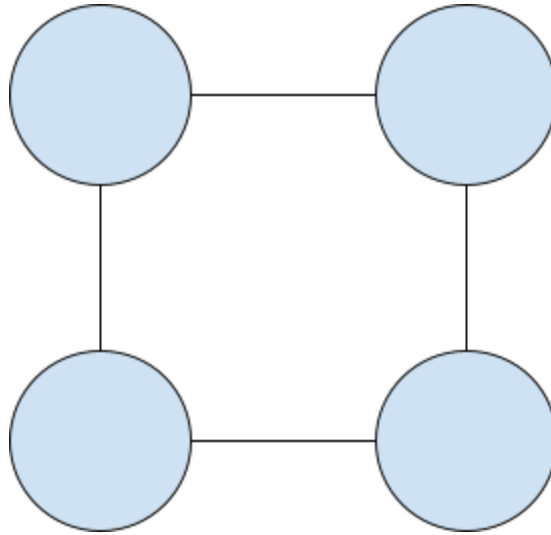
The folded torus is a solution that keeps all the links short. Here is a 4x4 folded torus:



It is not immediately obvious, but if you look closely, you will find that you can go in a circle both up-down and left-right, so this is still a 2D torus, the nodes have just been moved around. This generalizes to much larger toruses of any dimension. The general pattern, which works for any dimensionality with even torus widths, is that all links skip a node except at the end where there is a direct connection.

This leads to a significantly more complex-looking connectivity, as you can see in the diagram above, but the result is that all the links are at most 2 units long. This enables a torus to use link technologies that only work for short links, such as copper traces.

For a 1D torus that is 4 wide in particular, there is an even simpler solution with very short links:



This simplest torus shape is what we'll use twice in the networking proposal later in this chapter.

Router topology

A router is a node with many ports which can transfer data between the ports, connecting the nodes that connect to it in a flexible way. This is an 8-wide router topology, with the router above and the 8 other nodes below:



Router topologies can get highly complex once you need to connect more nodes than there are ports on your router, so that you need to connect routers to each other using yet other routers. I won't get into this area in this doc - you can easily find material on this elsewhere.

Link types

The economics of the networking for your chip depends critically on what type of link you choose to use. This section discusses a variety of link types.

In general, the shorter the distance, the more bandwidth you can get per dollar. This isn't just that longer cables require more material to make - you end up needing completely different and more expensive solutions the longer the distance is. This is also true in home use of technology in ways that aren't immediately noticeable. E.g. a long (active) USB cable is a whole different level of technology from a short (passive) one, but from the outside they all just look like USB cables.

The list below is ordered by the distances involved, from short to long. This also roughly orders the links by bandwidth per dollar, from high (good) to low (bad).

Connection on the same chip

The highest bandwidth, lowest latency and cheapest connection you can get is between two components on the same chip. This factor is in large part what leads to creating very large chips. The economics for this are good up to a certain point. Larger chips are more expensive in terms of yield, so that limits things, though see Cerebras' wafer product as a counterexample on that.

Chip-to-chip connection via silicon bridge

Two chips are placed right next to each other and connected via another small piece of silicon that is placed on top of both chips. That small piece of silicon is a small chip in itself, leading to increased expense for this solution. This is a technically difficult method because the 3 chips in this scenario have to be aligned with extremely fine precision.

It might seem more sensible to connect chips directly at the side, but this isn't normally done because chips are printed from the top, so there is nothing on the side to connect to. You might imagine printing something on to the side, but printing something onto a third chip entirely is much easier - you can do that using existing technology for making chips.

This is how multi-tile designs like Blackwell are done and it is also how HBM is connected to a chip. For HBM the connecting chip is called an interposer and the interposer is one part of the increased expense and complexity for HBM compared to GDDR memory.

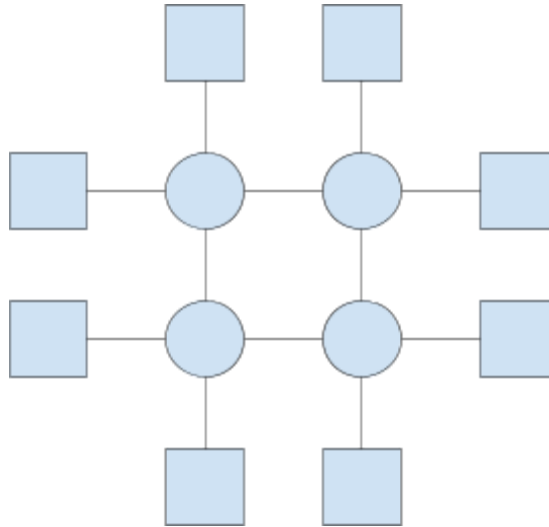
Connection via chip pin

Your chip has high-bandwidth pins coming out of it. This is how a chip is connected to memory, network and any other components like an SSD.

Connection via passive copper trace

These are lines of copper on your motherboard aka Printed Circuit Board (PCB). Copper traces can be cheaper and higher bandwidth than copper cables as long as the distance is quite short. One reason for this is that the connection from a copper cable to your chip is going to be carried over a copper trace, so a copper cable is really cable + trace, not just a cable. Be aware that if you need a very high bandwidth for copper traces delivered to one central point, such as the pins of your AI chip, you will need a more expensive PCB with many layers.

If you can place your AI chips close to each other on the same PCB, then you can use short passive copper traces to create a very fast network between them. For example you might use a layout like this to connect together 4 chips, where the circles are chips and the squares are other components, like memory or an SSD.



The benefit of this particular layout is that the copper traces can be very short.

Connection via active copper trace

If you need a high-bandwidth copper trace to go far on your motherboard, which just means more than a few centimeters, you may need repeaters to strengthen the signal along the way, making the copper traces more expensive and higher power.

Direct PCB-to-PCB connection

Just as for chips, you can place two PCBs right next to each other and connect them in some way. This connects copper traces from one PCB to the next. This is what I mean by “snap-together” motherboards.

One option is mezzanine connectors, where one PCB is placed on top of the other at the edge, connecting at the edge, creating a gradual staircase of PCBs going up (or down) or going up-down if you place the next PCB edge over and then under and then over again and so on. Another option is placing the boards right next to each other at the same level and connecting via some sort of mechanism at the side of the board, e.g. a cap that connects pins from the edge of one board to the next.

Connection via backplane

In this scenario, multiple PCBs connect into a backplane PCB which connects them together. This kind of connection is usually at a 90 degree angle. This is the configuration for components connected via PCIe on a traditional motherboard, with the traditional motherboard acting as the backplane for the PCIe components. For very short distances the backplane may be able to passively connect traces between the other PCBs. For longer distances, the backplane has to be powered. You can also imagine placing a router chip on the backplane, creating a routed topology without the need for expensive cables.

The backplane is usually done with some kind of connector, so that the other PCBs are not permanently attached to the backplane. So the failure domain is much reduced for this approach than for having one big PCB, because if one of the connected PCBs fail, that connected PCB can be replaced individually - you don't have to replace the whole system. The backplane itself can also be replaced without throwing away the PCBs it connects.

The limitation for a single backplane is how big you can make it with good economics and how close together you can place the PCBs it connects. If the connected PCBs require a large air-cooled heat sink, then you will not be able to pack that many of them onto one reasonably-sized backplane as they will need to be physically widely apart to make space for the heat sinks and fans. If you use water cooling, you could potentially fit a large number of PCBs on one reasonably sized backplane.

Connection via copper rod

You can connect two PCBs together at a distance using a rigid rod of copper or other material, i.e. a cable that is rigid instead of flexible. Such "cables" can be less expensive and lower power, but this is not a standard method, so you may need a large order to make it economical. You might consider something like this to create a 3D arrangement of tightly packed chips. It will usually be more standard and simpler to use flexible cables, but this is another option.

Connection via flexible copper cable

The next step up in terms of increased expense and distance is flexible copper cables. When people speak of "an ethernet cable" this is usually what they mean, though ethernet is a protocol that can run on any type of link. So-called twin-ax cables are also in this category of flexible copper cables. If you are running your highest bandwidths across flexible copper cables, as is standard, you may be doing something that could be done more cheaply using one of the alternative approaches above.

Connection via optical cable

Optical links are superior for long distances since they have less degradation of the signal across long distances. Google and more recently also Nvidia use optical links to provide high bandwidths for their AI chips. This is the most expensive solution.

Why would Nvidia and Google use the most expensive solution? Because it is the easiest to work with and neither product is cost optimized (both companies will certainly claim to be optimizing for cost, but it isn't true - both products are optimized to feel familiar and convenient to use, and cost takes a back seat to that because profit margins are very high). An expensive optical cable can let you connect an expensive accelerator to an expensive router 100m away at maximum bandwidth, so this way your expensive datacenter can be physically arranged any way you want (within reason). That's very convenient and a genuine advantage. If you try the same thing with cheap on-motherboard copper traces, you have to space things within centimeters, not 100 meters.

Optical cables, optical ports and optical routers are expensive, but GPUs and TPUs are even more expensive than that, so the expensive networking isn't such a high percentage of the overall cost.

Perhaps you will choose to use optical cables, too. But do you really need them? I'd suggest to see if you can't find a more economical solution.

The specter of multi-hop waste

You might think that the bandwidth of your network is the sum of the bandwidths of the links. This is the ideal case, but you may not be able to realize this. For example, a routed topology will never be able to do this - it always wastes at least half the bandwidth and a large routed topologies with several levels of routers will waste much more than that. This section goes into detail on why this is.

Never in my life have I heard anyone say that routers are inherently at most 50% efficient, so I think that this is not completely obvious. So I'm going to explain it in some detail, even though this is really simple.

During a given AI workload, the average useful bandwidth per chip is:

$$\text{average_link_utilization} * \text{all_node_links_bandwidth} / \text{average_hops}$$

This formula is simplified but is accurate enough to give the right intuition.

Link utilization

Average link utilization is 100% if all links are used to capacity all the time in both directions. Idle time on a link reduces the utilization. If a link is only used 80% of the time and only in one direction, then utilization is only $80\% / 2 = 40\%$.

All node links bandwidth

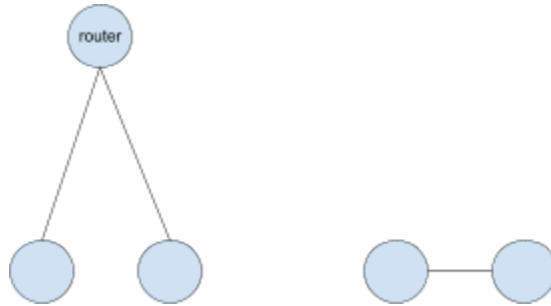
We are calculating a per-node average number, so this is the sum of the bandwidths of all links on a single chip.

Average hops

This is how many links a bit has to traverse on average to get to where it needs to go.

For a topology with a single router, the average hops is always 2, since the path is always chip -> router -> chip. If only neighbors communicate, the average hops for a torus is 1, since there is no router in-between chips. So a router is always at most 50% efficient, while a torus has the potential to be 100% efficient.

Consider this diagram:



On the left, two chips are connected through a router. On the right, two chips are connected to each other directly. The bidirectional bandwidth between the two chips is the same either way. So half the bandwidth is being wasted on the left. We bought 2 cables (and a router) and only get 1 cable's worth of bandwidth on the left (here including ports and any other associated overhead in "cable").

Here's another scenario:



On the left you see 4 chips connected via a router and on the right you see 4 chips connected as a torus. In this case, the number of cables is the same either way, so you might think this is equally efficient. But no. Every chip on the right has 2 cables connecting to it, while on the left you only see 1 cable per chip. So bandwidth per chip on the right is twice that on the left. So it's the same as before - the router is wasting half the bandwidth and that's before considering that using a router also requires the additional expense of buying a router, so it's even less than 50% efficient economically.

Another way of understanding it is that the router is not a useful destination in itself, so every port on the router and all of its bandwidth is wasted. Every port on the single router is matched by a port on a chip 1:1, so exactly half the bandwidth is wasted. Or potentially more if you have multiple levels of routers where some cables connect a router to another router, at which point the bandwidth of both ports that such a cable connects is wasted.

Once you run out of ports on a single router, you need to use multiple routers and perhaps multiple levels of routers. There are many schemes for how to do this and they in general tend to be complex and involve long cables. Routers are already expensive and needing many of them in this way makes routing even more expensive for the bandwidth that it provides. Routed bandwidth is expensive bandwidth.

Going back to the formula at the start of this section, the reason the torus can be twice as efficient as the router is that `average_hops` can be 1 for a torus while for a router it is always at least 2. For the router, the distance can be greater than 2 if you use multiple levels of routers. But let's focus on a single router, which is the best case for a routed topology, where the distance is always 2.

All this is why Google TPUs use a torus topology: you get a 2x on bandwidth, you don't have to buy a router and the links can be shorter and therefore cheaper.

Hold on, you might say, a workload on a torus *can* be 100% efficient, but is it always? No. The torus loses efficiency if communication between nodes goes further than a neighbor. If two nodes on opposite ends of a 16-wide torus have to communicate with each other, then that will require 8 hops. So a torus can achieve an average hops of 1, but it can also be higher. For a large torus, it can be much higher. This is why routers exist, despite their inherent inefficiency - they are still less inefficient than a large torus for random traffic.

The average hops on a torus depends on the algorithm and the workload. The neat part is that almost all AI networking *can* be done so that only neighbors communicate, yielding 100% efficiency on a torus! But MoE workloads are not like that. So before MoE, toruses had a massive advantage over routed topologies for AI networking. After MoE, it is a more complicated question. There is more on MoE in the section on networking for MoE below - the conclusion will be that *small* torus dimensions are still good for MoE, but not large ones.

The big 3: Reduce-scatter, all-gather and all-reduce

The big 3 classic AI networking operations are: Reduce-scatter, all-gather and all-reduce. These cover almost everything an AI workload needs to do with weights and activations over the network. Also, the big 3 can be done using only neighbor communication, so they can be 100% efficient on a torus topology. Nice!

In more recent times, the big 3 have become the big 4, adding MoE. Unfortunately, MoE workloads cannot be done using only neighbor communication, degrading efficiency on a torus. This section concerns just the big 3. MoE is discussed in the next section instead.

All-gather

For all-gather, you have X bits on each of N nodes and you want all the nodes to have all NX bits. So if a node has a 4 and the other node has a 6, all-gather will put (4, 6) on both nodes.

So this is an all-to-all broadcast. Let's look at how to do this on routed and torus topologies.

All-gather on a routed topology

Here is the algorithm for this on a single router: Every chip sends its X bits to all the $N-1$ other chips. So every chip sends and receives $(N-1)X$ data on their one bidirectional link, so total time is $(N-1)X$ divided by the link bandwidth.

The average communication distance for one router is 2, so this algorithm wastes half the bandwidth. If you need several levels of routers, the algorithm becomes more complex and the communication distance increases above 2 and so you waste even more bandwidth. The exact average distance depends on how you wire up the extra routers. There are many possible different schemes here and each requires a custom algorithm.

All-gather on a torus topology

Here is the algorithm for this on a 1D torus: Each chip sends $X/2$ of its bits to the left and the other $X/2$ of its bits to the right. Then every chip sends all the bits it received in the previous step forward, so the bits from the left neighbor is sent to the right neighbor and vice versa. After $N-1$ steps of this, the all-gather is complete and all the nodes have all the bits. So every chip sends and receives $(N-1)X$ data on their two bidirectional links, so total time is $(N-1)X$ divided by the link bandwidth. This is assuming you pipeline to hide link latency (which you should) and there is enough data to transfer to cover the round-trip latency (which there should be).

The average communication distance for this is 1, so this algorithm doesn't waste any bandwidth. This is why it is twice as fast as a routed topology using the same number of cables and ports and without having to buy a router.

The algorithm is similar on a D-dimensional torus. For simplicity, let's say $D=2$. First, do the 1-D algorithm on the rows of the torus. When that is done, take the resulting XD bits per chip and use the 1D algorithm on the columns of the torus. This will correctly distribute all the XN^2 bits across the torus.

But this only runs at half efficiency, since we left column links idle during the first step and the row links idle during the second step. To fix this, divide the data into two halves and run the exact same algorithm in parallel but with row/columns switched. This will run two copies of the same algorithm on disjoint links. The result is 100% efficiency and this is again twice as fast as the routed topology.

For D dimensions, you do the same thing, but with D steps and with D parallel invocations of this algorithm. The result is again 100% efficiency. This is in contrast to routed topologies that get less efficient for this the larger they are.

There are various academic reference for this, [here is one](#).

All-gather on an asymmetric multi-dimensional torus topology

What if the dimensions in a D-dimensional torus do not have the same bandwidth? This is going to be one of the configurations that we'll consider in a later section, in part because this works out quite well.

Suppose we have an $N \times N$ torus where the row (first) dimension is slow and the column (second) dimension is fast. Suppose more precisely that the fast column dimension links have precisely N times as much bandwidth as the slow row dimension links do.

In this case we cannot efficiently run two instances of the algorithm as we did on the symmetric torus, because the first step in the column-first instance will be done much sooner than the first step in the row-first instance, leaving the fast column links idle for a long time as they wait on the slow rows to get done. So what to do instead?

First, we are going to pipeline the operation, so that we have many pieces of data each of a smaller size X . Then we run the 1D algorithm on the slow row dimension on the first piece of X data, then we run it on the second piece and so on until we have done all the pieces for rows. As soon as the first piece is done for rows, we start running the 1D algorithm for columns on the result for the first piece, then the second piece and so on. This leaves some links idle at the beginning and the end (as pipelining always does) but as long as we have many pieces, this is a negligible effect.

How fast is this? Well, when we run the 1D algorithm on the slow rows, the result is that each node now has XN data instead of X data. Then columns now have to do an all-gather on the XN data, yielding a result with XN^2 data per node. So the column links have to transfer N times as much data as the row links do. But the column links also have N times as much bandwidth as the row links do, so the pipeline ends up being balanced and we achieve 100% utilization of the links, except for the first piece and the last piece of data, but this overhead can be made negligible by having many pieces (easy to do) or by co-scheduling different such pipelines with partial overlap (hard to do). So an asymmetric torus of this kind can still do all-gather at near-100% efficiency.

Reduce-scatter

For reduce-scatter, you have NX numbers on each of N nodes and you want to sum them all up and distribute the results of the sum evenly across the N nodes, leaving each node with X distinct numbers from the sum. So if a node has numbers (1, 2) and the other node has numbers (3, 4), then the sum is (4, 6) and the first node would end up with a 4 and the second node would end up with a 6.

Believe it or not, you can do exactly what I wrote above for the all-gather, but in reverse. So the order of operations is reversed and whenever the all-gather algorithm would take a number and duplicate/send it to another node, instead a number comes in from that node and that number is added to the local number, yielding one number that is then itself passed on in the opposite direction of all-gather. Everything works out the exact same in this way, including how fast it is. So I won't get into details on this - it's just all-gather in reverse with sums. So also for reduce-scatter, torus topologies run at 100% efficiency and routed topologies run at half speed at most.

All-reduce

For all-reduce, you have X numbers on each of N nodes and you want to sum them all up and distribute the entire sum vector of X numbers to all N nodes. So if a node has numbers (1, 2) and the other node has numbers (3, 4), then the sum is (4, 6) and both nodes end up with (4, 6).

If you first reduce-scatter and then all-gather, you will have performed an all-reduce. Doing it that way is also optimal, so that's how to do it. So also for all-reduce, torus topologies run at 100% efficiency and routed topologies run at half speed at most.

Ring attention

Ring attention is technically distinct from the above 3 operations, but the communication pattern of only needing to talk to neighbors is similar, so you again end up with routing running at half speed compared to a torus. With some complications, this also works on a bandwidth-asymmetric torus.

Meshes

You need somewhat different algorithms on a mesh topology. It turns out that these algorithms run at half speed of a torus - same slow speed as a routed topology. Yet a mesh only saves a few links compared to a torus, so tori are preferred for AI over meshes.

Where you might end up with a mesh anyway is if you have a big torus but you only want to use a subset of it. The subset will not have the wrap-around links for the dimensions where you took a subset, so you end up with a mesh in those dimensions. This can be fine as long as your use-case runs OK with a half-speed network for operations like all-reduce.

Mixture of experts versus network topology

Mixture of experts is the spanner in the gears for a big torus topology because mixture of experts requires communication beyond just neighbor-to-neighbor communication. You need to take multiple hops.

For a single router, the average communication distance is always 2. This is inefficient compared to neighbor-only communication on a torus, where the average distance is 1, but if the torus needs to send data to non-neighbors, then that increases the average distance above 1 and, perhaps, above 2, at which point the torus is even more inefficient than a router.

So the question is: what does the average communication distance become on a torus with MoE? Suppose MoE needs to send tokens around uniformly at random. Then the average distance on a 4-wide torus is 1. That is because there is a 25% chance that the token is already on the right node, so that is distance 0, there is a $2 \times 25\% = 50\%$ chance that the token needs to go to one of the two neighbors and there is a 25% chance that the token needs to go to the opposite node at distance 2. So the expected value is:

$$0 * 25\% + 1 * 25\% + 1 * 25\% + 2 * 25\% = 0 + 1/2 + 1/2 = 1$$

The same average distance on a router is:

$$0 * 25\% + 2 * 75\% = 1.5$$

So the torus still wins. The true calculation is a bit more complex due to the possibility of single-link congestion on multi-hop routes in the torus, which can also happen on a router (suppose all tokens go to the same chip). Overall the torus still wins. So we can see that it is not so simple as routers having an advantage for MoE.

MoE routing is not uniformly random, it depends in part on properties of the document that the token comes from, so you might also be able to exploit the statistics for this to significantly improve the average MoE distances on a torus (just relocating the token is not so easy, since you then have to move the KV cache with it, but that's OK if the whole document prefers certain experts over others). You may want to do this even for a router if you end up doing MoE across multiple levels of routers.

When comparing to a router, keep in mind that a large routing network cannot be handled by a single router, so the average distance for large router topologies will rise above 2. How much above 2 it will be depends on the exact topology used (how you wire the routers together).

What about a 4x4 torus? Well each dimension introduces the same average distance, so the overall average distance is $1 + 1 = 2$. That's roughly the same as for a router and you didn't even have to buy a router. And for a 4x4x4 torus the average distance is 3, which is worse than for a single router, but still you don't have to buy a router. So there are both pros and cons here.

Another thing to notice is that you might have more experts than you have nodes and many tokens activate multiple experts. This means that the same tokens may need to go to multiple nodes. Depending on how many nodes there are and what proportion of experts are activated per token, this can much reduce the number of wasted hops on a torus, shifting the balance in favor of the torus again. In some cases like this, MoE traffic might start to look like all-gather traffic, at which point the torus is again strongly preferred.

Yet another relevant factor here is within-layer (tensor) parallelism within an expert. If experts are large enough, you can parallelize them across several chips to reduce token latency. This again shifts the balance in favor of toruses, as toruses are most efficient for this kind of parallelism. So we see advantages for tori both for a large number of experts (where data must be sent to many nodes) and for a few (where we may be able to parallelize).

So a 4x4x4 torus is still a good choice even with MoE. Perhaps not coincidentally, you can rent Google TPUs in this exact topology. They call it a cube.

Where there is NOT an advantage for toruses is if the torus is quite large. You can [book a 16x16x24 torus of v5p TPUs](#). The average MoE distance on this thing will then be $16/4 + 16/4 + 24/4 = 14$, which is really not great. Even here, you might be able to parallelize the last 24

dimension, bringing the MoE distance down to $16/4 + 16/4 = 8$. This is $16*16*24 = 6144$ chips, so a router network will also need many routers, so the comparison distance is not 2 for a router, it is something larger. The TPU torus network is so fast that MoE probably still works well enough, it's just not as cost effective in terms of how much bandwidth had to be provisioned.

Long story short, MoE is fine for smaller tori, but not for huge ones. If you stick to $4x4x4$ tori, or smaller ones, you should be OK versus a routed topology - also for MoE.

A proposal for a networking approach

I can't tell you how you should network your chip. It depends on too many details about your chip and fab prices that I don't know what they will be for your chip. What I can do is describe an idea for an approach that I think is interesting. I'm not saying you should necessarily use this configuration, but it might be a worthwhile exercise to do the cost math to ensure that your solution is superior to this one.

The topology

The base version of the idea is a $4x4$ torus of $4*4=16$ chips. Let's call this a *donut*, since it is a 2D torus. If you buy 16 off-the-shelf routers with N ports, you can connect together N donuts, to create an $Nx4x4$ topology that we'll call a *box*. So one router connects together N donuts at the same position in each donut, e.g. position $(2, 0)$ in each of the N donuts will be connected on the same router.

The idea is to wire this up so that each $4x4$ torus is densely packed, so that distances are short. This enables using links that can be both cheap and high bandwidth. High-bandwidth routers and cables are expensive, so we won't use any of those. Instead, we will use cheaper lower bandwidth routers, so that the N dimension will have much less bandwidth per link than the torus does. This way we have high bandwidth in the torus and everything is still cheap. If $N=16$, this scales to $16*4*4=256$ chips. That should be enough for most inference use cases.

It is not a big problem that the N dimension is lower bandwidth. A single donut will often be enough parallelism already. If not, then you can use the N dimension for between-layers parallelism, which has much lower bandwidth requirements (if coupled with within-layer parallelism). Note that "lower bandwidth" is by the standards of AI accelerators - this may still be a very fast router compared to most other uses. It just doesn't have to be some exotic technology to reach bandwidths not otherwise possible.

Big toruses have a problem for MoE workloads, but a $4x4$ is small enough that this is not such a big issue - see the previous section on MoE versus network topology. Average uniformly random pair distance on a $4x4$ is 2, almost equal to the 1.5 you get with a router.

Large toruses have a problem with failure domain. If you put 256 chips in a torus, and one of them fails, then you've disconnected the torus so the entire 256 chip system has a problem. There are ways around this, but it's a menace to deal with. In this approach, the failure domain

is limited to a donut. If a chip fails, only the donut that it is part of has to be serviced. This is because the donuts are connected via routers and having a missing donut on those routers doesn't interfere with the functionality or connectivity of the remaining donuts. This is a similar effect to Google's fancy optical routers, where they achieve similar resilience to failed chips, but this is using cheap off-the-shelf regular routers.

You are going to have to provision a datacenter network connection to your AI chips anyway. The Nx4x4 approach extends this connection to be part of the AI network instead of something external. You will reach the datacenter network also through these routers.

Loading weights for a big model onto an AI chip can take quite a while, so you end wanting to have a fast network from storage nodes to your AI chips. Yet the steady state usually doesn't require so much bandwidth - stored KV caches can be large, but you shouldn't be exchanging *all* the KV caches at the same time. So a network that is provisioned to load large weights quickly lies idle most of the time. The Nx4x4 topology reduces the waste here, since the N dimension serves both to transport weights and, potentially, as part of the AI chip network itself.

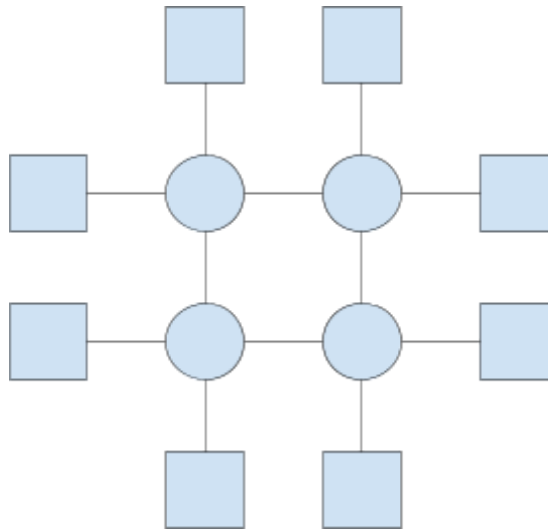
What happens if a *router* fails? It might seem that the failure domain in this case is the entire Nx4x4 topology, since we will be missing the connection to one chip in every donut this way. However, every chip has 4 neighbors in the 4x4 torus, so you can route traffic to chips with a failed router through the torus, with each of the 4 adjacent routers taking up 1/4th of the slack of the failed router. This will consume a bit of bandwidth on the torus, but this is small as the torus network is much faster than the router network and each link is impacted by only 1/4th of that. You might also pull in bandwidth from further away in the torus than just the direct neighbors. You can also use this approach to temporarily increase bandwidth to load weights onto a subset of the topology - if a user rented a single chip, potentially you can offer up to 16x the bandwidth (or 64x if you use Nx4x4x4) for peak transfers like loading weights.

Many customers will not be using an entire 4x4 donut. Suppose a customer starts a model running on a 4-wide ring (1/4th of a donut) and has to load a large amount of weights. The other chips on that donut of 16 chips probably are not using their router links to capacity, so you can temporarily boost the weight loading bandwidth by as much as 4x by using all the routers and using the otherwise idle torus links within the donut to deliver that data to the 4-wide subset that this customer booked. That won't work if the other customers on that donut are using their router links to capacity, but this is unlikely. So it is a significant but only best-effort optimization.

Physical realization of Nx4x4

This part of this section is a proposal for a physical networking realization of the Nx4x4 topology that is intended to achieve high bandwidth at low cost. I am not a hardware engineer and this particular section is somewhat beyond my actual expertise - never the less, I think this proposal is interesting and perhaps you will, too.

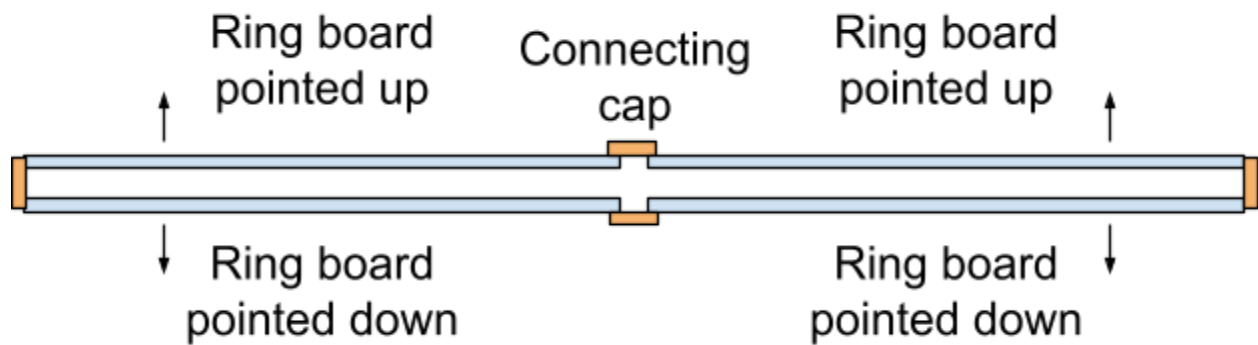
If a 4x4 is a donut, let's call a 4-wide torus a *ring*. The donut will be built out of rings. Let's start with looking at one ring. A ring is realized on a single motherboard like this, where circles are chips and squares are external components, like memory or SSDs.



This amortizes the cost of the motherboard across 4 chips and makes it cheap to connect these chips together in a fast 4-wide torus, since the chips are close to each other on the same motherboard. If you are using air cooling, there may be some thermal challenges here, but something like this should be possible. You could imagine many more chips per motherboard to push these benefits, but since these chips will be soldered onto the motherboard, this will inflate the failure domain from 4 chips to many more chips.

We will build a donut by connecting 4 rings together. There are a bunch of ways of doing this. I will propose a configuration that results in very short distances to maximize bandwidth and minimize cost.

You can imagine placing 4 ring boards next to each other and connecting them at the left/right edges using pins with some kind of cap to connect them. That works, but this will create a line (mesh) not a torus. We need to connect the leftmost ring board to the rightmost ring board, but these are far from each other. For TPUv2 and TPUv3, a problem similar to this was solved using an expensive long optical link. I'll propose something else instead: Fold the leftmost 2 ring boards under the rightmost 2 rings boards. Now you can close the loop at the right side, since the two ring boards we wanted to connect are now right next to each other. It will look something like this:



An alternative is to use a folded torus with skip links and all boards pointed up, but this causes longer distances between connected chips, which may require additional expense. Ideally, you will be able to make the ring boards with a small enough width so that the chip -> connector -> chip path is so short that it can be a passive copper trace connection all the way from one chip to the next. Short passive copper traces are both cheap and fast. That's the point of this up-down configuration.

Each ring board will have 4 network ports, one for each chip. So that will be 16 ports per donut, and each of the 16 ports will connect to a different one of the 16 routers in a box of 16 donuts.

The approach suggested here is minimal in per-chip cost and also has minimal development and other fixed costs. A big benefit of low-cost high-bandwidth networking is to allow you to spread equivalent performance across many economical medium-sized chips instead of expensive huge chips or expensive tile packages. The downside is that the software now needs to be aware of the torus topology to capture the full bandwidth. Though on a 4x4 torus, the average path length for random node pairs is 2, which is nearly the same as for a single router, so even software that does not take the topology into account may run quite well.

[[[Asymmetric bandwidth toruses for nerds You might consider an asymmetric torus, where the on-motherboard torus dimension has the most bandwidth and the between-motherboards dimension has 1/4th that bandwidth. This works well for all-reduce, reduce-scatter and all-gather workloads. But I have not so far been able to figure out a way to make an asymmetric torus work well for MoE workloads, so I don't think that I can recommend this, even though I would really like to. If you can figure out some way to make an asymmetric torus work with MoE, and there might be a way, then I think that a 4D $N \times 4 \times 4 \times 4$ topology with 1/4th the bandwidth at each dimension becomes very interesting, as then the routers at the N dimension only has to provide 1/64th the bandwidth to keep up. Routers are expensive, but if they only need to run at 1/64th the speed, then their economics start making a lot more sense than otherwise.

Well, there is *one* way I can see how it might work: If you could guarantee that each expert in MoE will be large enough to parallelize 64 ways, then that'll work, since then you can use within-layer (within-expert) parallelism for the 4x4x4 dimensions, leaving just a routed topology for MoE, but the premise of 64-way parallelization within an expert seems difficult to guarantee. Perhaps possible, though. If that works for you, I would seriously consider an asymmetric

Nx4x4x4 torus. The wiring can be, in order: traditional routers with long cables, short copper cables, between-MB edge-to-edge connections, on-MB connections.]]]

[[[Tool use, agentic orchestration and replacing the host for nerds You may recall that part of my recommendation is to not have a host computer/CPU to control the AI chip. Instead, I propose to put modest CPU cores directly onto the AI chip so that it can control itself. However, we *do* at times need heavy CPU compute *somewhere* in the datacenter to support AI - this is something that happens especially for AIs that use tools and where those tools are very heavy on CPU compute. E.g. compiling and running large amounts of code can require a lot of CPU cycles and this is something that AIs now do - write, compile and run code.

You may have heard that it is agentic AI that requires heavy CPU resources. That's not how I see it, since more precisely it is *tool use* that can explode CPU requirements. The relationship to "agentic" is that agents commonly use tools, so the idea is not quite wrong, either. Here "tool use" is an expansive concept, including e.g. vector database retrieval or searching Google.

Another claim I've seen is that agentic *orchestration* heavily spikes CPU requirements. Here "orchestration" doesn't mean tool use or running the tools themselves, it means just keeping track of the interaction between multiple agents and tools. This doesn't make much sense to me, since keeping track of these interactions shouldn't require a lot of CPU cycles. Orchestration might slow things down on the CPU if you implement it in Python on a single thread, though, or some such cockamamie approach - such things are common in AI since AI practitioners frequently are not also experienced software engineers. So this can happen, but it shouldn't be happening. So how did we end up with the idea that agentic *orchestration* is a primary CPU bottleneck? I think the deeper reason for all of us talking about "agentic orchestration" as a bottleneck, instead of "tool use", is that "agentic orchestration" sounds new and fancy, while "tool use" sounds old and boring. So you seem more sophisticated if you use the wrong term, saying "agentic orchestration" when it's really primarily about "tool use" in terms of the potential for very high CPU use. I'm happy to be educated on this if I'm wrong, but I don't think I am. It's really just because using the wrong term sounds more fancy.

However, tool use is real and not going away, so we *do* need access to heavy CPU compute and preferably in the same datacenter as where the AI accelerators are stored. The onboard CPU cores I recommend should be enough for "agentic orchestration", but not for heavy tool use like large compilation jobs triggered by an AI. So now it sounds like maybe we need to bring back the host computer to house our AI accelerator? That will supply a bunch of CPU compute right next to the accelerator. I do not recommend this. The idea of the host is that it controls the device and there is no reason to reintroduce that. Many AIs do not use tools so heavily, so in this case you'll be stranding capacity on the host, and with heavy enough tool use, you will need far more than one computer, the host, to satisfy the CPU load. So the host is still a bad idea, I think. You simply need the datacenter to have CPU capacity somewhere and enough bandwidth between the AI accelerators and the CPUs to have this work out. You also need reasonable latency, but this is not hard within one DC. When I worked at Google, all compilation happened "in the cloud", never on my local machine, and you can just do the same thing with AI. If the AI

will do retrieval, you of course also need access to CPUs that can execute this. This can happen partly locally, using the local CPU cores and SSD that I recommend provisioning on your AI accelerator, but for very large databases (e.g. searching the internet) you may need access to outside resources for this as well. You also need a way to transfer KV caches and weights to your accelerators. So no matter what you do, you do need a nice connection to the rest of your datacenter and general datacenter capacity, and that is all that tool use requires, too.

There is a caveat here for video (de)compression and encryption. For most data you can stream it elsewhere for processing, but video in particular requires (de)compression to happen locally, since uncompressed video can be too large to reasonably transfer over a network. So you'll need to ensure that your AI system as a whole, somehow, is capable of doing fast enough video (de)compression in particular, which can be a heavy-duty computation. Same thing for encryption, which doesn't change bandwidth requirements, but where for policy reasons you may want to insist on encrypting all data in motion, which then has to happen locally.

There is a common misconception that the AI devices will be idle while tool use or other CPU work occurs and then, presumably, the CPUs would be idle while the AI device works (this is commonly what happens for AI chip hosts - they are mostly idle most of the time, a huge waste). There is no need for that. You just need to ensure that your AI device has other AI work to do while it waits and the same for your remote CPUs. This has similar tradeoffs as using pipelining to avoid waiting on the network inside a transformer - see the chapter on parallelization.

There is one idea I like that somewhat revives the idea of the host, but in a different and more economical way than what is traditionally done - place a host/CPU on the other end of a short twinax cable from your AI accelerator, perhaps in a 1:1 ratio to your AI chips. But this CPU will not be a host, since it will not control the AI device (the CPU cores on the device do that). It will simply be a general CPU available for general DC work that may be unrelated to AI. However, when the AI uses tools, or otherwise requires extra CPU oomph, the datacenter software will try to preferentially place that CPU load on CPUs close to the AI chip, e.g. perhaps right on the CPU at the other end of that short (i.e. cheap) twinax cable. You could even connect the CPUs together in a similar Nx4x4 topology, this approach is so cheap that this is feasible, especially if you are making your own CPU, which creates something like a 4xNx4x4 topology if you connect four CPUs on a ring board to each different Nx4x4 topologies. This can also expand the best-effort reach of being able to service peak bandwidth, since you can wire it up to connect otherwise independent tori. Though just placing your CPU clusters elsewhere in the DC, as is traditionally done, works, too, and will likely be quite a bit simpler to manage. **]]]**

[[[Alternative for router-loving nerds If you have the scale for it, an alternative is to make your own 16 port router chip on a backplane or you might try to buy one with the right properties and cost. This starts looking more like what Nvidia is doing. You'll preferably need water cooling for this, so that you can pack 16 ring boards onto a not-too-large backplane PCB - e.g. 2 cm spacing yields a 32 cm PCB dimension, which is perhaps not too large to still use passive copper connections. The backplane will have four 16-port router chips on it, to create a 16x4

topology. You can also have two backplanes, one on the left for the 2 chips on the left half of the ring board and one on the right for the 2 chips on the right half of the ring board. This will minimize distances and on-motherboard wire congestion. This creates a larger topology without making MoE routing worse and still maintaining economics. You can still use cheap, slower routers on top for an Nx16x4 topology or potentially just wiring the 4 custom routers up directly into an external router topology (so in this case you'd connect to the routers directly, instead of adding another link to each chip) that is less economical on a bandwidth per dollar basis, but which is amortized over now $16 * 4 = 64$ chips. **]]]**

[[[Proper amortization math for nerds Suppose you make a network that is 1/8th the price of what Nvidia offers, but your chip is also 1/8th as powerful as an Nvidia GPU. Have you saved on the network? No, you haven't. To match an Nvidia GPU configuration, you'll need 8 of your configurations and also therefore need to buy 8 times as much networking gear. So you haven't improved anything (except perhaps it's a benefit that your solution can scale down further), just matched Nvidia (and at same throughput, your token latency might still be worse). Therefore the proper math is networking dollars spent per unit of *realized* performance from your system. This is yet another obvious-sounding-when-stated connection that maybe isn't that obvious until you hear it: networking cost and per-chip performance cannot be considered independently. **]]]**

Co-design

Co-design is optimizing your software and hardware together - each needs to support the other. For any given problem, it could be solved in software or hardware or a combination, and you need to figure out in each case what is right for your particular AI chip. That's co-design.

Good co-design is not natural for either one of software and hardware professionals. There seems to develop a natural instinct that some problems should be solved in hardware and some in software, and that's "just how it is". The particular boundary may differ between companies, but there does tend to develop such an instinct. This may surface as insisting on doing it in the instinctual way even when pressed on it in a conversation, but that is not the harder problem to deal with - the harder problem to deal with is that this conversation never occurs in the first place because no one considers that the problem could be solved in another way. Also, while there are many common elements, some parts of software engineering and hardware engineering are quite different, so it is rare for one person to have expertise in both, which ideally you would have but realistically this isn't something you can likely hire - the best people in your company on either thing are likely going to not be specialists in the other thing at the same time. So you need them to talk to each other. That's co-design.

A good rule of thumb is that the hardware team should never design a software-facing interface on the chip without consulting the software team about that. Many things should be designed as a collaboration between software and hardware, not as hardware making a proposal that software then has to decide on (this tends to lead to software-tolerable designs, instead of software-positive designs). Many more things are software-facing than you might think. *Anything* to do with performance is software-facing, for example. *Any* hardware limit is probably software

facing. If you do not have this rule, I can just tell you right now that your software-facing interfaces are probably going to be trouble to use in software. The less you talk to software people, the more trouble it is going to be later. This is one reason you may want your hardware team to sit physically close to the compiler / software dev team. Shouting over the divider: “Hey, Steve, what about X?” “Oh, yeah, that’s fine, we don’t care about that, but how is Y coming?”, “Oh yeah, about that ...”. You don’t want your “formal co-design process” to squeeze out these very efficient and direct conversations. Neither do you want important decisions to be made without anyone noticing in this way - but if the alternative is that a hardware engineer would just guess what software needs, as is otherwise very common, then that’s still better.

My experience with co-design

This section is a bit specific to my own work, but it has some ideas for co-design overall.

For a period of some years, I was the software person that would be consulted on most hardware matters for TPUs. I eventually transitioned myself out of this by connecting people more directly as the XLA software team became larger with more experts in various areas. A few examples of what I worked on, so you can get a sense for what I was doing: I designed the interface for DMAs for TPUv4 to be much more software friendly than previously (the previous functionality required some kind of bit-pattern PhD to understand and also wasn’t general enough), I completely changed how memory protection worked on TPUs (the previous solution was so complex that we never used it, so I simplified it), I got quite a bit into the details of the performance properties of HBM on TPUs and I rejected some changes that otherwise would have made it into the chip on the grounds that it would be too challenging to program.

I also was consulted in terms of how to set hardware limits for many things on the TPU. It ended up as “consulted” but in the beginning, it was more like “discovered”. There are many more examples of needing to set a hardware limit for something than you might think if you haven’t been involved in such work. For some of these, early on in the TPU process, the hardware team had simply guessed and the software team wasn’t even aware that there was a question and that a decision had been made. This is a natural outcome of not enough co-design combined with tight schedules. E.g. how many DMAs should it be possible to have out-standing simultaneously in which contexts? Well, there are many contexts, so you need a limit for each one. And it matters because if there isn’t enough capacity, then a whole core will wait idle for DMA queue space to become available. A single underdimensioned queue could destroy performance overall. Yet grossly overprovisioning everything maybe isn’t the right call, either. Or maybe you need features to enable *one* DMA to do what several DMAs would otherwise do, but that requires complex DMA features, and that still doesn’t get you out of the need of having to figure out what the queue limits should be. Such matters need to be decided as a close collaboration between software and hardware teams - that’s co-design.

Good luck figuring out mathematically optimal answers for these decisions. Instead, it becomes a judgement call - preferably informed by simulations and other investigations, but that isn’t always feasible within time and resource limits. This is a judgement call you cannot make as

well if you don't have any experience actually programming the chip. Which even your software team will not have that much experience of for your first revision of your chip - a challenging situation. But certainly a hardware engineer is not likely to be most qualified to figure this out just on his own, yet in the absence of well-functioning co-design, that is what will happen. So you can see that you need excellent software engineering competence in your company for designing your chip, even before any software is written.

In any case, the hardware team, over time, came to place a lot of importance on consulting their users, i.e. the software team, and became appreciative of my advice as part of that. It also was a way for them to stop endless debates on such hardware limits, that no one could know the real answer to, anyway - we'll just ask software and then they can't complain later if there is a problem. I think this was a well-functioning situation, but it did concern me. It concerned me because, in many cases, the hardware team would simply ask me and then just go with that, on the (very reasonable) argument that my team needed to use the hardware so we need to listen to software. I was for this reason very careful about my responses in these situations, and I think it all worked out fine. Yet the problem is that I'm not a hardware engineer. I do not possess the expertise that someone develops from decades of hardware engineering work, no software engineer can, though of course doing this co-design work did greatly improve my understanding of hardware concerns (I find that if you just think of the math and consider that it has to happen on a 2D plane, that's already enough to understand many things). So it's not quite right to simply take software's word for what is needed, either, because these decisions almost always involve the need for both software and hardware expertise.

What is needed is *respectful negotiation*. To consider: do we really need this? What's the cost? What's the benefit? In both software and hardware - my concern above can then be rephrased as that I had to take both sides of this negotiation myself in some cases. Your software and hardware leads are probably (hopefully!) not going to have a problem doing respectful negotiation with each other. However, individual hardware team members who just want to get on with their work might be happy to let software make some decisions in a hallway that then they cannot get blamed for later and their colleagues will have a harder time contesting because "software said so". So ban that, right? Well, that's not great, either, because continually asking software in this way is a *good* thing - hardware engineers will make better decisions, and become better at their craft, if they can easily access the perspective of the users for their product - software engineers. You will defeat much of this knowledge sharing if you ban one set of engineers from talking to another set of engineers. But it does need to *also* involve respectful negotiation.

If the alternative to a decision being made in a hallway is a hardware engineer just guessing what will work for software, then the hallway is better. You could require *all* decisions to go through a formal co-design process, as an alternative, but then you will cause your hardware team development velocity to grind to a screeching halt. Also I imagine that this is not as interesting to work in this way, so I would guess that there would be a morale impact, as well.

So you can see how finding the right balances here is challenging. Which solution is best for you might well depend on what people you have available in your company for a given project, not necessarily so much on finding some kind of generally optimal process.

One thing that's important is just to share information. During my work on TPUs, I made a huge spreadsheet of all the capacities, bandwidths, limits etc. on each TPU. Previously, you'd have to closely study the spec or go ask someone to find this information. I know that this was useful as I could see that the resulting document received many thousands of visits.

Intellectual quality and good judgement

If you want to figure out what the impact of something, let's call it X, is, you have to include the derived consequences of X. You can't just look at X itself. The error of failing to do this happens *very easily* in co-design. It is the #1 most common cause that I've seen in co-design where something appears to be certainly true but actually it's wrong. Here are some real co-design examples from my own career where experiments were misinterpreted.

What is the impact of op fusion? For example, op fusion is an important optimization for AI accelerators. It involves doing multiple operations on data when you bring it into a faster memory, so that you do not have to bring it there again later. How would you quantify the impact of this optimization? Well, XLA already has extensive support for this, so it should be easy to quantify: turn this optimization on and off and measure the difference. If you do that, you'll get numbers showing a tremendous impact. Yet these numbers, even though they are derived from an actual experiment, are just a fantasy.

The error is that turning off fusion is something that has derived consequences. The most important derived consequence is that, if XLA had not supported op fusion, then never in a million years would we ever have designed XLA as it is. The one change leads to other changes. Indeed, other systems with poor fusion support are NOT designed like XLA. In particular, XLA decomposes many ops into tiny little operations. This is a fine idea *because* of the extensive fusion support. But if you then take XLA *as it is* and just turn off fusion, you've created a truly terrible system. So you are greatly over-estimating the impact of fusion as a technique. Fusion *is* a powerful technique, but the numbers you get this way mean very little.

How much scalar compute do we need? Suppose you are designing the next version of an AI chip that is going to have faster systolic arrays. The question is if you need to add extra scalar compute to keep up with the now greater amount of work that has to happen to service the systolic arrays. Well that's easy, we just take our existing ISA simulator, crank up the clock on the systolic arrays and see if we end up being bottlenecked on scalar compute. That's an experiment, that's the *scientific method*. Science proves it: we *do* need more scalar compute. Or do we? In fact these numbers do not at all settle the matter.

The problem is that increasing the need for achieved scalar computation is something that has derived consequences. One of the consequences is that the compiler team will optimize scalar

computations to a much greater extent, which maybe they hardly did at all before since there wasn't a need for it. Now there is. Maybe just mildly unrolling hot loops is sufficient to keep up. So even if your experiment seemingly implies that you need to double your scalar compute, maybe you don't actually need to change it at all - due to derived consequences. Maybe you could halve it and still keep up, who knows - there might simultaneously be an opportunity in the *opposite* direction.

Can we support dynamic shapes on TPUs? This was a hot topic on the XLA team for a time. Dynamic shapes refers to a situation where the tensor/array shape/bound/size is not known at compile time, so it has to work with all or many different cases.

If you took XLA team development practices *as they were*, and you imagined having to support dynamic shapes, you would have a hard time. But having to support dynamic shapes is something that has derived consequences. The practices and APIs would change to make this activity much easier than it would be before you even started on any of that. So no, we can't support dynamic shapes because it would be impossible. Except, actually, yes, we could, with some trouble, by changing how we do things. If you don't take derived consequences into account, the answers you arrive at will not be the right answers. Dynamic shapes for XLA was always a possibility - the better argument for not doing it was that it was a lot of trouble to do and customers actually didn't require it so strongly as it was claimed that they did.

Though you can also go wrong by taking into account derived consequences that are not necessary. For example, if you consider supporting dynamic shapes in the sense of dynamic array bounds / sizes, you might then say that we also need to support dynamic *rank*, so that we don't even know if the array is 2-D or 3-D when we write a kernel or produce one from XLA. Dynamic rank is very difficult, even if probably still possible. Or you could imagine that layout has to be dynamic (which dimensions are most major), and there are $N!$ (N factorial) possible layouts, which is super-exponential in the rank N , so that seems like a big problem. But that's an incorrect derived consequence. You can do dynamic array sizes without dynamic rank or layout.

Is structured sparsity attractive? It's easy to figure out if A:B structured sparsity works (see the section on structured sparsity for systolic arrays for details). Just take every B elements and set all but the A largest of them to zero. See what the impact is. The quality for this will be quite bad, so that settles it: structured sparsity doesn't work.

This is another larger class of co-design mistakes: there is never just "it works" or "it doesn't work". There is only "it does/doesn't work if you try it *this way*". Frequently people will try something, observe an unfavorable result, and simply declare that "it doesn't work." Out of politeness, perhaps no one questions this declaration, so now the company believes something that is simply untrue - perhaps catastrophically untrue. This also works in reverse: even if "it works" using some method, that won't matter if your customers can't or won't execute whatever method you used to make it work.

So when you point this out, you might get a response like “more could be done, but we don’t have time for that - it didn’t work in the simple way, so I think that we need to move on”. That’s a sensible response, especially for a small company with few resources. It might also just be a signal that the employee would prefer to do other work - maybe someone else can continue the task. Though you will at times work with people who prefer to go on the attack instead of the ethical path of just acknowledging limitations of whatever they did. E.g. you might get an angry response along the lines of: “I proved that it doesn’t work. What is your proof that it does?” This misrepresents both that they proved that it doesn’t work and also that you ought to have proof before you are allowed to ask questions or have a different opinion. In this case, you may need to get someone else to figure out what the real answer is - perhaps you’ll need to do it yourself. You should in this case also consider if the initial results were done correctly, since using aggression to cover up details in my experience is a strong indication of not having done careful work. People who work in a careful way tend to like explaining that they did so.

Omitting relevant factors Apart from taking or not taking into account derived consequences, there is also the practice, intentional or not, of omitting relevant factors. This one is a big one, too, and the impact of this problem is so omnipresent and corrupting that, essentially, you cannot trust numbers just in general. Numbers are never evidence *on their own*. You have to understand what each number means and numbers *rarely* mean precisely what it seems like they ought to mean. Especially not in co-design.

How can every AI chip startup have faster chips than Nvidia Blackwell? Well, how can every programming language be faster than C? Because that is a common claim for new languages, too. It is due to omitting relevant factors. Here are some examples:

- A relevant factor might be that there were 1000s of benchmarks one could report, and this particular one happened to give the desired result, so that’s the only one we *do* report. But not the other ones. Even this can happen innocently - you had that ONE benchmark in mind already when you designed the chip and its software, so that’s the only good one and that’s the only one you measured.
- Another relevant factor might be that the competitions’ software or chip had parallelism turned off, but our software or chip had parallelism turned on. Let’s not mention that, or, perhaps, we didn’t even know. This is much more likely to happen innocently than you might think - few companies are experts on how to configure the competition’s products.
- Maybe the competition sells their chips individually while we sell our chips in modules of 1000. So let’s compare 1:1 of “our product” versus “their product”, that seems fair. (I receive emails with claims where I cannot imagine any explanation other than this.)
- Maybe our per-chip performance is 2x the competition, but our cost is 10x. Let’s focus on the 2x per-chip performance.
- There is random variation in our measurement. Let’s measure our thing 1000 times and report the best one, but let’s only run the competition’s thing that one time. This easily happens innocently if you just vary something those 1000 times, and you think you’re finding out which configuration is best. Which you might *also* be doing.

- Our chip can sustain superior performance for a time, then throttles down to be much slower. Let's report that without mentioning the duration (I'm looking at you, Nvidia).
- Our chip offers great performance at low prices, but offers so little accuracy that it's not terribly useful. We made ONE model work for this, so that's what we talk about. That this won't work for most models - let's not mention that.

The common thread here is that the *numbers themselves* are correct. They are *not* fraudulent. There is no lie. Not exactly. We just omitted some additional information that some people may have found relevant. I say "we", but this is not actually something I've ever done. Not knowingly, at least.

This isn't just a kind of thing that happens on purpose. It often and I would say *usually* happens without the person bringing the numbers to you knowing it. Numbers are meaningless. I'll say it again: numbers are meaningless. Experiments are meaningless. Data is meaningless. Nothing means anything. It's all bullshit. *Unless* you understand the context and meaning. You have to prove the *meaning*, not just the numbers. The numbers never speak for themselves - never, ever. Few people have wrong numbers, but perhaps *most* numbers are received with wrong meaning. Yet most presentations of information are done as if you don't need to prove or discuss the meaning of the numbers, rendering large parts of many presentations uninformative in a way that the presenter and much of the audience may not have considered. When people take one number that they don't understand and observe that it is bigger than another number that they also don't understand, this is when you can really draw some unfounded conclusions. But it *appears* sensible - 5 is bigger than 4, what's hard to understand about that?

You will rarely be able to distinguish lying numbers, from wrong numbers, from irrelevant numbers, from confused numbers from inaccurate numbers. The attempt is a kind of immaturity in dealing with numbers, I think. Numbers that you don't understand don't mean much and that's all it is. That's my conclusion, anyway. So if you hear your competitor claiming 10x or 100x Blackwell performance per chip. Calm down. Relax. It might be the end of your business, but probably not - it probably doesn't mean what it sounds like it means. Could still be trouble, but probably not like it's presented.

This is true in all areas of human endeavor, but I've been involved with co-design for a long time, and I can just say that this is *especially* true for co-design. Co-design really gives you many opportunities for doing experiments that yield numbers that just don't mean what you think they mean. The reason for this is that in co-design, you are talking about *changing* things in *new* ways, and that very often involves novel important derived consequences and novel relevant factors to take into account. If you want to investigate something in co-design, your first idea for how to do it is probably bad. It'll give you a number, sure, but the number is unlikely to mean what you think it does if you don't think about it very carefully. It might mean nothing at all. A thing that is often missed is to ask: "how would or could we change the compiler or other software in relation to this hardware question, and what effect would that have?"

What I'm saying here isn't novel or complex, but executing on this is hard. It's not what people do naturally, not even highly compensated people that can get in at places like Google.

Here's a particularly dangerous failure mode: There is a way of doing things where all decisions must be based on data and experiments, but there is no requirement for the intellectual quality of what's going on. Such a policy is a way to feel better about making mostly random decisions. The hard part is and always will be *good judgement*, not merely having numbers. Within the context of good judgement, *then* numbers can be very powerful, but the hard part there is providing that context, and that is usually not called out when people talk about being "data-driven" or "evidence-based".

At the end of this rant, this is where I should outline the better way of doing things. The trouble is, if I could define intellectual quality and good judgement precisely, in a way that is easily applied, then I suppose I would have also described the inner workings of a superintelligent AI right there. I don't know how to do that. But I know it when I see it and so do you. It's about really thinking things through and getting it right. Numbers *are* necessary as a component of that, but they are *not* the intellectual shortcut that they are very often made out to be.

As determined by an undisclosed process You may notice that this document doesn't have many numbers in it. There are some, but not many. It's not an accident. Surely, you will believe that I would have been able to put numbers to everything here. Some kind of numbers found somehow. I decided not to try to confuse you in this way. Many relevant numbers are outside my time, resource and access constraints to obtain properly ("hey vendor, what do you charge Nvidia for HBM?"), so numbers I would give you wouldn't be the ones you need. They would be "some" numbers. "Some" numbers are not a benefit. So I didn't do that and thereby I improved the quality of this document by 7 units of quality as determined by an undisclosed process. That's a number. See what I mean? *Lots* of decisions are made everywhere based on numbers determined by an undisclosed process. It's standard.

When you see a number, see if the suffix "as determined by an undisclosed process" is true. If it is, then you have something to potentially ask about now. Though I can tell you that you will often not get a satisfactory answer if you do ask, even if the number is actually OK - you will often get an explanation along the lines of "we did some work and then this is the number". It also tends to bother people if you ask too much about their numbers. It is often better to probe their mental model of whatever they are talking about and actually just ignore their numbers. This is often better received, as well, since questions about numbers can be received as questions about the potential for fraud or incompetence, or as an attempt to embarrass, since the idea that numbers *in general* are something to doubt *the meaning of* is not widespread, so why else would you be asking? You think I'm too dumb to copy-paste correctly onto my slide?

One reason some people don't want to explain their numbers is that they perceive that providing these numbers is part of their value at the company. If it becomes clear to anyone else how it's done, then they perceive that their value at the company will be reduced, and there is also the potential that some kind of error will be found and they perceive that this will reduce their

reputation. If further questions are raised as a result of anyone knowing what they are doing, then this may also cause more work for them. So they don't want you or anyone else to know and they perceive your questions as a potential threat to their livelihood. They've chosen an unethical and company-harming strategy. If you are a leader with positional authority, I would not suggest to tolerate such information-hiding behavior from any employee under any circumstances, but if you don't own the company, maybe you'll have to deal with such behavior. If you lead, certainly do notice and reward it when people are being open. Even if you don't lead, this is something that you can point out, that a colleague is doing something positive. Keep in mind that people who aren't good at explaining things, or who simply don't know the answers but don't want to admit that, or who just are a bit aggressive by personality (or all three!), may *look* like they are trying to hide information, but it isn't necessarily on purpose. Sometimes it is on purpose. It *usually* isn't. Do stay calm, kind and respectful in all cases.

Also learn from the world outside your company

People outside your company have good ideas, too. So you need to study your competition and you need to read academic papers. Or *someone* in your company does. In a short-staffed startup, this might be a problem as it takes a lot of effort to do. It can also subtly get you to think that ideas are things that come from the outside, instead of putting a lot of effort into figuring out novel ideas on your own. You need to do both, of course.

A big problem with learning from the outside world is that people outside are often themselves confused somehow and also motivated to market instead of informing you. So something everyone is saying is going to revolutionize everything might actually have no real use, while something that no one is talking about might be critically important. This is because the public wants to be excited about the news, while you want to do something difficult and practical. It's not the same thing at all. This leads to the tiresome need to replicate and understand things in detail yourself, instead of thinking that what people say is what is true. Which is hard work, and you can't do it for everything that might be relevant because there is too much to do. Hiring can also be difficult for such things - Google hires 100s of AI researchers, but practically none of them had any interest in hardware design as it relates to AI when I worked there.

I can't tell you what the right balance is for your situation, but I can say that Nvidia places a lot of effort on watching the outside world, while Google does this not as much, instead preferring to do their own thing. Both companies do both, of course, but the emphasis is different.

An example of this is that when I showed this doc to Google contacts, the response was "that's interesting". When I showed this doc to Nvidia contacts, the response was "how would you quantify the effect of X, Y and Z?"

You can also see these different approaches in systolic array innovation: Right out of the gate, Google had an idea with BF16 that caught Nvidia unawares. A Google engineer (not me) had a good idea there on his own. What happened next is that Nvidia started adding all kinds of novel numerics to their systolic arrays, showing Nvidia's ability to adapt to an unfortunate situation and

master it. Now Google is behind. Nvidia was more powerful in following up on this than Google was. It doesn't matter now that Google was ahead on this initially.

Co-design needs to be grounded in the product

Co-design is important, so let's hire someone to do it. Or maybe several people. That can work out well if you hire the right people, but I think that this is a particularly difficult situation to place a person or team in. Co-design needs to be informed by the practical realities of your company's work and a co-design team won't have that context initially. So they need to develop that context. I'd suggest that there may not be a better way to develop that context than to concretely do your company's work, be that hardware work or compiler / low-level library development or even negotiating prices with vendors. A *critical* component of co-design is the prices of things, both your own sale price and the prices of components and there can easily be no one in the co-design room who is engaged with this, which is very unfortunate.

So this speaks in favor of having the co-design team be made up of people who also have other responsibilities at your company, so that they can use the experience from those responsibilities to inform their co-design work. Yet now you have a problem of limited capacity that is devoted to co-design, because these people also have their other job to do. You also might have a problem with co-design meetings becoming a way for your entire existing team to just chat with each other without much of a concrete result (I've seen this many times), because co-design then isn't their primary responsibility. Of course don't let that happen - you need a more focused, smaller group with an actual responsibility to deliver, otherwise you might think you're doing co-design but you aren't. The alternative is to hire full-time co-design employees who will then have to figure things out by interviewing everyone. That can definitely work, too, but I think it's a difficult approach that requires some pretty strong people who are good at staying grounded and it requires the rest of your organization to be welcoming in sharing their expertise - hopefully, your organization is like that. I can't tell you how to navigate this optimally, but I do have a tale on this from Google.

[[[Tale: Wait, who is this?]]] Google hired a team to do something somehow TPU co-design related in the TPUV2+ era, but this had limited connection to the TPU v2+ project. In fact, initially, no one in the software team knew that this team existed as far as I'm aware. I was myself surprised to hear of this team's existence after having already attended the real TPU co-design meetings for months. Evidently, there was a whole separate series of weekly meetings, with completely different people. The TPU hardware lead did attend these meetings and his mentioning it was how I found out about it. He subtly suggested that attending may not be a good use of my time. It would have been better if I had listened to him.

The person who was hired to run this team had received the Turing award, the equivalent of the Nobel prize for computer science. It was a series of meetings discussing somehow the idea of AI hardware. They were doing their own thing. When I joined the meeting series as the primary software contact for TPU v2+ co-design, there was never a single question for me about that.

In the actual TPU co-design meetings, we had some things that we were unsure about, and that it would be wonderful for someone to investigate, so I floated some of these topics in this other parallel meeting series. This was completely unwelcome. That's not what they were doing.

So it wasn't terribly clear to me what the purpose of this was and I stopped attending. However, if you look at journalistic coverage about TPUs, you'll get quite a different impression. This is how Gemini summarises it:

"Patterson joined Google in 2016 as a Distinguished Engineer and became the public face and primary academic authority for the TPU's design philosophy."

The result was to associate Patterson with TPUs in the public mind, which generated a Turing award halo for the Google brand as a whole. So I'm not sure if this is a cautionary tale or just a tale.]]]

Combinatorial search

If you didn't know already, something you might have started to notice as you read all this is that AI chip design involves making a large number of decisions and these decisions all connect to each other. There are so many connections between different decisions that you cannot possibly understand the full implications just by thinking about it. So what to do about that?

The standard approach I've seen here is something like this: study the matter carefully, come up with a proposal, discuss the proposal, elect the proposal as the Plan Of Record (POR) and then evaluate changes based on their impact for the POR. A change is then adopted if it improves the POR or otherwise it is rejected. Sounds reasonable enough, but is it? In optimization, this approach is known as [hill climbing](#). That's a Wikipedia link and if you go read the Wikipedia page, you'll see right away one of the problems with hill climbing:

"Hill climbing finds optimal solutions for convex problems – for other problems it will find only local optima (solutions that cannot be improved upon by any neighboring configurations), which are not necessarily the best possible solution (the global optimum) out of all possible solutions (the search space)."

This is not a particularly novel insight here, that hill climbing gets stuck in local optima. Yet nevertheless, the standard co-design process that I've seen is a process of hill climbing and it does indeed get stuck in local optima and no one does anything about it.

In another section, I mentioned how a bloated over-expensive design can be hard to design yourself out of, because if you consider a more economical approach one expense at a time, in isolation, then each specific expense is probably only going to be a small percent of the overall expense, so it isn't justifiable to pursue it. But all together, the savings add up, and this does more than you might think. It's not linear. The savings simply add, yes, but suppose you have two ways of saving \$0.5 on a \$1.01 cost, which combine to a \$1 savings. Individually, each

technique is a 2x on cost effectiveness (\$1.01 becomes \$0.51), which may not be worthwhile. But, *together*, the techniques deliver 100x on cost effectiveness (\$1.01 becomes \$0.01). What's going on here is that it isn't about how much you saved on one item, but how many items you can get for a dollar, and that *isn't* linear in cost, since you divide by the cost and $1/x$ is not a linear function. So you cannot consider even *unrelated* decisions in isolation because of this effect. Each savings makes all other savings more powerful and each bloating makes all other bloatings seem less serious. The net result is that bloat protects bloat when you hill climb your design. You can't decide to unbloat your design because your co-design hill climb is stuck in a local optimum. This sounds dumb when you say it like this, surely reasonable people would recognize the issue, it's just some simple arithmetic, but this is actually *not at all* obvious when you sit in the process concretely over time.

Suppose you reject three independent ideas for unbloating your design over a period of three years. Yet if the three ideas were presented together at one time, you would have approved the combination, because the impacts combine superlinearly. Do you really think that this is obvious to have a three year memory like this, connecting these things? I can tell you that for many people, such as myself, this is not obvious.

And then, of course, there are also *related* decisions that *do* interact beyond this factor. There is *a lot* of that. So I think co-design needs to be informed based on a combinatorial search across many different sets of decisions so that you can consider plans that are different from your current plan in multiple aspects at the same time. Combinatorial search is exponential, so you cannot do this yourself in your head, you need a fast computer and maybe even some clever optimization software. This isn't normally done as far as I've seen. In fact I've never seen it. But I think it's a good idea. This optimization needs to take software decisions into account, as well, as these are critical influences on what your chip needs to deliver.

If you try this, I can tell you what will quickly happen: the model you try to optimize becomes so complex that you stop trusting it. This is just inevitable if you pursue this in earnest. Your model will not be accurate and, even if it is, you won't know that for sure. However, it can give you ideas for approaches that you wouldn't have thought of yourself. If your model outputs something ridiculous, use that as a way to understand what is wrong with your model, fix it and rerun. The model can also help you do sensitivity analysis, where you understand how sensitive your decisions are to your particular assumptions.

You might not want to let a combinatorial search simply make your decisions for you. You may still use a traditional hill-climbing co-design process. In fact, surely that will happen. But it seems unlikely that this exercise will not inform and improve your judgement. The trade-off is that this approach is even more difficult than traditional co-design: you need to reason correctly about not just your current exact design, but the overall space of all (or at least many) designs. Teams frequently have trouble understanding the implications of just their own concrete current design, so this is no small thing. You also need to take uncertainty into account, which is not so easy to do well. This way requires more intellectual firepower and a lot more time and effort. But I hardly think that you can engage in this exercise and not have valuable insights if you did it well.

Another problem is that, to do this well, everyone involved in co-design needs to know a lot of sensitive information. Like the prices you expect to get from your vendors. How will you optimize tokens per dollar without knowing what the dollars are? So you can accept that and get a more-well-designed product or roll the dice with some other approach.

A critical look at GPU features for AI

Nvidia GPUs contain some expensive features that are not needed for AI. If you remove these features, what remains is roughly a CPU with a systolic array and high memory bandwidth. So the conclusion of this section is that Nvidia GPUs are successful in AI because Nvidia is most excellent at building GPUs and writing software for them, *not* because CUDA, the programming model that Nvidia GPUs implement, is best suited for AI.

Nvidia GPUs are programmed in a programming language called CUDA. CUDA is a modified version of the programming language C++ and programming an Nvidia GPU using CUDA is similar to programming a CPU in C++. CUDA exposes some hardware features that Nvidia GPUs support that are not traditionally found on a CPU. This section critically evaluates some of these features.

Kernels and warp specialization

GPU programs are separated into “kernels”, which are separate sub-programs that run (usually) one after the other. This leads to concerns of optimizing the running of these sub-programs to avoid overheads like kernel launching. Kernels are the way they are because that is how ancient Nvidia GPUs ran shaders for graphics.

This is in contrast to what happens on a CPU, where programs are just one big binary that runs for as long as the program runs. The CPU model is superior. Let me tell you why.

If your ML workload does one thing at a time, then kernels don’t get so much in your way. As long as you make sure to enqueue the next kernel in a CUDA graph before it has to run, and you don’t have too many kernels, then it’s OK. Kernels are no advantage here, they are something bothersome to deal with, but it’s not that much of a bother. It’s OK.

The problems start when you want to do more than one thing at a time. Suppose you want to overlap compute (like systolic array usage) with network operation, which you definitely DO want to do for a sophisticated inference or training deployment. All-reduce operations aren’t just fire-and-forget network transfers - you have to do a little bit of computation every once in a while to keep the operation going. To handle this on a GPU, you could have a separate all-reduce kernel running at the same time as your compute kernel, but you cannot multiplex two kernels onto a single SM, so one entire SM would be dedicated to using the network if you do it this way, and this wastes the other resources on that SM, like the systolic arrays. So that’s not good

if you want high utilization of the device. Instead, you can create a single kernel that does both compute and network simultaneously. So you'll have to implement the multiplexing yourself in software inside your kernel. This is already *quite* bothersome compared to what you do on a CPU - just have two separate CPU threads doing their own thing separately.

But it gets worse. What if your all-reduce takes more time than the next compute kernel? We don't want to have the compute kernel wait for the all-reduce to complete. One option is to manually implement the all-reduce into multiple different compute kernels, so the next compute kernel picks up the all-reduce where the previous compute kernel left off. That's possible, but definitely not a nice thing to have to implement. This goes beyond just bothersome - this is a fundamentally silly situation. GPU kernels are silly. It's a graphics legacy from a simpler time of just running shaders.

There is a way to fix this in software on GPUs. It's called a megakernel. The idea is that your entire program is just one big kernel. Now you can dynamically treat warps pretty much as multiplexed CPU threads and you can run a network operation on a warp that will go to sleep (and not waste compute) when you don't need it to do anything for a while until a network transfer completes. This is precisely how it is on a CPU, except CPU threads are much easier to work with than warps for this. But network warps don't require so many resources compared to a compute warp, so to make this all possible at high performance, Nvidia had to add a hardware feature called "warp specialization" to enable warps to request different amounts of resources.

All CPU binary (and TPU binaries) are already "Megakernels" in this way. Now we use that on GPUs, too. We made CPUs the way we did because that is best for sophisticated use cases, so as usage of GPUs becomes more sophisticated, GPUs will have to converge towards looking more and more like CPUs. Megakernels is another example of that.

So what we've seen here is that warp specialization is essentially an awkward way of making GPUs work more like CPUs. Which is an improvement compared to not having that, but it's not an improvement on how an actual CPU with an OS works.

[[[Kernels on TPUs How about TPUs? TPUs already run normal large programs just like a CPU does. So that's good! However, we still have the same problem on TPUs anyway, but for a different reason. The problem on GPUs was that we want to multiplex and GPUs do support multiplexing for warps, but we can't multiplex warps between different kernels. So the silly kernels are getting in the way and the solution is to get rid of them in favor of a megakernel, but CUDA wasn't really made with megakernels in mind. On a TPU, the problem is that there is no multiplexing feature on the hardware. So you end up having to multiplex things in software. XLA is in the business of heroic software optimization, so, amazingly, it can sometimes do this automatically on TPUs. The result is a silly situation for both TPUs and GPUs.

What happened in ancient times is that hardware and software engineers over many decades figured out how to solve all these problems. The result is modern OSes and CPUs. Google and Nvidia are now slowly rediscovering these solutions - poorly. I propose that we discard all this

silliness and just put regular CPU cores and OSes on the accelerators and program them accordingly. That's where it'll end up anyway, we might as well skip right to the end state.]]]

Path divergence (vector subsets)

Nvidia GPUs have a feature called “path divergence”, where the GPU can operate on a subset of a vector. This is also possible on a CPU, but it carries a higher overhead as managing the subsets is then done in software instead of being a hardware feature. For AI, this usually is used when a tensor (i.e. an array) inside the model contains a dimension which is not a multiple of the vector size. It is a good idea for AI practitioners to choose dimension sizes that *are* a multiple of the vector size, which is not hard to do, but not all AI practitioners do this, leading to this feature being used on a GPU. This is a feature that is convenient but not necessary nor even a good idea to put on an AI accelerator given the cost to do this in hardware. Much lighter weight vector instructions can achieve the same effect for AI workloads. Also, there really should be some kind of AI police telling AI practitioners to be more careful in choosing the dimensions of their tensors (i.e. arrays). A size of 257 can be *half* as fast as 256. You also risk triggering rarely-used cases in the underlying infrastructure that may not be as well tested and therefore more likely to contain bugs. “Powers of 2 are right, otherwise your code's a blight” would be a good phrase to memorize.

Software managed cache

A “cache” is a kind of especially fast on-chip memory, also called “SRAM”. A cache can be *automatic*, where the computer tries to guess what is going to be needed in the cache later and retrieves it automatically ahead of time. This can be very convenient for the programmer, but sometimes the guess is wrong, leading to slowdowns. A cache can also be *manual*, which requires the programmer to go to the extra trouble to tell the computer exactly what data should be placed into the cache ahead of time. This is a lot of trouble for the programmer, but can be more reliable. Automatic caches are far more convenient than manual ones, but for high performance computing, the programmer has to be aware of how the cache is being used either way to reach top performance, and it is in this case actually convenient to be able to control the cache explicitly, instead of trying to guess what the automatic cache will do. So for high performance computing, it's the other way around - manual caches are more convenient. CPUs normally have automatic caches, while Nvidia GPUs have both an automatic cache and a manual one. The manual cache on a GPU is called “shared memory”. For high performance AI workloads, this actually makes GPUs easier to program than if it wasn't there, as complete manual control of the cache is preferred in this context. For everyday programming of less critical workloads, manual caches are a nuisance and this is likely why Nvidia GPUs also offer an automatic cache. Both the automatic and manual caches are good features on Nvidia GPUs that any AI accelerator should also have, in my opinion. CPUs commonly have a way to give hints to the automatic cache, but do not commonly have a way to take complete manual control of the cache the way you can on an Nvidia GPU. An AI CPU ought to do what Nvidia does here: have both a manual and an automatic cache. Ideally the whole cache is automatic by default, and you can request a portion of it to be converted to manual as needed.

Warp scheduling (memory access pipelining)

To program an Nvidia GPU, one must divide a workload into many smaller tasks called *threads/warps* (the full picture here is very complicated, and there are multiple levels of organization that one must use beyond warps - programming in CUDA at high performance is complex). You can think of warps as small programs that perform vector operations. When a warp accesses memory at a given address, that address may not be available in the cache. In this case, the warp has to wait idle while the memory loads the address, which can take many cycles. To avoid hardware resources sitting idle, an Nvidia GPU will quickly start running a different warp, one that is not waiting on memory currently, and then return to the idle warp once the data has been loaded from memory. Nvidia calls this *warp scheduling*. In this way, while individual warps may frequently be idle, the hardware is, ideally, never idle. This is an expensive feature and one might say that this single feature is the largest part of what makes a GPU a GPU instead of a CPU.

In graphics processing, a shader often has to access memory in unpredictable ways, and warp scheduling allows Nvidia GPUs to run such shaders at high efficiency even with unpredictable memory accesses. When implementing an AI workload on an Nvidia GPU, you will find yourself using many warps and thus relying on warp scheduling, so then it would be reasonable to suppose that warp scheduling is part of the Nvidia GPU magic that makes AI workloads run so quickly on Nvidia hardware. Not true. Warp scheduling is used on Nvidia GPUs because this is required to use all of the memory bandwidth on an Nvidia GPU, but not because it is needed for AI. Warp scheduling is simply an artifact of the CUDA programming model that uses warps. It is an artifact of the graphics heritage of Nvidia GPUs (the G in GPU).

What replaces warp scheduling on CPUs and other accelerators is software pipelining (ChatGPT complains here that CPUs also use out-of-order execution for this - true, but I am here talking about a higher level of granularity than that). Software pipelining achieves full utilization of hardware resources for AI, just like warp scheduling does, but faster and cheaper. Software pipelining is complex in software if you do it explicitly, but if you do it with a proper library to hide that complexity, it's actually quite easy to do (I get into this in another chapter). Software pipelining can also be done automatically by e.g. a C++ compiler, but this is not what I'm proposing or talking about here (that's what Intel Itanium did and that huge bet wasn't terribly successful).

The hardware structure that enables software pipelining to function for memory is a simple hardware queue for memory accesses, be those fetch (with a wait), pre-fetch (ahead of time) or explicit DMAs. All CPUs have this. On TPUs, all memory accesses are actually DMAs and they do go into a hardware queue that enables software pipelining to function.

[[[Irrelevant story Also, you may wonder how I can be telling you details of TPUs that sound like they might be non-public, specifically that TPUs function exclusively on DMA memory accesses. Well, let me tell you a story. Once upon a time, when I worked on TPUs at Google, I gave a conference talk describing how we program TPUs at Google, in hopes of raising

Google's profile in AI hardware and helping with hiring (Google or Nvidia are both great places to work). You can see [the slides here](#). I was very careful to not mention that TPUs exclusively access memory using DMAs (not counting the so-called SparseCore on TPUs, which functions on completely different principles), which was a bit awkward not to explain, but it just seemed like an internal detail the audience didn't need to know about. The policy was that you could talk about anything that could anyway be determined by our customers on their own using benchmarks, but nothing else. I was then interrupted by a VP from Google, sitting in the audience, announcing loudly to all 100+ external people from our particular industry that what I was avoiding talking about was "DMAs! It's DMAs!". That's a direct quote. So I am well and safely secure in saying that this is completely public information after that announcement.]]]

If *you* are building an AI accelerator, I would not recommend only allowing DMAs, like on TPUs, because my recommendation is to build a traditional CPU with a systolic array, and traditional CPUs do have memory access that isn't a DMA, so you should, too. I do recommend *supporting* DMAs, though, between all available memory spaces and over the network. In particular, it is important to support DMAs between the manual caches (i.e. SRAM) on separate cores or chips, by-passing HBM entirely - you will consume too much HBM memory bandwidth (and associated power use) if you require DMAs to first write to HBM and then bring it from there into the software managed cache. This feature will allow your networking fundamentals to use little or even no HBM bandwidth, which is important if you pipelined it so that network transfers occur concurrently to other computations. It will also reduce the latency and general overhead of your networking primitives.

Needing a host computer

An Nvidia GPU requires to be connected to a *host* computer. GPUs are separate devices from the host, and it is the host that is orchestrating the kernels that the GPU runs. The host also processes the AI's input and output. This is also true for TPUs, and, in both cases, it adds non-trivial cost to the system to have to buy a host computer to operate a GPU or TPU as an AI device. If the GPU or TPU were its own computer, the cost for the host would be unnecessary. One might argue that this simply moves cost from one place to another one, since the device then still needs the capacity to operate as a computer and it is that capacity that has a cost regardless of whether you put it inside or outside of the AI device. That is only in part true, because the device duplicates features of the host, the device can be more specific to its own needs than an off-the-shelf CPU can be and there is a cost to having two components and connecting them together. You do not need to network two devices if they are the same device.

The graphics legacy of GPUs might be part of the reason for the host-device split. GPUs were like that originally due to details of the graphics consumer market. Then when GPUs were repurposed for AI years back, this split did not change, even in places like the datacenter where it might make sense to change it.

Apart from the economic cost of purchasing and operating a host computer (including an increased failure domain), this host-device split also adds unnecessary overhead and data

transfers, as all data and commands have to go through a separate node that didn't need to be there. E.g. this can increase kernel switching overhead.

Nvidia has more recently started selling CPUs that are fairly tightly integrated with their GPUs. This reduces some of the overheads of the host-device split, but this does not go so far as removing that split. These systems also have a very high cost, so there are no savings for the customer from this. Still, this *is* a step in the direction of what I'm suggesting here.

The need for many threads and kernel switching overhead

Switching the kernel involves stopping doing one thing and starting doing another thing. The CUDA programming model involves some additional overhead here, related to CUDA's focus on many threads:

- 1) Each thread needs to initially do some computation to figure out which thread it is and what it should be doing. Each thread frequently does very little work, as that better fills the GPU, leading to a problem where figuring out what to do at the start *can* be a big part of the computation, since every thread has to do it anew. Careful optimization reduces the impact of this, but it is still an unfortunate property of CUDA.
- 2) At the start of a kernel, *all* threads (in some cases warps) will be simultaneously and independently figuring out who they are and what to do. This means that the systolic arrays on the GPU lie idle during this period of duplicated startup calculations.
- 3) When a GPU kernel stops, it has to wait for the slowest, most unlucky thread to be done with its work. This may be a thread that encountered many cache misses and which was also started late during the kernel run, perhaps even after most other threads were already done. So the whole GPU can be waiting idle while 1 thread completes its work.
- 4) The hardware must have a fast way to keep track of the very many independent threads/warps. This adds additional hardware cost (chip area and power).

I am here ignoring the organization of threads into warps, blocks and grids. That can make the issue worse, as it can introduce additional separate waits on unlucky threads at the end of blocks, even if the kernel overall is not about to be done. It can also help, since the GPU might start running a block of the next kernel if most blocks of the prior kernel are done.

Nvidia offers much advice on how to combat kernel switching overhead and, with careful optimization, it is possible to avoid having kernel switching overhead be a serious degradation on GPU utilization. However, this is extra effort to solve a problem that can be traced to CUDA, so I think that it is fair to say that having to optimize around these issues is a negative of CUDA.

CPUs and TPUs are better on these issues because they do not require so many independent threads and TPUs are also quite deterministic, so there is less space for a core to be unlucky than there is for a CUDA thread to be unlucky, and there are 1000s of CUDA threads but only 2 cores on a TPU chip. You could view the 8x128 vector width/registers of a TPU as containing 1024 threads, which is often a useful way to think of it when comparing a TPU to a GPU, but

those threads do not cause the issues mentioned here as they are not independent the way GPU threads/warps are and, again, TPUs are more deterministic on top of that.

[[[Comparison to kernel switching overhead on TPUs for nerds Zero cycle kernel switching overhead is possible on a TPU, assuming the next kernel was preloaded. It often takes a bit longer due to the fact that most kernels on a TPU are software pipelined and software pipelines have epilogue/prologue overhead to get started/stopped - see the section on software pipelining. A sufficiently clever software solution could overlap the epilogue (stopping) of the prior kernel with the prologue (starting) of the next kernel, but this is complex, so while I implemented the software pipelining for TPUs, I never pursued this optimization while I was at Google. It could be done, though, which would lead to the possibility of long-run precisely 100% utilization of the systolic arrays on a TPU, even while changing kernels. The similarity or even equivalence of thread / warp scheduling and software pipelining is not obvious at first sight, but here we see it in action yet again: both involve a period of lower utilization at the start and end of a kernel. Though this period can be eliminated on a TPU (even if I never did that) and the chaos of threads on a GPU makes the issue much worse, since the device must wait idle for the most unlucky thread. Software pipelining on TPUs does not have as serious of an issue with a most unlucky thread, since it replaces the chaos of many threads running concurrently per device with a single deterministic thread of execution (or a few). These differences explain why the optimization of fusing completely unrelated kernels into one kernel, to reduce kernel switching overhead, is more important on a GPU than on a TPU.]]]

[[[Details on startup comparison for nerds Above it says that it is a consequence of the many threads in CUDA that it takes a while for all the threads to start using the systolic array at the start of the kernel. But in the previous nerdbox, we saw that this is the *same* on a TPU (that doesn't use many threads), since it also incurs a software pipelining prologue (startup) overhead. So it seems that CUDA and its many threads are being blamed for this unfairly. Well, there are two components to this. One is the part of the GPU kernel startup overhead that corresponds to a software pipelined loop prologue, e.g. you have to wait for the first bits of data from the memory to come in at the start of the kernel. We saw in the nerdbox above that this overhead *can* be eliminated with software pipelines by overlapping the epilogue/prologue of separate pipelines. With CUDA, this isn't an option - you can overlap GPU kernels, but that doesn't fully solve it. So this is an unfortunate aspect of CUDA, though CUDA doesn't get the full blame here, as software pipelines are usually not optimized in this heroic way anyway, so *then* it ends up being the same. However, there is also a separate component of this overhead where CUDA does get the whole blame - every CUDA thread has to figure out what it is doing independently. This commonly leads to startup involving actually heavy computations for CUDA kernels, extending the startup period beyond what would be caused by a software pipelining prologue. In contrast, on a CPU-like device like a TPU, the "what are we doing?" calculation is very light-weight or even absent (you always start at index 0). It is usually possible to minimize the impact of this second issue on a GPU with careful optimization, but the issue is still an unfortunate consequence of the CUDA programming model.]]]

The next section is all about software pipelining and the contrast of it to warp scheduling.

Software pipelining for AI kernels

In this chapter, I'll explain what software pipelining is and how to make it easy. Software pipelining is necessary for AI software and it is the most difficult thing in AI software - but it is only so difficult if you don't do it the way I'll suggest doing it in this chapter.

I will also explain how a CUDA feature called warp scheduling is actually a hidden form of pipelining but done in hardware. Warp scheduling is an expensive feature to implement in hardware, once all derived costs are taken into account, incurring a hardware cost for all Nvidia GPUs. I will argue that software pipelining is preferable for AI. Also because you still need to software pipeline even on GPUs at times.

What is (software) pipelining

Suppose you need to do three operations A, B and C in a loop like this:

```
for i = 1, 2, 3, 4 do
    a_i = A(x_i)
    b_i = B(a_i)
    c_i = C(b_i)
```

Here A is some mathematical function that the chip implements. Here x_i is the input and c_i is the result of iteration i of the loop. Suppose the operations A, B and C use separate components of the chip it runs on and, for convenience, let's say that operation A() runs on a hardware unit also called A and similarly for B and C. Then if this loop is executed in the order written, unit B and C will be idle while A executes, units A and C will be idle while B executes and units A and B will be idle while C executes. That is no good, since then the average utilization of the chip is at most 33%. We want 100%.

To see how this can be improved, consider this timeline of the operations that are done:

1. A(x_1)
2. B(a_1)
3. C(b_1)
4. A(x_2)
5. B(a_2)
6. C(b_2)
7. A(x_3)
8. B(a_3)
9. C(b_3)
10. A(x_4)
11. B(a_4)
12. C(b_4)

We need to do something smarter than this. The first idea might be to start executing $B(a_1)$ at the same time as we start executing $A(x_1)$. Then both units A and B are busy at the same time. That won't work, since $B(a_1)$ depends on the result of $A(x_1)$ that we have named a_1 . So we cannot start $B(a_1)$ until $A(x_1)$ is completed. We need a better idea: software pipelining.

Software pipelining resolves this issue and leads to high utilization for a loop such as this. The idea is to take the above order of operations and change it to this instead:

1. $A(x_1)$
2. $A(x_2)$ and $B(a_1)$
3. $A(x_3)$ and $B(a_2)$ and $C(b_1)$
4. $A(x_4)$ and $B(a_3)$ and $C(b_2)$
5. $B(a_4)$ and $C(b_3)$
6. $C(b_4)$

The first step is identical to before, but in the second step, both A and B are busy and in the third step all the units are busy. The key is to realize that $A(x_2)$ does not have to wait until the loop is done with $B(a_1)$ or $C(b_1)$ because $A(x_2)$ doesn't depend on them. We can see that this method is faster since there are only 6 steps in the list above, while before we had 12 steps. This is software pipelining of a 3-stage loop (A, B and C are the three stages).

The first two steps are called the *prologue*. During the prologue, not all the units on the chip are able to do computations yet because there is not yet an input available for them to operate on. The prologue achieves roughly 50% utilization on average across the steps.

The middle two steps are called the *steady state*. During the steady state, all the units of the chip are busy simultaneously, leading to 100% utilization assuming that the steps take the same amount of time to complete.

The last two steps are called the *epilogue*. During the epilogue, the earlier stages of the pipeline are already done, so they have to wait idle while the latter stages of the pipeline complete the last pieces of work. This happens because the earlier stages of the software pipelined loop are running ahead of the later stages. The epilogue also runs at roughly 50% utilization.

Utilization drops from the numbers given above (50%, 100% and 50%) if the stages do not take the same amount of time. This is because the software pipelined loop cannot proceed until all stages have completed, so the faster stages will wait idle from the time that they are done until the slowest stage completes.

The epilogue and prologue each take $S - 1$ steps for an S stage pipeline, assuming the loop has that many iterations. So a pipeline with more stages has a longer prologue and epilogue, leading to lower utilization. So we prefer to have fewer stages when possible.

If the loop has N iterations, then the steady state will run for $N - 2(S-1)$ steps. The steady state is where we can potentially achieve 100% utilization, so it is good for N , the number of iterations, to be high if you want high utilization.

Why do we say software pipelining and not just pipelining? That is because the exact same solution also works in hardware, but to achieve the same effect, the implementation in hardware, i.e. hardware pipelining, is different, so it is customary to specify which kind of pipelining is meant. For example, the solution in hardware doesn't focus on loops the way it does in software.

Software pipelining is difficult by hand

I mentioned in the introduction that software pipelining is difficult to do. It is more accurate to say that it is difficult to do *by hand* as a programmer. In this section, I will give an idea of why that is.

Recall that this is the original loop that we were looking at, with now N iterations for an unknown N :

```
for i = 1, 2, 3, N do
    a_i = A(x_i)
    b_i = B(a_i)
    c_i = C(b_i)
```

Here is a *start* on software pipelining the loop above:

```
for i = 1, ..., N + 2 do
    if i <= N then
        a_i = A(x_i)
    if i >= 2 and i <= N + 1 then
        b_i-1 = B(a_i-1)
    if i >= 3 then
        c_i-2 = C(b_i-2)
```

If you execute this loop in your mind, or on a piece of paper, you will see that the order of operations is the same as in the pipelined order of operations shown in the previous section. However, we are still doing the operations one at a time, so we are not increasing the utilization above 33% here, we are just reordering things. Depending on what the operations A , B and C actually are, a modern CPU might be able to overlap some of these calculations anyway, but it might not. To actually software pipeline this loop assuming that A , B and C are operations that can run concurrently in an explicit way (like a DMA), we need to start unit A and then start unit B before unit A has completed its work, resulting in a loop like this:

```
for i = 1, ..., N + 2 do
    if i <= N then
```

```

        start executing  $a_i = A(x_i)$  and proceed immediately
    if  $i \geq 2$  and  $i \leq N + 1$  then
        Wait for  $A(x_{i-1})$  to be done and retrieve its result  $a_{i-1}$ 
        Start executing  $b_{i-1} = B(a_{i-1})$  and proceed immediately
    if  $i \geq 3$  then
        Wait for  $B(x_{i-2})$  to be done and retrieve its result  $b_{i-2}$ 
        Start executing  $c_{i-2} = C(b_{i-2})$  and proceed immediately
Wait for all C's to be done.

```

This is getting to be a bit more complicated now. Here we are telling each unit to start doing a computation and then the loop proceeds immediately before that operation is done. This is necessary to have more than one unit running at a time.

An operation is *live* if it is currently executing. The loop above has the property that it has up to two A's live at the same time and all the C's could potentially be live at the same time. For example, consider that on the first iteration, we make one A live and then on the second iteration, we make a second A live *before* we wait for the previous A to be done. So there are up to two A's live. Similarly for B and C. This can be a benefit, as then the next input to a unit A is prepared ahead of time, so that when unit A is ready for its next input, it will be available immediately without the loop having to get to unit A right at that time. However, this requires resources, be they hardware or software resources (memory), to keep track of two A's at the same time. This could be very expensive, for example if keeping track of an A takes 16 GiB of memory, now we need 32 GiB of memory for A where we didn't need that much before. So this is not always preferred. There is a way to avoid this, that I won't show here, but it gets even more complicated, with yet more if statements.

There is another problem with this loop. The trouble is that if statements are bad for performance in general. If statements can block compiler optimization and can also take time and resources for a chip to execute. So for these reasons, the traditional and most used way to software pipeline a loop such as this is to unroll the epilogue and prologue in a way that makes it possible to remove the if statements. It ends up looking like this:

```

If  $N \geq 1$  then
     $a_1 = A(x_1)$ 
    if  $N \geq 2$  then
         $a_2 = A(x_2)$ 
     $b_1 = B(a_1)$ 
    for  $i = 3, \dots, N$  do
         $a_i = A(x_i)$ 
         $b_{i-1} = B(a_{i-1})$ 
         $c_{i-2} = C(b_{i-2})$ 
    If  $N \geq 2$ 
         $b_N = B(a_N)$ 
    if  $N \geq 2$  then

```

$$c_{N-1} = C(b_{N-1})$$

$$c_N = C(b_N)$$

As before, depending on the nature of the chip one runs on and the nature of the operations A, B and C, we might need to create a more complicated version of this with “start executing” and “wait for” statements.

We didn’t eliminate the if statements in the prologue and epilogue. If statements can also lower utilization there. This is especially the case if we often run a loop with just a few iterations, since then the epilogue and prologue take up a larger fraction of the pipelined loop, potentially all of it. With further complication, you can also do something about that.

I’ve worked with software pipelining for years and I made *several* errors in the code above while writing this section, that I discovered on later readings. Hopefully it’s correct now. This is tricky to get right, though it’s perfectly doable to do.

The **real trouble** comes when I tell you that when people software pipeline loops manually, they don’t usually do it in a way that is so abstractly simplified as this. In what I showed you here, A, B and C are single-line things and their parameters and results are single things. That keeps this simplified. If you look at real software pipelined code, it tends not to be written like this (though it could be). It’s usually way more code (e.g. many screens of code) and it is far more complicated due to the particularities of the A, B and C that have not been abstracted away. Also, this is just 3 stages, software pipelines can have more stages than this. There are also issues of handling of memory and other resources through the pipeline. If you read the Wikipedia page on software pipelining, you’ll find yet other concerns I haven’t even gotten into here. Real-world hand-written software pipelined code tends to be far more complex than what I’ve shown here.

But it gets even worse from there. All data processing in AI is done in *tiles*. If you have a large array (what AI researchers call “tensors” is called “arrays” in computer science), then you cannot keep the whole array in cache (“fast memory”). So you must subdivide it into smaller pieces that should ideally be of a fixed size. The size of these pieces should then align with the vector width of your hardware. The size also should not be too large, because the size of your cache is limited and it has to fit in there. But larger sizes tend to be faster, so they should not be too small either, and certainly not smaller than the vector width of the chip. And also, ideally, you want the tile size dimensions to be divisors of the input array dimensions, since otherwise you have wasted area at the boundaries (leads to lower utilization). But then what if the input dimension size is a large prime number, so it has no reasonably-sized divisors? For such reasons, choosing tile sizes is a difficult business with many competing concerns. We’ve already seen that software pipelining can have consequences for memory usage (and usage of other resources, such as registers or capacity in hardware queues), so it turns out that how you software pipeline is connected to what tile sizes you can use. You may need (probably will need) several different implementations of many ops with different tile sizes and if you go this road you’ll end up software pipelining each of them, potentially each in different ways. So you won’t

have to software pipeline one loop for an op, you'll sometimes have to do it *many* times, and each of those times may choose a different software pipelining approach. All this is an incredible amount of work and complexity for a software team to manage.

If you go software pipeline some real code, then you'll probably see what I mean. It is possible to do but it is not easy to do. Not by hand like this. It is also difficult to read, understand and modify code that was written in this way. So what to do instead? That's the topic of the next section.

Write a tiled software pipelining library with op fusion

Since software pipelining by hand is hard, I suggest not doing it that way.

The traditional way to software pipeline is to let the compiler do it. You can certainly try that. Recall that if you have a custom AI chip, the people writing the compiler is also your selfsame software team, so this may not be that much easier, but it's certainly one way to go. However, if the stages in your pipeline (loop) are complex, this is a lot to demand of the compiler to figure out how to reorder it and to prove that such a reordering is correct. So if you go this way, you may prefer a hybrid solution where some things are handled by the compiler and some things are handled by the code that you write. In other words, your code is then written to help the compiler to figure things out, e.g. maybe it is partially reordered already as will need to happen for software pipelining, and then the compiler handles the rest. This is a traditional approach and is used in high-performance libraries like Eigen. One of the drawbacks is that a compiler may or may not apply a given optimization like software pipelining. So you might want to find some way to guarantee that the compiler will do it. That may not be possible to guarantee.

The traditional compiler-based approach still involves writing the loops and code by hand. I suggest creating a library solution instead. I'll assume that it is possible to program your AI accelerator in C++. If it isn't, you probably made a mistake. Google TPUs cannot be programmed in C++ and I think that this is and was a mistake, that I'm partly responsible for, since the hardware can easily run C++ code, there just isn't a compiler to make it happen and, back when I worked on TPUs at Google, I could have just taken a few weeks to make it happen (it isn't hard if you know compilers). I didn't. That was a mistake. Nvidia GPUs can be programmed in CUDA, which is similar to C++, and this is widely regarded as one of the reasons for Nvidia's success. So go do that, but don't create a different language like CUDA, just plain old C++ with some special functions (intrinsics) and data types should be enough.

By using C++, I do not mean relying on the compiler to auto-vectorize the code. Instead, offer types and functions / intrinsics that represent your vectors, systolic arrays etc. and that can be used explicitly. Just like CUDA has warps, you will have vector types / variables.

So now you are writing C++. I suggest creating a C++ templated library that follows the rather abstract and mathematical description of software pipelining in the previous section. So it's a library where you declare the stages A, B and C and you also have to declare the inputs and

outputs of each stage so that the library knows what they are and how much memory they require. Since *everything* in AI programming ends up involving fixed-size tiles, you are going to be happier if you create this library so that inputs are described as tile subsets of a larger array, of an explicitly declared fixed tile size, and you tell the library how to calculate the coordinates of the input and output tiles for each iteration of the overall loop. Now you can write code that looks like a regular loop without pipelining (you can e.g. use C++ lambdas for this that you pass to the library) and then you pass the stages to your library and out comes a pipelined implementation. You'll have to write this library for yourself because no such C++ library exists currently that I'm aware of. You will also want to make sure that your compiler, likely an LLVM backend that you'll write, is good at handling the kind of code that your library generates. If implementing this library gets too complicated, you could specialize this to 3 stage pipelines where the first stage is reading arrays and the third stage is writing outputs.

The most important use of pipelining in AI programming is to prefetch memory. First you prefetch/load a tile of data, then you process the data, then you write the data to the output and you repeat this many times in a loop. You want to overlap the reading and writing of memory with the processing of data and this is exactly a 3-stage pipeline that you need to write every time you do anything on an AI chip. The key is that you do the prefetch/DMA before you need the data and that is what pipelining achieves. If you set up your library so that it understands tiling and coordinates of arrays, then this kind of code becomes dramatically easier to write. You also want to consider that sometimes one operand of an op moves with every iteration of the loop, while another only moves occasionally, e.g. matmul is like this. So your library must be aware of this, so that it does not reload data for the unmoving operand at every iteration. Sometimes it is the output operand that doesn't move as much as the input, e.g. if you have a reduction operation. In this case, you want to end up with an output buffer that is preserved between iterations of the loop and is then written out only when the output moves (i.e. when that output tile is done / finalized).

One way to make it easier for a compiler to software pipeline a loop is to completely unroll the loop in the steady state so that it becomes straight line code, which is possible for loops going through a tile since tiles are fixed size. This can lead to too large of a code size, leading to yet another concern to manage that influences the choice of a tile size, since in that case large tiles leads to large code size also.

Tile sizes become so difficult to choose that I would seriously suggest that perhaps we should as a field require all arrays (what people call tensors) to have dimensions that are all powers of 2. This makes everything easier. This is something you might seriously consider to require especially in the case where you can control the models that your hardware will be running - you can then yourself craft the models so that the array sizes are powers of 2, or at least have many factors of 2. As far as I'm aware, there is never an important reason to have array dimensions that are not powers of 2 (except for input RGB images), it's just something AI practitioners do because "why not". The TPU team at Google didn't control the models that we had to run, so we had to support arbitrary array dimensions, but perhaps you are in a situation where you can avoid that. This is a great simplification. For many ops, fully half of the code for the op was a

huge heuristic for choosing the right tile size and the other half was the actual op implementation. You can avoid most of that if you do not support arbitrary dimension sizes.

Do you want to write such a library as what I'm suggesting here? Well, you decide. First, try not using software pipelining at all. If that somehow works well on your hardware, then I guess you're set (though then your hardware is weird and I think probably not good, but maybe I'm wrong). If not: Second, try software pipelining by hand a few times. Are you happy and productive while doing this? If so, you're set (I don't believe you if you say yes to this). If not: Third, consider writing a tiled software pipelining library like what I'm suggesting. If you can do a good job with that, I think you'll be happier.

If you set things up cleverly, you can probably integrate your XLA backend with this C++ library so that custom ops can be fused with built-in ops and each other. Now you'll be pushing the state of the art in programming AI accelerators. This is where we need to end up. That's what I believe, anyway.

In case you haven't realized this by now, I did in fact write such a library for Google TPUs and everything that is done on Google TPUs is done using this library (it's only similar because it's not exactly C++ and you can't use it externally and so you can't use it to write fusable custom ops (or custom ops at all)). At least that was the case when I left Google around the launch of TPUv4 and my understanding is that it's still true. I can talk about this openly and in some detail because I made an officially Google approved public conference presentation at [C4ML 2019](#) about how we programmed TPUs at Google. You may like to take a look at the slides for this presentation which you can find [here](#). I referred to tiles as stencils in that talk, since the library in question is mildly similar to [a stencil library](#). Those slides even include a (somewhat modified, simplified) example of the concrete API of this library as used to program Google TPUs internally at Google. You may like to take a look at that.

It appears that since then the Jax team has exposed a similar interface for TPUs in Python/Jax called Pallas. You can read about it [here](#). I haven't tried using it, but Pallas looks to me like an externalization of parts of the internal XLA:TPU backend structures. For your AI chip, you probably don't want to tie yourself that closely to Jax (or maybe you do? What do your customers say?), but it's still something that it may be interesting to look at if you start doing a library like this for your hardware, no matter if it is in Python/Jax, C++ (like I'm suggesting) or something else.

That slideset also explains how XLA does op fusion, which is an extension of the same library. If you have to do many operations on a given tile, there are often good reasons to load it into memory once and then create one pipeline to handle all the operations on that tile, instead of writing the result out to memory only to read it back in later. This is called *op fusion*. XLA does this automatically and on TPUs the way it's done is using the same tile-based pipelining library that I've been describing here. It turns out, if you declare your ops as pipelines with explicitly declared inputs and outputs, that are always tiles, then you can automatically compose (fuse) ops and that is how op fusion works in XLA:TPU.

So you can use this same kind of library to do tiling, software pipelining and fusion and these features fit well together in one library that you can write in C++. You can also extend this to parallel algorithms, where the communication queues between the parallel cores have to be software pipelined. So a communication queue becomes a declared operand the same way an array input or output would be, but with two end-points instead of one.

Using this library led to much increased productivity for the Google TPU team that I worked on. This is how we were able to launch TPUv2 with an XLA team that consisted of 5 people. TPUv2 ran the models of its time with high performance. All programming of TPUs is done via XLA, so the 5 people really were it. We also did the TPU hardware-software codesign and the compiler for TPUs is also inside XLA and done by the same team. There were a few more software people involved with the interface from TensorFlow to XLA and a few more software people writing the TPU firmware, which were separate even smaller teams. It's been reported [elsewhere](#) that "Google had hundreds of engineers working on XLA (depending on how you count)". Now this is referring to a time significantly later than TPUv2 times, where indeed the team grew, but I have to admit that I don't know a way to count that makes it "hundreds". Not even a single hundred, actually. All the people working on XLA:TPU sat within 10 meters of my desk (and not that many people worked on XLA:GPU initially). Google has a very compact office environment but it isn't *that* compact that you could fit 100s of people so close to me. Perhaps if we hadn't used this library that's how many we would have needed.

To be clear, when I suggest to use XLA, this is of course with the understanding that XLA itself supports TensorFlow, PyTorch and Jax already. The suggestion is not to get your customers to discard TensorFlow, PyTorch or Jax in favor of XLA. If you build an accelerator along the lines I'm suggesting in this document, it will have some similarities with TPUs, and so what you are doing is repurposing a significant part of Google's investment in XLA for your project. This is all above-board since XLA is open source and this is specifically an intended use of XLA. You'll still have to write your own backend, as the XLA:TPU backend is not open source. That's much less work than what you'll have to do if you don't adopt XLA or something like it. Interfacing a compiler with Tensorflow and, even more so, PyTorch, is quite hard. I know because I've seen *failed years-long projects* attempting this across the industry and this was certainly challenging also for the *strong dedicated multi-person team* working on this at Google for years. Part of what XLA does for you is to reduce a huge number of ML ops down to a *much* smaller number of HLO ops.

Business-wise, you might have a concern that, if you use XLA, then your customers can more easily switch to using TPUs (or other XLA-using accelerators), since TPUs use XLA. On the other hand, TPU customers can then more easily switch to your hardware, as can Nvidia GPU customers that also use XLA. If your offering does not offer benefits above what TPUs do, I'm not sure that you're in a good place in the market regardless. You'll be able to tell your customers that they are not locked in to your hardware software-wise, which is likely going to ease new-customer sales. The business trade-off there is that they actually won't be locked in.

XLA makes your hardware more fungible (not entirely fungible) with other hardware, for better and for worse. To compete in a market of a fungible product, you must be ahead.

Something to notice is that the industry has standardized on using transformers for everything. That may not last, but it's been going on for a while now. So there should be a space in the market for AI chips whose software at first primarily supports transformer structures and ops. You'll have to include things like MoE and advanced parallelization techniques under the heading of "transformers". Your software team *should* be able to go beyond that over time, but you need to start somewhere and transformers are the obvious place to start as things stand now. This is a contrast to how it used to be, where everyone was using a fully bespoke model structure. This factor helps a new AI chip to enter the market. Though I would advise to plan to have full generality, along the lines I'm suggesting in this section. Even "just" supporting transformers is no small thing, so you'll want good and general infrastructure even "just" for that.

[[[Automating kernel writing with AI for nerds This idea is speculative, but the truth is that AI can write a lot of code these days automatically. An idea is to take an off-the-shelf already-capable AI coder model, retrain / fine-tune it on a lot of examples of how to write code for your hardware (preferably after you've made this already as simple as possible by offering good APIs and libraries) and then make the resulting AI coder part of your offering to your customers. Much kernel code looks quite similar, so AI might have a good chance at being good at this. The suggestion in this section of this document is to offer these three primary interfaces: XLA, C++, LLVM backend (you need the LLVM backend anyway to support C++, so you might as well offer the LLVM interface to customers directly if they want it, though that would be a very sophisticated customer). Such an AI would offer a fourth option: Describe your op / kernel and have the AI write that op automatically as either a combination of other XLA ops or, failing that, an optimized C++ kernel written automatically, whichever way the AI comes up with that works. Anything can be written in C++ and it is very often possible to express an op using XLA primitives, even in many cases where it isn't immediately obvious how to do that (oh, and btw, the more tricky cases like this often involves use of the *iota* XLA op, so ensure that your hardware is capable of being programmed to generate an *iota* without a massive slowdown), so the AI has some options here. AI isn't today reliable at coding *correctly*, so you might require or heavily recommend the user to supply a significant test suite to root out any bugs, but ideally the AI would also generate a formal proof to prove equivalence to a simple math description that the user supplies - that you can trust, since formal proofs are automatically checkable. Is current AI strong enough to do formal proofs for kernel correctness? AI is already [somewhat capable at writing formal proofs](#), so it probably can do this, at least some of the time, if you give it many examples of how to prove kernels correct for your hardware / libraries / API specifically. I'm not sure how hard it is to get an AI to serve in this role currently, but if you can make it work, I think you can get some press coverage for your company/product (always nice) and it could, I believe, be quite useful if you do a good enough job of it. Another approach here is to use correct-by-construction directives like in Halide or TVM, and have the AI write the directives, which is not a reliability problem since the directives cannot make the code incorrect. Though now that AIs can write C++ code and also do formal proofs, that more flexible AI approach is perhaps now becoming viable.]]]

Why not X, Y or Z?

You have many, *many* options for how to let people use and program your AI chip. Here's a list, which is not *at all* exhaustive:

- 1) Why not make an integration with OpenAI's Triton for your hardware? Triton already supports software pipelining as a first-class concern, so that sounds promising.
- 2) Why not extend Jax-Pallas for your hardware? You might be able to take the TPU API for Jax-Pallas and do something similar for your device.
- 3) Why not get your hardware in there as a Mojo target? Mojo is a language extension of Python specifically intended to allow writing kernels while abstracting away the hardware as much as possible.
- 4) You could make your own extension of Python, if you don't like Mojo.
- 5) Why not support TVM, instead of or in addition to XLA? Some people use TVM and TVM has a Halide-like approach. Halide was one of the options for auto-writing correct kernels in the nerd box in the previous section, and TVM already has some support for that - so maybe that's interesting.
- 6) If you don't like XLA and TVM, why not make your own compiler that supports TensorFlow, PyTorch and Jax? Facebook did that with Glow (for TF and PT).
- 7) You might also support Glow!
- 8) Why not port Halide to your hardware? Halide allows correct-by-construction kernel transformations, just like TVM does. So if you don't like TVM, this is another option.
- 9) Why not make your own language that makes your hardware sing? It worked for Nvidia - CUDA is widely considered very beneficial to Nvidia's business.
- 10) Nvidia's CUDA Tile library is open source and not necessarily so specific to CUDA as you might think. Maybe you could appropriate the code or at least the ideas and make something like that work for your hardware?
- 11) Why not offer a long series of libraries for common operations, just as Nvidia does with Cutlass, cuDNN, cuBLAS etc. No reason to restrict support to XLA.
- 12) TileLang sounds kind of interesting, maybe you can adapt that for your hardware?
- 13) I know, everybody else is wrong, you can do much better, this list needs to be much longer! Come up with your own new idea!

The truth is, XLA is the most battle tested AI compiler out there and it is open source, so you can add your own backend. XLA is generally good enough on its own, but as proven by the existence of Jax:Pallas:TPU, at times your customers may need to implement their own kernels beyond and above what already comes with XLA. So you should have some way of doing that. It doesn't get more straightforward to offer a C++ interface here and that will let your customers do *everything* that you can do, so there will never be a question of whether your AI chip is generally programmable. This is also what Nvidia did with CUDA, except they made their own language and I'm suggesting to just offer the venerable C++ as-is with some intrinsics and built-in types. I think my plan is the minimum to do. You can do more than that, there are some options in the list above, which is by all means not exhaustive. I don't think you should do less than what I suggest. Supporting XLA and C++ are, I believe, a good recommendation for the

minimum thing to do for new AI hardware. That and an LLVM backend, but supporting C++ (via Clang) will need that anyway, so I don't mention it separately. This of course assumes that your chip is somewhat CPU-like, so that it is feasible to program from C++, just like TPUs are. TPUs could be programmed in C++ at high performance, Google just hasn't offered that as an option. I don't know why *you* wouldn't, if your chip allows it.

[[[**Explanation of why MLIR isn't on the list above for nerds** Why not put MLIR on the list? That's because MLIR is an infrastructure for defining an Intermediate Representation (IR). An intermediate representation is what a compiler uses to represent a program. If you need to define an IR, then using MLIR to define it, instead of writing custom code to do that yourself, is possibly a good idea. You may even be able to reuse some fragments of other IRs that were already defined using MLIR. It also allows you to use some surrounding infrastructure that people have written for IRs defined using MLIR, e.g. for managing passes and matching an IR pattern and transforming it into a different IR pattern. So far so good. However, I've talked to *many* professional software developers in the field of AI infrastructure who seem to be a bit confused about MLIR. They think of MLIR as somehow a replacement for everything, but if you ask them about details of that, they don't really know what to say further. I've talked to several highly intelligent people who had a bit of an "aha, OK, yeah I see, I guess I was confused" moment when I simply asked them to describe in more detail what they mean by "using MLIR". Given how much focus MLIR gets from people who do use it, it is reasonable to suppose that MLIR is actually a quite nice way to define an IR. How much focus does MLIR get from people who use it? A lot! And this is part of what confuses people, because it almost makes it seem as if MLIR was responsible for the whole project, as if MLIR was some kind of developer or person or programming deity. E.g. I have never heard a dev on Nvidia CUDA Tile talk about Tile without immediately mentioning MLIR and that is common for devs that use MLIR to define their IRs. MLIR is not an entry in the list above because it doesn't do what the things on that list do. It does something different: it lets you define an IR. I explain this in some detail here because I've seen highly intelligent people be quite confused on this.]]]

Software pipelining on Nvidia GPUs aka Warp Scheduling

High performance computing in CUDA *can* involve explicit software pipelining of the kind described in the previous section, but sometimes it doesn't. There are CUDA programmers who will never have heard of software pipelining and yet they are capable of writing high performance AI kernels. So how can that be true?

That's because Nvidia GPUs have a feature called *warp scheduling* that is a hardware feature that makes it possible to intermingle the execution of many separate small programs. This intermingling is equivalent to software pipelining, but done in a different way. Before, there was a loop like this:

```
for i = 1, 2, 3, N do
  a_i = A(x_i)
  b_i = B(a_i)
```

$$c_i = C(b_i)$$

But in CUDA, you instead write a function like this:

$f(i)$ is defined as

$$a_i = A(x_i)$$

$$b_i = B(a_i)$$

$$c_i = C(b_i)$$

The Nvidia GPU will then execute your function as $f(0)$, $f(1)$, $f(2)$ and so on. Each execution of the function is called a *thread*, these are collected into *warps* and many warps can execute at the same time. This is the same as a loop, but many iterations run at the same time. So what will happen is that some of the warps, at the same time, will be executing the first line using unit A, some will be executing the second line using unit B and some will be executing the third line, using unit C. If a unit is busy when a warp tries to use it, or if there is any kind of delay (e.g. from accessing memory), the GPU will stop running that warp and start running some other warp, so the hardware is (ideally) never idle. This has an equivalent result to software pipelining, but it requires a hardware feature.

So if a programmer has been using warp scheduling to achieve pipelining, then that programmer may never have had to deal with software pipelining directly. This is both an advantage and a disadvantage of Nvidia GPUs. If you try to software pipeline a loop by hand, then the CUDA approach is *far* superior in terms of complexity and, hence, in terms of programmer productivity. However, if you use a proper software pipelining library, like described in the previous section, then I would argue that it is actually easier to write a high performance kernel that way than it is to do it in CUDA using warp scheduling. The trouble for Nvidia is that warp scheduling, once you include all of the derived costs of having warps, is actually a very expensive hardware feature. If I am right, which I think I am, that high-utilization software pipelining using a good library in C++ is as easy or even actually easier than dealing with warp scheduling in CUDA, then this is a very expensive but unnecessary hardware feature that other AI accelerators can profitably leave out but Nvidia cannot since they are wed to the CUDA programming model. I predict that this will eventually be a competitive problem for Nvidia.

If you have written CUDA code, you may have encountered annoyances such as choosing grid sizes that lead to a kernel being unable to compile for using too many registers or there is not enough shared memory etc. A tiled software pipelining library in C++ might sound enticing because then perhaps you will not have such issues. Unfortunately, those issues will be precisely the same in C++ because what is being achieved with warp scheduling is the same, too. In CUDA you have to choose a grid size. In C++, you'll have to choose a tile size. You can indeed choose a tile size in C++ that is too big and you'll have trouble if you do it. If you set your compiler up to tell you when this happens, this is the exact same thing CUDA is doing when it tells you that you ran out of registers or shared memory. It's just the same thing. It's a problem to do with tiling, pipelining and the finite size of hardware resources, not specifically to do with CUDA.

Hiring for your AI chip project

You are likely going to need a larger and more capable software team than you think if you want to compete with Google and Nvidia for general deep learning AI. The software for this is quite challenging to write even for excellent engineers. The kind of software engineers that you need to hire are hard to find, expensive to hire and may have no good reason to work for your company. It's a challenging hiring market. Yet your success depends on it.

Sourcing candidates over email

Certainly use your personal network, but for myself, everyone who has hired me has done so from an initial contact over email. So I can talk about that.

You can of course start with LinkedIn and such services. You'll be contacting a lot of people that already have jobs and aren't looking for a new job that way, though. This may be more useful than it at first appears. I no longer even respond to job emails, I get too many, but this activity is still not completely useless to the people sending those emails, even if it might appear that way at first. The way I go about looking for a new job is to select some companies I'd like to work for and then I search my old email for attempts to contact me about a job from those companies. Then I respond to those emails, even if they are more than a year old. If that fails, I would see if I know someone who already works there, but that hasn't been necessary for me so far. So those emails you don't get responses to aren't necessarily wasted - you might get a response much, much later.

Something you might try is to make a list of old International Olympiad of Informatics (IOI) finalists, who are now adults, and see if you want to hire them. There are more than a hundred finalists each year (4 from each participating country). Many of these are likely to be interesting to consider for hiring, even if they aren't currently working in tech. There are other competitions whose rosters you might look at, too, like the International Mathematical Olympiad (IMO). I've only *ever* received *one* recruiting email that mentioned that I was a finalist in IOI and IMO, so, strangely, I suspect that this list is not much used by sourcers/recruiters currently. I think you might get some good hires from that. It's something I would have tried if I had started a company myself. (Was this a subtle way to get to mention that I was finalist in IOI and IMO? Well, it wasn't supposed to be, but I don't know how to offer the evidence that these lists are not much used currently by recruiters without mentioning it, so I did.)

Something to avoid is to have the sourcer write the blurb that describes your company, though this seems to be how most companies do it (a sourcer is someone who makes the initial contact with candidates that you may want to hire). If you are contacting someone about a job in an AI chip company, you are likely not the only AI chip company doing so. The sourcer has simply no idea what your company is about other than what you tell them, so whatever they come up with is probably not that convincing. This is *especially* bad if you are a startup, because then the only reason to work at your company is if you have a chance, with the candidate's help, to succeed in the market. So the sourcing email has to make it clear why your company would have a

chance to succeed above other AI chip companies. The sourcer can't do this very well. So you need to take an active role. This may seem obvious, but I receive a lot of job emails and it very clearly isn't obvious to most companies. What I receive is often some dry list of job responsibilities, maybe a boring press release and *nothing interesting* about the company.

When you do take an active role with sourcing, especially in how to describe your company, here's something that usually happens that's not great: you wrote something general about your company for the press and you send that same material to me. As if hiring is not important to your company. Don't do that. You wrote the press material for journalists that don't know much about the job. So it's all flat, simple and without much meaning to someone already in the field. Almost all job emails I receive read as if they were written for the press and it's just pointless. That's not what I would send to prospective candidates. As a candidate, there are some things that I do want to hear about, and most job emails I receive fail to tell me about *any* of them:

Do you have a desire that your company should be a good place for employees to work at? If you have such a desire, it might be smart to have a way to bring that out in your hiring email in some convincing way. Of course the candidate is not going to take your word for it, but it's still good to try to make the case. It's more fun to work at a place where you yourself and also all the other hires feel that it's a good place to work. So if that's what you are building, or even just trying to build, why not mention that. Don't make it sound like a cult, though - that's another problem.

Are you even looking for a senior hire? Most emails I receive aren't. Of all the companies sending me job emails, Amazon did the best job here. They have an idea of "Principal Engineers" being something special, and they have a separate hiring channel and separate sourcers for this hiring channel, which they make clear when they contact you. It's genuinely a special job at the company that they hire Principal Engineers for, and that is clear from the first email. I understand that this is in part just appealing to the candidate's vanity, but I don't see how that's a bad thing. I'd recommend finding subtle (or maybe overt) ways to appeal to a candidate's vanity, as long as you can do it skillfully without too obviously pandering, regardless of whether it's a senior hire or not. Imagine if you bring in a candidate for an interview and it becomes clear that the candidate doesn't know your company's name. That's the other end of the spectrum of appealing to someone's vanity and you can probably see that it's not great when it is turned on *you* like that. Subtly and/or cleverly appealing to a candidate's vanity is a good idea, I think.

Do you understand the area of AI hardware? Understanding the area of AI hardware is hard. Starting a company (of whatever kind) and then just being bad at it isn't hard. So you can see that there is going to be more of one than the other. It is quite difficult for a sourcer to on their own convincingly convey the idea that you and your company understand AI hardware, but it is quite possible to intuit that a company probably doesn't understand AI hardware in some of these emails I've received, even though they are mediated through a sourcer. One kind of unfortunate move is to claim numbers that simply can't be true. It turns out that ALL startup chips are WAY faster than Nvidia chips. I don't believe you, unless you can tell me how you are

so *much* better at making systolic arrays than Nvidia. There are some issues with Blackwell, but it's a *huge* chip and Nvidia is good at making chips, so stop telling me you are 10x+ faster than Blackwell with your current chip, unless you have some very good explanation for that that I'm pretty sure you don't have. In my career I've seen first hand that there are 1000s of ways to make invalid comparisons and then report only the result without mentioning the (bad) method (I arrived at the number "1000s" by desiring a high a number and then reporting it - see how easy that is?). I've received numbers where the only way to physically do it is if they are comparing a system of 1000s of their chips to a single Blackwell, for example. Such numbers tank your credibility. I have enough experience to know not to believe numbers just in general, not even if they look reasonable (the numbers are often correct numbers, but they very rarely mean what they are presented as meaning). But there is a more subtle problem here. The correct metric for your success isn't even being faster than Nvidia chips in the first place. What you have to deliver is foremost *tokens per dollar*, not so much tokens per second per chip. Token latency also matters, but probably not as much. Do you know how many job emails that I've received that say something about tokens per dollar? Zero. It has never happened *once*, and I receive a lot of these things. That's just amazing to me. These emails should be written very carefully, not quickly by a sourcer who doesn't understand your business like you do. If you send the candidate an email that suggests you don't understand the business, why shouldn't the candidate assume or at least suspect that... you don't understand the business?

This is an example of how material for the press looks like (which surely Sundar Pichai did not write himself): <https://x.com/sundarpichai/status/1986463934543765973> If you dig into this tweet carefully, you'll in the end realize that it contains precisely *nothing* relevant if you were trying to evaluate this chip (performance *per chip* is irrelevant, "peak performance" even more so, and internal use involves hands-on help from the dev team that you won't have as a customer). That might be good marketing, but if your hiring email looks like this, that's not good.

Do you have a plan for success? So if a senior person receives a hiring email from you, they need to know either that you have a plan for success with your company or that you understand that you do not have that yet and you need the senior person to help you with this once they are hired. One or the other. What senior person in their right mind wants to join a startup with no credible plan for success and no understanding of needing to find one? Look at your hiring email(s) and consider if this is something that's clear from it. Is it clear? I can tell you right now that you probably don't need to look because the answer is no, I receive enough of these emails to know that it's always no. How are you going to succeed? Why are you credible? Above, I gave you an idea of how you might start to convince someone like me on that (not that I'm looking for a job): Mention something about tokens per dollar. No one ever did before in any of these emails I've received, and it's the most important metric, so if you're the first to even mention it, now I think you're a bit credible already. Though I've somewhat ruined that one now by mentioning it here.

But even tokens per dollar is a bit dumb, because what's the quality of those tokens? Under which circumstances can it be delivered? And so on. For such reasons, numbers in general just aren't that useful as a way to show that you have a real plan for success. You need an

understanding of what you are building and a reason to think that what you build will be better somehow, and your hiring email should ideally reveal that you are someone who has thought about this skilfully. Your hiring email / material needs to be a work of art, because that's how you get the team that will bring you success. So you're going to get some sourcer who doesn't understand your business to write it? No! Also, it shouldn't be too long.

What if you said something about why your systolic arrays are better than other people's systolic arrays? I've never received a hiring email saying *anything* about that. I mean, your systolic arrays or the stuff around them is going to have to be better, right, otherwise you don't have a useful product. So there should be a reason. What is it? Maybe you don't want to tell me in case I join a competitor, OK, but can you make it sound plausible that you have something special, even without giving away all the details? Maybe, maybe not, but in the emails I receive, there isn't even the attempt. It's just "our chip is fast, come work here", pretty much. Often it's just "come work here" and that's it. In fact, in 1/3 of cases (this is not an exaggeration), it's "come work *somewhere*", they won't even say where because it's a general sourcer company.

Should you be revealing your proprietary wonderfulness to the job candidates? Don't send them your entire plan, but if you don't have enough good ideas that you are OK to give a few examples of how you intend to make your company's product excellent, then that's not great.

Doing hiring emails well sounds difficult. Can't we just hire a sourcer who is good at that and let them handle it? Well, OK, but you're competing with companies like Google who can hire people simply on the weight of their brand for being a good place to work. You are competing with Google both for hiring the best people and for producing the best AI chip. You are going to have to bring your A game everywhere, also in hiring. I can tell you right now that most chip startups are not bringing their A-game to hiring as far as I can tell from the hiring emails that I receive.

It's OK to be disappointed, but I would suggest not to become angry if a candidate that you have taken significant trouble to source and interview turns down your offer. It's just a very bad idea. A "no" today can become a "yes" in the future. Other people you want to hire may ask the person who turned you down what they think of your hiring process or your company. *The relationship did not end when the candidate said "no"*, for better or for worse. Some managers, even at fancy high-paying companies, do not know this, even though presumably they've turned down many candidates themselves. So if you run a company, I suggest that this is something to be aware of and to communicate to your managers.

Interviewing engineers

I've interviewed more than 50 candidates for hire at Google, and I've interviewed and gotten offers myself from top AI hardware companies like Google, Facebook, Nvidia, Amazon, OpenAI. So I know the process of how they interview for software engineers. They all do it roughly the same way and I doubt you can come up with a better process than they are using. The candidate gets 5 (normal) to 10 (unusual) interviews of 45 minutes with different interviewers, many of whom the candidate will not be working with (so they are independent and unfettered

by any manager's influence). For technical interviews, which are most of the interviews (except at Amazon, they have many behavioral interviews at the more senior levels), each interviewer gives you a computer science problem to solve in 45 minutes. You'd be surprised about how difficult some very well known computer science problems can be if you have to do it on a whiteboard in front of an interviewer. E.g. I interviewed several professional software engineers that couldn't get a binary search to work in 45 minutes on a whiteboard. I abandoned this question not because it proved to be too easy, but because it embarrassed people too much when they couldn't do it. It doesn't matter how difficult *you* think a problem *should* be, it only matters how difficult it *actually* turns out to be when you present it to candidates. You are not as good at predicting this as you think that you are. Therefore you must reuse the same problems many times so you can compare different candidates' performance. Otherwise you won't get a useful signal from your interview.

If you can't tell if a candidate's solution is correct or good, then you aren't competent to do the interview in the first place (unless the candidate is doing something very strange). That's OK, make sure some other people do the interviews, too. If there is a bug in the candidate's code, you should be able to tell them by giving them a concrete input where it doesn't work (or hinting in some other way, which is often better). If your problem leads to amounts of code so large that you can't keep track of it yourself, your problem is bad. So how do you hire your first engineer if you aren't yourself an engineer? I don't know. Please let me know if you do know. Maybe pay some people that you know and trust to help with the technical interviews who are competent to do them. What if you don't know and trust *any* engineers and you aren't yourself a competent engineer? Then I think that I advise against starting an AI chip company, but do feel free to let me know how it goes if you try. I'd be interested to know how you did it.

Don't use dynamic programming interview problems. I learned dynamic programming because I did competitive programming as a child, so they are fine for *me* as a candidate, but I don't think that they should be used. Why would your employees need to use dynamic programming? Do you know how many times I've used this in my career? *Zero* times. No matter how good you are as an engineer in general (which is what you want to hire for), if you haven't studied dynamic programming specifically, then you won't be good at it. Absolutely, some big company interviewers *will* use dynamic programming questions. I don't know why they do it (critiquing the question an interviewer asks me is not a situation I've ever gotten myself into, though I've interviewed people who do this - for the life of me, I can't imagine what they think they will get out of it). *You* shouldn't use dynamic programming questions. That's my advice.

If you have the idea that big serious companies in Silicon Valley use weird brainteasers as hiring questions, then you are wrong (unless you count dynamic programming as a weird brainteaser, which is borderline arguable). This is something the press has reported because it's *interesting*, not because it's *true*. It's not a thing to do. You definitely shouldn't do that. At Google, I shadowed an interviewer who would bring out the fun brainteasers only once he had determined that the candidate should definitely not be hired. That way it was more fun for him and the candidate (it can be saddening for candidates to fail at solving 5 problems in a row for 5*45 minutes). I thought it was disrespectful. The candidate is there to be interviewed for computer

science, so let him be interviewed for computer science. Don't give the candidate the idea that, should they return, they need to brush up on weird brainteaser questions. If you are not an engineer, but you like puzzles, and you think you can interview engineers using puzzles (this is the only way that I can imagine that this situation occurs at serious companies), then I don't have data to prove you wrong, but I wouldn't recommend this approach.

Talking to engineers

This section is relevant to your interests if you intend to run an AI chip startup but you aren't an engineer yourself.

To an MBA or a finance analyst, "proprietary" can be a positive word. It refers to the competitive advantage that a company derives from having something that other companies don't have. To an engineer, "proprietary" is a *horrible* word. To an engineer, "proprietary" means roughly "niche, low quality, difficult to work with, badly done, poorly supported and not useful for developing your skills because few use it and no one cares about it". This disconnect leads inexperienced MBAs to proudly explain how their workplace is full of "proprietary". As a customer, you should understand "proprietary" exactly as engineers understand it, because that is usually true. It's something that's bad but that you might have to deal with. Which an experienced MBA will know, so when you hear someone talk proudly about "proprietary", you know something about them that they don't want you to know. I'm sorry if this is offensive to you, whoever is reading this, but if it gets you to stop saying "proprietary", I did help you even if you are mad about it right now. Nvidia is not clueless, so you will not hear them talk about CUDA as "proprietary", even though clearly CUDA is a technology that has benefits for Nvidia and that other companies do not have. This is not an accident. Nvidia hires smart people who don't communicate that poorly. Take Nvidia's lead here. (Did I just imply that CUDA is low quality? No, it's only if the word "proprietary" is *used* that you can infer such things, it's not enough for it to *apply*.)

Be careful about using slogans when having a discussion. This is true for all people, but engineers are likely to have a particularly negative view of the practice of *sloganeering*. What is sloganeering? It is when you prepare a sentence to say when a topic comes up and you always use that sentence. The problem is that your prepared slogan is unlikely to specifically address the specific discussion that you are using it in and this is something engineering-minded people are especially likely to notice and dislike ("that wasn't even an argument!"). People, and higher level managers in particular, have many legitimate reasons to sloganeer. It's easy to end up saying the wrong thing sometimes and this can be a serious problem for a high-level manager that engages in many meetings and conversations every day. So they prepare slogans that they know are always OK to say. It can also just be a way to relieve the mental fatigue of meeting large groups of people everyday for months on end. You can sloganeer, and it has the intended benefits, but don't expect people to like it, especially not engineers. The especially unfortunate situation to avoid is if someone is trying to have a conversation with you that is necessary for the business, and you just refuse it by repeating a slogan.

Engineers that you employ ideally *should* care about the bottom line of your company. I can tell you right now that many engineers (and managers, for that matter) *do not* care about the bottom line of your company and you are going to have to accept that without being too angry about it if you want to succeed. Many people in business do not accept this and that's bad. Good engineers by definition *do* care about doing their jobs well - this is more important. Giving engineers equity in a large company does not change this at all, though in a tiny company it might (it might not). It's more of a personality trait, a moral feeling or the lack thereof, it's not primarily something to do with incentives. Finding competent engineers is difficult enough, requiring them to also be interested in the business of running companies is a requirement too far. When you communicate with engineers, or employees in general, you might subtly hit off-target because you assume that they are cheering on your bottom line as much as you are. This might be aggravating for a company owner, or a manager who does care about the bottom line, but I think it's just the way it is. "Great news everyone, our company was 3% more profitable than projections in Q3!". Don't send an email with that heading.

[[[My own personal plan What would I have done about this in my own company, if I had ever started one? I would target a moral good like "low cost and high quality AI services for all" and "serve humanity and make money, not necessarily at the same time". Then I would be inflexible in tolerating behavior that undermined those ends. This approach is easier if you don't have investors, so I would have tried to avoid that by relying on my personal ability to work (a significant asset), self-funding (I have significant savings and wouldn't need any early outside funding), early revenue and loans against revenue instead of taking investment, or, potentially, securing investment on unfavorable terms to the investor. My particular path on this would have been specific to my exact abilities, not necessarily a plan I would recommend in general: 1) bespoke technical and AI services for open communities like LLVM, Linux and Wikipedia (slim profits, low cost as no employees, but builds company capabilities and reputation), 2) a general service for such things for any community or company (higher profits, relying on previously built capabilities), 3) a service for others to create bespoke technical and/or AI services for anything (higher costs, will need employees, building on previous capabilities), 4) build and use own economical AI hw (don't have to find the customer if you are the customer, may require investment money here), 5) selling that hardware or access to it. In terms of aligning to a moral direction, many companies do this. Even Amazon - serving customers is a legitimate moral good. I wouldn't require employees to personally care about making money and serving humanity, but I *would* require external behavior that is in that direction and I would be highly intolerant of behavior specifically undermining those goals. I would be initially quite forgiving of people who simply admit whatever it is that they are doing that is off-brand and then accept correction. The idea of needing a perfect record for leadership is wrong. Secret immoral coalitions, on the other hand, those break companies and projects. I've seen it: "let's pick a superfluous huge technical fight so we can say that we are delayed due to the other side instead of admitting that we are delayed due to ourselves, never mind that this maneuver will make us even more delayed". I could provide technical supervision as required myself, so for as long as this would stay, say, less than 100 employees, I would not *have* to hire or retain star technical employees, just competent ones, though I'd still *try* to hire and retain star employees, of course. **]]]**

[[[Cautionary Tale: I'll take your keyboards!]]] I can tell you a story where Google's then CEO, Larry Page, failed to communicate well to engineers. If he had asked even a single engineer, or even *imagined* asking an engineer about his communication, this problem would have been avoided. This is not meant to be understood as a critique of Page's tenure as a CEO at Google, he seemed like a good CEO to me and I'm anyway not competent to evaluate his work, I'm just sharing a specific moment in time with you. Google holds weekly company-wide meetings called TGIF (Thank Google It's Friday), which were then [moved to Thursday](#) and backronymed to TGIF - Thursday Googler Information Forum. Back when I worked at Google, there was a time when Larry Page was clearly very upset that his employees were not taking mobile devices seriously enough and this was the topic of an entire TGIF one week. So far so good. Unfortunately, he then told everyone to consider how they could personally help the mobile effort (good), he suggested that people make an effort to work on mobile devices (Hmm) and then uttered the fateful words, with emphatic emphasis, much frustration, even a bit of anger: "I will *take* your keyboards". To understand how unfortunate this was, you need to know that almost all of the audience at TGIF are programmers. Highly paid valuable employees who program for a living. Programming is possible on a mobile device without a keyboard, but that is definitely not a sensible way to program. The threat to take people's keyboards didn't make much sense and was widely mocked at Google at the time. Yet perhaps you can tell what happened if you are a manager. Larry Page doesn't work with programmers, he (I assume) meets primarily with high-level other managers, who indeed *could* work without keyboards. And these leaders evidently weren't doing what he wanted, so he wanted to tell them that in an emphatic way. But he didn't consider that he wasn't talking to an audience of managers, but rather he was talking directly to 1000s of proud engineers. He was in that moment not in a mindset to consider how his words would sound to the engineers that ran his company at the bottom. And it caused him to embarrass himself that one time. Don't let that be you.]]]

From this anecdote, one can perhaps tell why some managers might prefer to sloganeer instead of ever saying anything on-the-spot. I still think it's a bad idea, but a manager who says the wrong thing can be remembered for it even 10+ years later, as you can see here, so it explains some of the motivation behind sloganeering. I'd suggest to just accept that you'll say the wrong thing every once in a while and deal with the consequences instead of sloganeering. Larry Page wasn't fired by Google's board for threatening to take people's keyboards. Nothing bad happened. The employees had something to laugh about, and it surely *did* put mobile more in peoples' minds while they were laughing. So maybe in the end it was even a good thing for Larry Page to say as a CEO even if it made him look foolish that one time.

[[[Cautionary Tale: I don't want to hear it]]] I once worked with a person who at first appeared to have significant technical talent, though he wasn't a software engineer. Until I observed him, just the one time, having a technical conversation with some engineers that were at a lower level of authority than him, and he suddenly angrily started insisting at length on something that, it was clear to me, didn't make any sense, even though the people he was talking to were making it pretty clear what the problem was with what he was saying. It was something you could only insist on if you didn't understand software in general. So it then became clear to me

what was going on. He had memorized things engineers that he trusted told him, without really understanding it in detail. So the things he would say would appear to come from a place of software engineering sophistication. Almost always. Except he didn't really understand and it would lead him, rarely, to insist on something that he wasn't right about without knowing it. Which it would be difficult to talk him out of because he didn't have the detailed knowledge necessary to reason it out. If you want to override the decisions of your employees, which you might at times have to do, you need to listen to them carefully when you do it, so you understand what the problems with what you are saying might be. So imagine a heated conversation where you don't want to be wrong, and now the highly paid doofus in front of you is telling you that he won't do what you are saying because "that's dumb". Tell him to shut up, right? No. You *do* need to get him to tell you why he thinks it's dumb, even if he's bad at explaining it, even if it *appears* clear to *you* that he isn't making sense and even if you are tired of listening to him. You might even have to diplomatically help him reason it out, in the middle of an aggravating situation for you, if he can just intuit that it's bad but doesn't really know how to put the argument together (which is quite likely if what he says is "that's dumb"). What if he's right? You might later give some feedback about respectful communication, but you do need to get it out of him why he's thinking the way he is. I'm an experienced software engineer, but if an intern disagrees with me, I want to know what he's thinking. He's probably wrong, but maybe he's right. Sometimes he is, I can report. I suggest you do the same. If interns can sometimes correct me, maybe they can sometimes correct you, too. Sometimes people know things you don't know, even if you know a lot. It's just the way of things. And maybe that intern is your future boss, who knows.]]]

[[[Cautionary tale: Wait, what about the customers? I once found myself in the middle of a political battle at my work place, where one group of people wanted to do a thing and another group of people wanted to do another thing. I was in neither camp because I was, seemingly, the only one who thought it was important to do both things, which everybody else ignored because they wanted to do their own thing and they didn't want to do the thing the other people wanted. In any case, someone had made an hour long presentation about how what the other camp wanted was impossible to do. After, a person several levels up from me in my organization pulls me aside and, in a whisper, asks me how convincing I found this argument. My response was a very direct "not at all convincing", because it truly wasn't. His face fell, he seemed sad and then quickly exited the conversation. A few weeks later his side lost politically and he left his position for a different thing that, as far as I know, never went anywhere. The other side then took over, over time realized that they did have to do what the side they politically defeated wanted to do, too, but by then many of the people who *wanted* to do that thing had left the team, leading to a tremendous loss of necessary expertise and now having to hire a bunch of new people to rebuild - people that were hard to find. I consider this a failure on my own part, because I let this play out without taking a sufficiently active role in getting these two groups to collaborate, which is what the project needed to have happen. That wasn't my responsibility, it was happening several levels above me, but I could have done more in service of the project, so it was wrong of me not to. But what I wanted to focus on in this cautionary tale is the failure to elicit more information from me by this person several levels up. He needed to equip himself with the knowledge of the weaknesses in his own argument, if nothing else so that

he could prepare responses to them, as well as to understand the situation better in technical terms, and I could have given him all of that, and he just didn't make use of that opportunity, and ran off instead, because what I told him wasn't initially what he wanted to hear. I think I could have saved his job - he needed to change his approach because what he was saying didn't make sense. So he got thrown out of his own place in the company. You need to understand why people disagree with you, even if in your mind this is purely a political struggle. The idea to use (possibly foolish) technical disagreements as proxies for personal and leadership conflicts and ambitions breaks projects and companies and, certainly, if you own a company, you should not tolerate it. Otherwise, the lesson being learned by the company is that the political concerns are more important than having an excellent product to sell to customers. Neither would I suggest engaging in that activity if you don't own a company. It forces you to insist on foolish inflexible ideas (you can't give ground) and people around you, who can evaluate what you are saying in technical terms, will not be able to forget it later after the smoke clears. You mark yourself as someone who cannot be relied on to follow the facts of a situation, someone who has other concerns. Hoping that your own management won't notice this is a bit optimistic, I think, even if they maybe aren't going to create a problem by saying so. It's not great, even if you win. In this particular tale, the result was very bad for both sides in the conflict, the problem for the "winning" side just took a while to become clear. In the long view, not even the winning side achieved a benefit from this conflict, but there were high costs all around. It's perfectly possible to win a political war in a way that hurts you far more than surrendering would have. I rate this situation as a disaster for *everyone*, "winners" and losers alike, and myself included, because it degraded the quality of the overall environment that I existed within. The knock-on effects of this led me to seek employment elsewhere. In hindsight, I might have taken it as a cue to take more control of my environment, though that's not what I did. I would have had to become a part-time politician to do that. Perhaps that is what I should have done. **]]]**

The common thread in these cautionary tales is leadership that insists on something that turns out, for technical reasons that they are unaware of in that moment, to be wrong. And then, in the latter two cases they *can* course correct but they don't try that approach, with disastrous results.

[[[Cautionary tale: Engineers read contracts Be careful what you put into your contracts. I wanted to work at OpenAI, above other companies, but I ended up declining their offer purely because of the terms of their contract. If you get an offer from OpenAI, I'd suggest making sure you understand what their contract says, which might require you to hire a lawyer. Perhaps they've improved the terms since I applied there - it was a long time ago. I should hope so. You should listen to what a company tells you when they think you aren't paying attention. That's what all employment contracts are. They don't think that you will read it. **]]]**

So how should you talk to engineers? You go about talking to engineers the same way you go about talking to anyone else. Respect their expertise, whatever it is. Listen to what they say. Spend the time to understand what's going on as best you can. You can override your employee's decisions when necessary, but listen well when you do it, do it respectfully and be careful to explain well why you are doing it. Which is a fine way to deal with anyone, I think.

Objections and answers

I have received feedback from some of the people who have read this document. I've reworded this feedback as objections in this section, where I also answer those objections.

OS interruption of the systolic arrays

If you run Linux on an AI accelerator, won't you have problems like OS interrupts interrupting the chip, leading to unpredictable idle cycles on the big expensive systolic arrays and other such variances?

Source: [Sean Silva](#)

Answer: The proposal is to have several cores per chip, only some of which have an attached systolic array. In this way, interrupts and other OS busywork can be restricted to the non-systolic-array-bearing core(s) while AI processing is taking place.

Few have access to the kind of CPU required for this

Nobody who wants to pursue an AI CPU accelerator in earnest owns modifiable IP for a high-enough-performance CPU capable of running Linux. The ones who have that, like Amd, Intel and Apple, are going for "a bit better CPU inference", not trying to stand up to Nvidia GPUs in the market.

Source: [Sean Silva](#)

Answer: I agree that Amd, Intel and Apple, and perhaps some other companies, are in a good position to do what is proposed here and I also agree that they do not appear to have attempted it so far. Google would be in a position to make an Axion-based future TPU along these lines, but that hasn't happened either. For this plan, I think that access to an existing high-end CPU is a definite plus, but not required. Starting from an existing high-performance CPU could even be a detriment if the existing CPU you start from is ill-fitted in some dimension to the requirements and you are unwilling or unable to modify it to correct that.

Also, these AI devices do not have to do that much scalar work, so it is not required to supply a high-powered CPU for this. If you want to do heavy lifting in pure Python, then you need a lot of scalar (as opposed to vector, not in the sense of "superscalar") capacity on the associated CPUs. The design proposed in this document assumes that the code written for the device isn't doing any silly things like that. You can still use Python, but not for an inner loop or anything heavy like that. This *is* a legitimate usability point since naive AI practitioners do in fact write silly code like that some of the time. In fact, in some cases, no one could ever supply a CPU that is fast enough to support some of the input pipelines that some AI practitioners write at first, until they learn better ways of doing things.

An exception here might be video decompression as part of an input pipeline (or potentially output pipeline, if you are generating video). Unless you are AMD, Intel or a similar company

that has access to high performance cores, that may be too heavy weight of a computation for your custom CPU core(s) to handle in real time. You may need to add specific off-the-shelf support for this to the hardware or require use of a light weight or an especially vector-compute-friendly video codec. A factor that helps here is that processing video can also be very heavy on the AI side and the input pipeline decompression does not have to be faster than the AI can process the video output from that. I do not know of a reasonable workload for the input pipeline that is heavier than video decoding and that has to run on the device. If there would be heavy scalar compute required, which I do not believe is the common case, then you would often be able to do that on a computer elsewhere, but for video decompression specifically, this solution is not ideal since uncompressed video requires a tremendous amount of bandwidth.

As is well known, TPUs use a Very Long Instruction Word (VLIW) instruction set. This reduces the cost in chip area and power per scalar core. This fits AI applications well as there are few branches in AI code and the ones that exist tend not to be data dependent. So everything is very regular. If you should decide to make your own CPU, this is perhaps something to consider.

If you cannot make an on-chip CPU work as a replacement for the host, I would still suggest to put some simple additional cores on your AI chip anyway. In the networking section, you will notice that I suggest overlapping the use of the network with running compute kernels. You will have an easier time being flexible on this arrangement if you have the option of running the network algorithms on different kernels entirely, so that you do not bother the important cores that need to be focused on keeping the systolic arrays busy. An underpowered core in this scenario may need to do nothing more than adding some numbers and enqueueing DMAs. Another option is to let the systolic arrays operate directly on data from DMAs, in some cases freeing the important cores up for other work, since otherwise the core needs to feed new work to the systolic array every few cycles - with a DMA, that can instead be every few thousands of cycles - though op fusion into GEMM ops challenges and may defeat this simplification.

If you do end up provisioning a separate host computer, keep in mind how much performance your approach gets out of a single chip. Fixed costs of development are usually larger for higher per-chip performance, which means that large companies like Google and Nvidia may have an easier time building larger chips with more per-chip performance. The good news is that this doesn't matter for what you can deliver - it is total system performance that matters. The bad news is that this does mean that you will incur a higher multiple of per-chip economic overheads, like host computers per chip (the ratio is usually several AI chips to one host), to make an equivalent system. So you may need to be more cost-conscious on these overheads, like a host computer, than they need to be. I think they should be more cost-conscious, as well, but the need for you to be cost-conscious on this may be even more pressing than for them.

The science is missing

Some of the recommendations here could perhaps better be reworded as experiments to conduct to figure out if something is possible, instead of direct statements.

Source: [Sean Silva](#)

Answer: I agree that these are my opinions. They are *informed* opinions and I think that they are *true* opinions, but yes, someone making an AI chip absolutely ought to verify for themselves rather than just take my word for it. The understanding conveyed through this document is what would have been my starting point for my would-be company. That understanding and plan would surely have changed as the work progressed in ways I cannot predict. Also, an approach that may have been best for me in my situation may not be best for someone else in their situation.

This design may not be so generally applicable

When reading this document, the impression is that this one design will be good for everything, but with e.g. the advent of disaggregated prefill/decode, it seems unlikely that any one specific design will scale to all use cases.

Source: [Sean Silva](#)

Answer: If we assume that AI will continue to be based on large matmuls, then I think that this design is generally appropriate for most AI applications. The primary exception I can think of is very low power edge AI, like a motion detector running on a tiny battery - this design assumes a certain scale above that so that you can power a CPU, and so wouldn't be appropriate for that application. For a smartphone, I think this design is still appropriate, though of course you'll want to scale some things down in that case, e.g. you likely won't have a torus network in that case. The fundamental argument in this document is that systolic arrays become more superior the larger they get. On an edge chip, the systolic arrays will be smaller than for a datacenter chip as the vector width (and possibly batch) of models you run there is smaller, so from the time that systolic arrays are the only way to go in the datacenter, there will be a delay until the same thing is true for mobile.

I certainly believe that this design can serve as both prefill and decode as-is. Though if you wanted to do a prefill-only or decode-only chip, all you need to do is to adjust some constants like HBM capacity, HBM bandwidth and vector width / systolic array dimensions. I personally like the option of *managed* aggregation (see the section on that) as an alternative solution to disaggregation, avoiding the need for distinct prefill / decode hardware while still achieving high utilization of both memory bandwidth and systolic arrays, but it will be interesting to see where the market goes on this in the future. A strong deployment of GQA plus speculative decode is also a force pushing against the need for disaggregation, since it decreases the difference between prefill and decode - see the section on decode for details on this.

Tokens per dollar isn't everything

This document champions tokens per dollar as the most important metric, but there are other things that matter. E.g. for an AI assistant that writes code, users may much prefer to pay twice as much per token if that means that the AI assistant produces better code.

Source: [Rama Govindaraju](#)

I agree that many metrics matter for an AI assistant and token quality is certainly one of them. I do not get into the economics of an AI assistant here as the document is focused on AI hardware and its associated software, but I think it is an interesting topic that, I believe, actually supports my focus on tokens per dollar in this document. How is that?

Suppose we are building a code-writing AI assistant called Cod (that's a fish, but it sounds like Code). Many concerns, including high-quality training data, are going to be critical for Cod, but let's focus on the implications of hardware choice for Cod inference, since that is what this document is about.

Suppose we have a choice between hardware A and hardware B. The two are roughly equivalent, but A costs half as much as B to produce the same tokens (or comparable tokens) in terms of total cost of ownership. Cod went with hardware A while their biggest competitor, Bass, went with hardware B. This gives Cod some options compared to Bass, including:

1. Cod can offer lower prices for the same tokens to their customers.
2. Cod can use twice as much computation to produce each of their tokens, delivering higher quality tokens at the same price.
3. Cod can scale up to support twice as many customers with the same initial investment.

So better economics in delivering tokens can be translated into higher quality tokens at the same price, not only lower prices or higher profit. This may force Bass to also switch to hardware A. The force of competition then, in the end, results in end-users having access to both lower prices and higher quality. In this way, more economical AI hardware gives the public access to not just affordable AI services, but also higher quality AI services.

In fact, that exact reasoning is why I wrote this document. Nobody paid me to do this. I'm not trying to help make someone who makes AI chips richer. Not that I mind making people rich, but I don't care about that enough to write this. I wrote this because I think it might help the public to have more access to both affordable and higher quality AI.

Hold on, one might say, if we use the improved economics of hardware A to produce higher-compute tokens at the same price, then tokens per dollar has not changed, so it seems like you are focusing on the wrong metric somehow. But tokens per dollar actually has improved, because it is of course "tokens per dollar" of *comparable* tokens. If you tried to produce those high-quality tokens without hardware A, then it would cost twice as much. So hardware A has halved the cost of these high-quality tokens.

I think tokens per dollar is the most important metric, but yes, there are other important metrics, too, like token latency. So should these other metrics be ignored? No, of course not. E.g. your AI chip has to be able to be powered on. That's a critical concern. By focusing on dollars per token,

I do not mean to have anyone ignore the importance of being able to power the AI chip on - and so it is with other concerns.

OK, but is it even possible to achieve a 2x improvement on economics? Yes. The AI and AI hardware industries are still immature. Nvidia is charging high prices. There is much more than 2x left to gain in AI hardware and associated AI software even without any improvement to chip lithography.

What about analog compute?

This document champions systolic arrays, which are the traditional silicon transistor way of doing matrix multiplication. But what about analog compute, like what EnCharge is doing? Or photonic computation where matrix multiplications are done using light?

Source: Various

I hope that all works out! So far these approaches are not proven to be superior to traditional systolic arrays in the market. The fortress of systolic arrays looks pretty sturdy to me, *but* if someone can replace the very idea of a digital circuit with something else, well, maybe the story changes. I don't have a background in these alternative technologies, so I can't comment much on it.

Why more FLOPs?

Larger systolic arrays do not necessarily work I think - you can check our short article on this matter here <https://arxiv.org/abs/2601.22001>, you are not necessarily bound by that FLOPs. Regarding low-precision (eg. 4-bit), check it out here <https://arxiv.org/abs/2509.09505>, where we report on both accuracy and real gains, full code (RTL+compiler+simulator) would be open sourced soon.

Source: [Yiren \(Aaron\) Zhao](#) in [this LinkedIn post](#).

Yiren is absolutely correct that if you are not compute bound, then adding more compute doesn't help - it just lowers the utilization of that compute. Though consider a larger systolic array that is clocked down (runs at a lower frequency) so that it has the same throughput as a smaller one. The larger one is still more efficient - up to a limit. So the efficiency of larger systolic arrays is independent of needing more FLOPs - up to a limit. Therefore I would not necessarily underdimension your systolic array even if you don't need more compute - in your co-design exponential search, you should include the possibility of larger clocked-down systolic arrays.

The implicit point here is that an AI chip needs to be a balanced device, which Yiren is of course also right about. Your co-design process cannot be about just maximizing compute on the chip, it needs to carefully consider the complex dependencies between systolic array compute, vector compute, scalar compute, memory bandwidth, memory capacity, network bandwidth, watts/thermals, thermal-induced-throttling, latencies of various things and many such factors.

The first paper referenced performs measurements on a specific combinations of sw, hw and workload. The paper empirically observes that some of these combinations are not compute bound. The paper concludes that other things also matter, e.g. memory bandwidth and memory capacity and there is a recommendation of disaggregation and decode/prefill-specific devices.

The first paper listed here does not take into account many of the optimizations and trade-offs that are described in this doc. This is unsurprising and perfectly understandable: if it did, it would have been years of hard work for a whole team of software engineers, similar to producing the software for a new AI chip, and doing parts of the co-design for said chip, instead of a short paper using off-the-shelf hardware and software. But it's not a bad paper to read - you need to at least understand all the points being made there, even if they may not apply to your particular project as-is.

This also gets into the central point that I started this document with: how sophisticated the *use* of your device will be is completely critical for what device you should build. If you assume perfect software, and your customers do something else, your device may not be attractive to customers. So you need to be involved to ensure that the use of your device will conform to your assumptions about the use of your device. This is a difficult area to manage.

The second paper, [Wu et al.](#), is long with a lot of interesting ideas. I don't want this document to become a general paper critique service, though there is a good point in that paper, that you can increase the K dimension of a systolic array without increasing the other dimensions. I've now added this idea, with a link to this paper, to the section on non-square systolic arrays.

Structured sparsity sucks

The suggestion in this document is to support structured sparsity. Yet I haven't heard of anyone who uses structured sparsity of NVIDIA GPUs at scale. From my experience working with structured sparsity, the accuracy drops are just way too large and the performance gains in the end are not large enough to offset those. People have tried hard to get the best out of the format (me included, see [my paper on this: A Proximal Operator for Inducing 2:4-Sparsity](#)) and the trade-off is pretty bad.

Source: [Jonas M. Kübler](#)

I'm putting the source here as Jonas, but he definitely is not alone in the industry in believing this.

This doc is admittedly not an ethnographic study of current deployment practices, but instead about what I suggest ought to be done. The best deployment will be using 2:4 is how I read the evidence. That's not to say that everyone is doing that currently, you are right about that, though Nvidia used it for MLPerf:

<https://blogs.nvidia.com/blog/tensorrt-llm-inference-mlperf/>

and Tencent is using it:

<https://www.nvidia.com/en-us/on-demand/session/gtcspring23-s51299/>

This goes back to one of the introductory paragraphs for the whole doc:

"The biggest tension you'll find in this document is between, on one side, maximizing tokens per dollar and, on the other side, grossly overprovisioning your AI chip with collectively extremely expensive and unnecessary features and capacities so that customers will receive something that resembles what they are already using and used to."

I think that lacking structured sparsity is an example of this - wasteful but familiar and easier to deploy.

Hundreds of billions, perhaps to-be trillions, are flowing into AI hardware and therefore the value of easy recipes for 2:4 structured sparsity is worth at least double digit billions of dollars to humanity, possibly \$100B+. In light of that and the fact that most AI is currently running on Nvidia GPUs, I do agree that Nvidia would benefit from investing more in providing easy-to-use recipes for 2:4 sparsity and general research in this area. Their [blog post about this](#) is nice, but clearly hasn't been enough to pull the whole industry along on this.

Suppose you are making a new AI chip and, perhaps with great difficulty, you figure out a general recipe to make 2:4 or even 1:2 structured sparsity work for transformers in general. Which I think is perfectly possible. It doesn't matter how difficult figuring that out was for you. What matters is simply that your recipe can be picked up and applied by AI practitioners without too much trouble. I think it would benefit Nvidia to do more here. They don't have an interest in doing this for 1:2 sparsity, since they don't support it, but they do for 2:4. The price to humanity of not doing this is double-digit billions of dollars, possibly \$100B+. Assuming it works at all, but I believe it does and Nvidia must think so, too, since they support it.

I don't want my document to be cause for starting a general paper review service, which it has started to become, but I did take a look at your paper. The paper has a very nice and interesting idea with the proximal gradient method! I had never considered that perfect 2:4 sparsity can be expressed as zeroes of a third degree polynomial and that this can be used to gradually introduce 2:4 sparsity.

That being said, the paper leaves out many possible directions for improvement, so I think based on this paper one can only conclude that the approach in the paper, along with the two one-shot methods that are also tried, don't work well enough to make 2:4 sparsity worth using.

In Nvidia's published basic recipe for how to get 2:4 structure sparsity to work, they suggest an end-to-end method:

<https://developer.nvidia.com/blog/structured-sparsity-in-the-nvidia-ampere-architecture-and-applications-in-search-engines/>

The paper in question here uses a matrix-local objective instead, presumably to save on computational cost versus continued end-to-end training. I think this might be part of the difference that makes Nvidia's recipe work while the results in the paper are negative.

Here are some further options that one could try that are not included in the paper (some of them also mentioned in the section in this doc on structured sparsity):

- 1) Reorder columns for better matches - the column order is arbitrary and can be reordered for inference with no runtime overhead.
- 2) Leave a few outlier columns dense - this can be done without changing the format by adding a few zero columns.
- 3) Leave a few outlier rows dense - this is a bit more complex, but not hard.
- 4) Use gradual approaches to sparsity other than (or in addition to) proximal gradient. The second method in Nvidia's blog post linked to above is one example.
- 5) Use lottery hypothesis training to induce 1:2 or 2:4 structured sparsity. This is the most expensive but probably also the most effective technique. This might be done early - you often don't need to train all the way to convergence to restart the lottery.
- 6) Modify the model hyperparameters and regularization during original training for better prunability. E.g. one might place a slightly higher regularization loss on the smaller of two (or two of four) elements.
- 7) Set the lesser one of two (or two of four) elements in a block of 4 to zero for one step some small percent of the time during training. This is just elementwise dropout, but used to a different purpose. This should better prepare the model for later pruning, but requires modifying the training recipe.
- 8) Restrict sparsity to FF matrices. It's not ideal, and would be better to avoid, but this makes some sense since most of the ops in attention are activation-activation matmuls, which are not so easily amenable to structured sparsity anyway. I have no doubt that structured sparsity can be used to good effect in attention, even for activations, but I will believe that this is tricky to figure out how to do it. So one option is to just not do it.
- 9) Use further ideas like these. I'm not a structured sparsity researcher, this is just what I came up with thinking about it a bit. Jonas' paper is a good example - I certainly didn't think of that. There must be *many* more ideas that can be used here.

I really believe that ideas such as this are more than enough to make 2:4 structured sparsity work quite well for large models. It might also be possible to push this all the way to 1:2 structured sparsity - I don't know if it is or not. Either way, getting everyone to easily be able to use this is worth many billions of dollars to humanity. Truly enormous sums are being and will be spent on AI inference and this is a multiplier on the cost effectiveness of those vast sums.

Systolic arrays are bad for depthwise convolution

Hybrid/SSM models Causal Depthwise 1D convolution that does many parallel small matrix multiplication is a performance killer for TPUs huge systolic array.

Source: [Trung Ngo](#)

Yes, that is true: You will see a few models that use lots of depthwise convolution, and those won't run as well on modern Nvidia GPUs or Google TPUs because (separable) depthwise convolution does not fit well on systolic arrays. Such models can be a good fit for underpowered devices like edge devices or non-AI CPUs without systolic arrays.

The Mamba line of models is an exception here and those models are probably what Trung is thinking of. Mamba models do include a depthwise convolution and Mamba is not for underpowered devices. However, this depthwise convolution is very small, so there is hope of doing it using vector capacity on a GPU or TPU, ideally while doing other things on the systolic arrays that they are better suited for. In this way, exceptionally small depthwise convolutions *can* run well on modern AI hardware using custom kernels. Mamba is doing something unusual here - depthwise convolutions are not a normal choice in frontier models except if targeting underpowered devices.

Regular convolutions are complex to implement well on large systolic arrays, but it can generally be done if the filters are big (deep/wide) enough. But *depthwise* convolution is another matter. Depthwise convolution is sometimes used because it requires very little compute compared to full convolution, making it suited for underpowered devices like ultra-low-power edge devices or (regular old non-AI) CPUs.

The problems with depthwise convolution include that it has low arithmetic intensity (little compute per unit of bandwidth required) and it sums together very few numbers (low K dimension). E.g. a 3x3 depthwise convolution does many $A \times 9$ times $9 \times B$ matrix multiplications, and such operations do not fit well on large systolic arrays since 9 is very small here. Depthwise convolutions are also not as flexible/powerful as regular convolutions, but on an underpowered device, they might still be a good idea since they maximize what you get out of a little bit of compute.

Another reason that an AI researcher might use a depthwise convolution is to minimize the theoretical FLOPs/token of a model. However, since depthwise convolutions do not run well on modern AI hardware, this gives a false impression of efficiency - except if targeting underpowered devices. Or, if you use a very small depthwise convolution, you might be able to get it to run well on modern AI hardware anyway if you use a custom kernel (CUDA on Nvidia GPUs, Pallas on TPUs).

Are routers superior to tori?

If I am understanding correctly, you acknowledge MoE routing suffers on a torus, consider routed topologies as an alternative but tentatively suggest they may not be preferable, and cite Google's 3D torus as evidence torus remains workable for MoE. However, you never discuss NVSwitch, which already implements a Clos-like crossbar specifically solving the MoE routing problem. Does NVSwitch change your analysis?

Source: [Aygul Galimova](#)

I used to have a long response here. I later decided to expand that response into an entire new chapter, which became what is now the chapter named “Network topologies for AI in a MoE world”. That chapter is my response to this question.

What about existing AI CPUs?

I noticed that you don't mention Arm's Scalable Matrix Extension (SME). Apple's AMX and Qualcomm's QMX have realized the idea of a CPU-integrated coprocessor for matrix multiplication, shared between multiple cores. Interestingly, Arm's SME is built around Outer Product computation to perform matrix multiplications, rather than a systolic array as used by Google's TPU and NVIDIA's Tensor Core. It seems that ARM's SME is far more flexible than a systolic array, at the cost of power and area efficiency. I am curious about your opinion on ARM's SME and, presumably, why you think the tradeoff is not worth it. Source: [Jan-Eric Schäfrich](#)

When I talk about an AI CPU, I mean a device that is still intended specifically for use to accelerate AI. So in such a scenario, you need the most efficient matrix multiplication you can get, both in area and power, and that is a systolic array, not an outer product engine. For uses not related to AI, perhaps other structures may be more appropriate, but systolic arrays are king for economical AI.

I do not go into detail with these other architectures in part because I am not so closely familiar with them as I am with Google TPUs in particular and also Nvidia GPUs. However, I am aware of them and my reason for seeing them as quite different from what I'm talking about is that they do not appear to be super optimized AI devices that can replace Google TPUs or Nvidia GPUs on a token latency or tokens per dollar basis. If AI accelerators are Formula 1 race cars, these CPUs are just fast regular cars. What I envision is AI CPUs that win Formula 1.

As far as I can gather from public documents, these chips lack the memory bandwidth and network bandwidth of top AI accelerators, and their matrix multiplication units are small at 32 x 32 or smaller. For comparison on systolic array size, Nvidia GPUs are medium-sized at 64 x 64 while recent Google TPUs are large at 256 x 256. Recall that 2X side length is 4X the size, so 256 x 256 is a whopping 256x larger than e.g. 16 x 16.

These devices are what you get if you take a regular CPU and add a bit of “AI stuff” to it. I'm envisioning designing a CPU chip from the ground up just for top-end economical AI. That's not what these chips are. **Not yet**, anyway. I look forward to seeing where these companies go with these AI CPU products in the future!

I've also been asked by others what I think of ARM's new [AI CPU](#) (“Arm AGI CPU”). I got excited about that for a few seconds, too, but then I realized that this product is just a regular CPU that they are marketing for use with AI accelerators. I don't know what makes it more suited for AI-related workloads than other CPUs other than the marketing. They intend this to be the host computer for separate AI accelerators, like e.g. an Nvidia GPU. So this is not at all what

I'm talking about in this doc - in fact I'm proposing getting rid of the host computer entirely (or *maybe* leave a super-cheap host to act as a glorified router).

ARM's CPU for this might be a fine CPU, I don't know, but the marketing seems a bit odd to me. The argument is that "agentic" requires very heavy CPU compute, with no details on "agentic", so you have to buy a really expensive host computer that they will sell you to support these heavy "agentic" workloads. I don't know why "agentic" would always be such a heavy workload on a CPU, but I think if you do have the odd workload where you do require heavy CPU compute (whether it qualifies as "agentic" or not), which seems like it would probably be rare, then you should be able to just book a regular cheap computer in Cloud to do that. Just like you normally would. Not some kind of high-priced special "agentic" thing. These "agentic" CPUs seem to me to a bit like fancy branded bottled water - it's just water. It's just a CPU. Though if anyone knows a reason that ARM's agentic CPU is uniquely suited for agentic workloads, do let me know! (and yes, I have read their marketing - it didn't answer this question). I am looking forward to seeing what ARM might do in the space of actual AI CPUs as I envision here - this product just isn't that.

Same thing with [Nvidia's Vera CPU](#). They market it for use with agentic workloads. It's just a CPU that you might use for AI workloads. It should be evaluated in relation to regular CPUs, just as you should evaluate fancy branded bottled water in relation to tap water.

If these "agentic" CPUs were so good, why aren't they marketed for general use? Tool use, which is what agentic workloads require heavy CPU capacity for, is just general computation. It's not a limited or special-purpose thing, unlike AI acceleration which requires special purpose matrix multiplication hardware. Maybe some of these "agentic" CPUs really can stand up to general purpose CPU products from Apple, AMD and Intel and then we should see them marketed and being deployed also for general purposes in the future. In that case, well that's nice. That hasn't happened so far to my knowledge, however.

Fixed function hardware like from Etched and Taalas

How do you view Etched and Taalas's fixed function approach to low cost token inference?

Source: LinkedIn DM from someone who prefers to stay anonymous

These companies produce fixed-function chips, so you buy the chip and it can only do one model and if you want to update the model, or do a different model, you must throw away your old chips and buy all new ones. In return, there is a significant benefit on power usage for calculations because the circuits on the chip can be specialized to the exact weights used. You also get a significant benefit on bandwidth for retrieving weights, because those weights are burned into the chip.

Such designs have been used in the past for fixed function devices like cameras. Here I will discuss use in a datacenter, which is not standard.

I think in the far future, we will make significant use of fixed function hardware like this. I'm skeptical of the benefits for use right now, though. When it comes to memory bandwidth, kv cache is currently the big problem for memory capacity and bandwidth and fixed function must still store and retrieve the kv cache data because it is not fixed like the weights are. Also, attention calculations multiply together two activations, there is no weight involved, so it is only for FF calculations that fixed function has a real advantage. But you still need to do attention.

You might have seen huge electricity use efficiency gains claimed for FF, like 1000x. But that will be for batch = 1 and high-bit arithmetic, like 16 or 32 bits. The numbers are not as impressive for high batch and low-bit arithmetic, like 4 bits. But you can still get a significant factor for specifically FF even in that scenario, but nothing like 1000x. The real limitation is that the price of electricity is not the most expensive part of a datacenter. Buying the chips is the most expensive part. So even if this used no electricity at all, but required you to buy new chips twice as often, that's not economical. And it only helps for FF, not attention.

There are also limits on how large the model can be, because there has to be space on the chip to burn in that many weights. You could use multiple different kinds of chips, with different subsets of the weights each, but there is also a significant cost to develop new chip manufacturing masks every time you want to buy a chip with a new model, so this will multiply that expense. Also, there is a lead time to order a new model, so a company using this approach will be behind the times by a few months on that account alone.

However, in the fullness of time, eventually, once model development settles down and everything is more mature, perhaps this will be the future, even in data centers. I don't think that time is now, though. You can imagine everything being processed first through standard layers used for everything, that change only every 5 years, and then processed further through later general layers, on non-fixed-function hardware. You can also imagine some kind of fixed general layer that is run before every other layer. This combination may be worth it later. Only for FF, not attention. But I don't think this is close enough to being relevant to do today because the development of models has not AT ALL settled down and it won't for a long time, I don't think.

There is perhaps a space for cases where an operator CANNOT get more electricity anywhere in the world, so they can run this or they can be unable to install any more hardware. Perhaps in this case fixed function wins. I would not bet a company on that, but perhaps it will succeed in some fashion based on that anyway.

AMD did buy Taalas, so perhaps they see something I'm missing. But this is my view: it's a nice idea whose time has not come yet and won't come for a long time.

Do note that for Etched and Taalas in particular, they might just be using fixed function as their gimmick. Every AI chip startup has a gimmick because investors want to see a gimmick and also journalists write about gimmicks. Gimmicks give you attention. It's possible that tomorrow

we'll see a whole new chip from these companies that isn't fixed function, or only nominally fixed function, and then nothing I say here would be relevant to that.